PurePy

Language Specification, v0.18.0

Copyright © 2026 PurePy contributors. Distributed under the MIT License.

Contents

Summary **4**

1 Conformance **9**
1.1 Conformance suite . . . 9

2 Syntax **10**
2.1 Lexical syntax . . . 10
2.2 Contexts . . . 10
2.2.1 Context operations . . . 11
2.3 Expressions and statements . . . 12
2.3.1 Type annotations . . . 12
2.3.2 Operators . . . 16
2.4 Modules and programs . . . 16

3 Well-formedness **19**
3.1 Types . . . 19
3.1.1 Subtyping . . . 20
3.2 Expressions . . . 22
3.2.1 Operators . . . 27
3.3 Statements . . . 28
3.4 Pattern matching . . . 32
3.4.1 Pattern checking . . . 32
3.4.2 Shapes and shape typing . . . 35
3.4.3 Matching . . . 36
3.5 Modules and programs . . . 38

4 Operational semantics **41**
4.1 Values and environments . . . 41
4.2 Auxiliary definitions . . . 41
4.3 Expressions and statements . . . 41
4.4 Pattern matching . . . 45
4.5 Modules and programs . . . 46

A Auxiliary functions (normative) **53**
A.1 Syntactic analyses . . . 53
A.2 Types . . . 55
A.3 Classes and modules . . . 60
A.4 Values . . . 62

B Excluded Python behaviours (informative) **67**
B.1 Mutable state . . . 67

B.2 Implicit coercions 67
B.3 Compatibility with mypy 67
B.4 Runtime errors 68

Summary

This chapter collects the notation of the specification: the metavariables (Figure S1), the judgement forms with the figure defining each (Figure S2), and the auxiliary functions with the section defining each (Figures S3 and S4).

Names and literals		
x, f, c, α	variable; idiomatically f a function, c a class and α a type parameter	§2.2
q	name of a module, class or type alias	Figure 2.2
n	number; also sequence length and exit status, both integers	Figure 2.3
w	string	Figure 2.3
ℓ	literal	Figure 2.3
$\odot$, $\ominus$	binary and unary operator	Figure 2.5
Syntax		
e	expression	Figure 2.3
p	pattern	Figure 2.3
g	comprehension qualifier	Figure 2.3
b	case of a `match`	Figure 2.2
s	statement	Figure 2.2
d	function definition	Figure 2.2
τ, σ	type	Figure 2.4
ψ	type expression, a type as written	Figure 2.4
ν	primitive type	Figure 2.4
π	type or field scheme	Figure 2.4
ι	import	Figure 2.2
ϕ	field list of a dataclass	Figure 2.2
t	top-level statement	Figure 2.2
μ	module body	Figure 2.2
u	argument: an expression, pattern or value	Chapter 2
a	argument list	Figure 2.3
Programs		
$\mathcal{M}$	program, a finite map from module names to bodies	§2.4
$\mathcal{P}$	predefined modules, each with its environment and context	§2.4
Contexts		
Γ, Δ	context, a finite map from variables to context entries	Definition 2.1
θ	context entry	Definition 2.1
C, D	class, a fully-qualified class name; also the type of its instances	Definition 2.2
Σ	class table, a finite map from classes to class entries	Definition 2.3
R	static outcome of a statement	§3.3
ζ	optional return type, τ or ε	Figure 3.10
Shapes		
k, l	shape, denoting values of a type, idiomatically l residual	Figure 3.15
H	finite set of heads excluded by a shape	Figure 3.15
h	head, a literal, class or list of length n	Figure 3.15
β	bindings of a dictionary shape, a finite map from keys to shapes	Figure 3.15
K, L	ordered sets of shapes, idiomatically K matched and L residual	Figure 3.16
Values and results		
v	value	Figure 4.1
ρ	environment, a finite map from variables to values	Figure 4.1
δ	dictionary entries	Figure 4.1
r	result of statement evaluation	Figure 4.1
o	outcome of statement evaluation	Figure 4.1
r	result of expression evaluation	Figure 4.1
m	match result	Figure 4.1
κ	termination kind	Figure 4.1
f	meaning of a primitive, a partial map from a sequence of values to a result	Figure 4.1

Figure S1: Metavariables used throughout specification

Judgement	Reading	Defined in
Well-formedness		
$\Sigma \vdash h : \tau$	head h has type τ	Figure 3.15
$\Sigma \vdash k : \tau$	shape k has type τ	Figure 3.15
$\Sigma; \Gamma \vdash k, p \rightsquigarrow_{\mathsf{h}} K, L$	shape k splits at the head tested by p, giving K and L	Figure 3.17
$\Sigma; \Gamma \vdash k, p \rightsquigarrow K, L$	pattern p matches shapes K of k, leaving L	Figure 3.16
$\Sigma \vdash C[\vec{\sigma}] \asymp \tau$	$C[\vec{\sigma}]$ is the instance of C at τ	Figure 3.14a
$\Sigma; \Gamma \vdash p \lhd \tau \dashv \Delta$	pattern p checks against type τ, binding Δ	Figure 3.14b
$\Gamma \vdash q : \theta$	name q denotes context entry θ	Figure 3.1
$\Sigma; \Gamma \vdash \psi \rhd \tau$	type expression ψ denotes type τ	Figure 3.2
$\Sigma; \Gamma \vdash e \rhd \tau$	expression e synthesises type τ	Figure 3.4
$\Sigma; \Gamma \vdash e, \vec{e} \rhd \vec{\sigma}, \sigma$	callee e at arguments $\vec{e}$ synthesises parameter types $\vec{\sigma}$ and result type σ	Figure 3.5
$\Sigma; \Gamma \vdash e \lhd \tau$	expression e checks against type τ	Figure 3.8a
$\Sigma; \Gamma \vdash \vec{g} : \Delta$	comprehension qualifiers $\vec{g}$ are well-formed and bind Δ	Figure 3.8b
$\Sigma; \Gamma; \zeta \vdash s : R$	statement s has static outcome R, with return type ζ	Figure 3.10
$\Sigma; \Gamma \vdash d \rhd \tau$	definition d synthesises type τ	Figure 3.11a
$\Sigma; \Gamma \vdash d \lhd \tau$	definition d checks against type τ	Figure 3.11b
$\Sigma; \Gamma; \zeta \vdash \tau, L, \vec{b} : S, \tau', L'$	cases $\vec{b}$ have static outcome S, transforming remaining type τ and residual L to τ' and L'	Figure 3.12
$\Sigma; q; \Gamma \vdash t : R \dashv \Sigma'$	top-level statement t of module q has static outcome R, extending Σ to Σ'	Figure 3.13
$\Sigma \vdash \Gamma, x \ \mathsf{imports}\ \theta \dashv \Sigma'$	name x imported from module signature Γ has context entry θ	Figure 3.18
$\Sigma; q \vdash \vec{\imath} : \Gamma \dashv \Sigma'$	imports $\vec{\imath}$ of module q yield context Γ	Figure 3.19
$\Sigma \vdash \theta, \vec{x} \ \mathsf{loads\text{-}as}\ \theta' \dashv \Sigma'$	loaded module θ extended along chain $\vec{x}$ of submodule names gives context entry θ'	Figure 3.19
$\Sigma \vdash_{\mathsf{load}} q : \Gamma \dashv \Sigma'$	module q statically loads with signature Γ	Figure 3.20
$\vdash \mathcal{M}$	program $\mathcal{M}$ is well-formed	Figure 3.20
Evaluation		
$\vdash v : \theta$	value v is well-formed at context entry θ	Figure 4.2
$\vdash \rho : \Gamma$	environment ρ is well-formed at context Γ	Figure 4.2
$\rho, p, v \rightsquigarrow m$	pattern p matches value v with match result m	Figure 4.12
$\rho, s \Rightarrow o$	statement s evaluates to outcome o	Figure 4.3
$q, \rho, t \Rightarrow o$	top-level statement t of module q evaluates to outcome o	Figure 4.5
$\rho, e \Rightarrow r$	expression e evaluates to result r	Figure 4.6
$\rho, \vec{e} \Rightarrow r$	expressions $\vec{e}$ evaluate, left to right, to result r	Figure 4.8
$\rho, \vec{g} \Rightarrow r$	comprehension qualifiers $\vec{g}$ evaluate to result r	Figure 4.11
$\rho, x \ \mathsf{imports}\ r$	name x imported from module environment ρ has result r	Figure 4.13
$q, \vec{\imath} \Rightarrow r$	imports $\vec{\imath}$ of module q evaluate to result r	Figure 4.14
$v, \vec{x} \ \mathsf{loads\text{-}as}\ r$	loaded module v extended along chain $\vec{x}$ of submodule names gives result r	Figure 4.14
$q \Rightarrow_{\mathsf{load}} r$	module q loads to result r	Figure 4.15
$\mathcal{M} \Rightarrow \kappa$	program $\mathcal{M}$ evaluates to termination kind κ	Figure 4.15

Figure S2: Judgement forms

Function	Result	Meaning	Section
Syntactic analyses			
assigns(t)	set of variables	variables declared or assigned in t, not descending into nested functions	§A.1
declares(t), declares($\vec{\imath}$)	sequence of variables	names declared in t, or bound by imports $\vec{\imath}$, in order and with repeats, not descending into nested functions	§A.1
scope(Y, s)	context	each variable declared or assigned in s unbound; defined iff names declared in s, names in Y and pattern variables of s pairwise distinct	§A.1
binds(p), binds($\vec{g}$), binds(s)	set of variables	variables bound by pattern p, by generators of qualifiers $\vec{g}$, or by patterns within statement s	§A.1
captures(e)	set of variables	variables of enclosing scope captured by lambdas in e	§A.1
captures($\vec{g}$)	set of variables	variables of enclosing scope captured by lambdas in qualifiers $\vec{g}$	§A.1
fv(e), fv($\vec{e}$)	set of variables	free variables of expression e, or of each expression in $\vec{e}$	§A.1
Types			
base-type(ℓ), base-type(τ)	type	base type of literal ℓ, or of type τ	§A.2
$\sigma \vee_\Sigma \tau$, $\bigvee_\Sigma \vec{\tau}$	type	larger type if comparable, and union otherwise	§A.2
$\sigma \wedge_\Sigma \tau$	type	greatest type below both	§A.2
subterms(τ), subterms($\vec{\tau}$)	set of types	subterms of τ, or of each type in $\vec{\tau}$, other than unions	§A.2
type-args$_\Sigma$(σ, τ), type-args$_\Sigma$($\vec{\sigma}$, $\vec{\tau}$)	context binding type variables at types	type arguments from one-way unification of σ with τ, or of $\vec{\sigma}$ with $\vec{\tau}$	§A.2
elem-type$_\Sigma$(τ)	type	type of elements drawn from τ by generator	§A.2
shapes$_\Sigma$(τ, H), shapes$_\Sigma$($\vec{\tau}$)	ordered set of shapes or of shape sequences	shapes of τ once heads in H are excluded; sequences of shapes of $\vec{\tau}$	§A.2
shape-type(k)	type	least type of shape k	§A.2
matched-as$_\Sigma$(Γ, τ, p)	type	type τ is matched as by pattern p	§A.2
remaining$_\Sigma$(Γ, τ, p)	type	type remaining from τ if pattern p fails to match	§A.2
resolve-op$_\Sigma$($\odot$, σ_1, σ_2), resolve-op$_\Sigma$($\ominus$, σ)	type	result type of most specific overload at the operand types	§A.2

Figure S3: Auxiliary functions: syntactic analyses and types

Function	Result	Meaning	Section
Classes and modules			
submods(q)	context	stub for each submodule of q	§A.3
containing(q, q')	sequence of module names	containing modules of q' other than q and modules containing q, in prefix order	§A.3
own-fields(ϕ)	sequence of bindings	fields declared by field list ϕ, with their type expressions	§A.3
fields$_\Sigma(C)$	field scheme	fields of class C with their types, inherited first, under its type parameters	§A.3
ancestors$_\Sigma(C)$	set of classes	C and its base classes	§A.3
field-map$_\Sigma(C, \vec{u})$	map from variables	argument supplied for each field of C; defined iff arguments cover them exactly	§A.3
resolve(ρ, q)	value	value denoted by name q in ρ	§A.3
name(q)	string	string literal naming module q	§A.3
Values			
eq(v, v')	Boolean	structural equality; undefined between functions, between Boolean and number, and between two `nan` values	§A.4
eq-elems($\vec{v}, \vec{v}\,'$)	Boolean	elementwise equality of value sequences	§A.4
contains(v, v')	Boolean	whether v' is element of v	§A.4
contains-elems($\vec{v}, v$)	Boolean	whether $\vec{v}$ has element equal to v	§A.4
elems(v)	sequence of values	elements of list, tuple or string	§A.4
iter(v)	sequence of values	sequence drawn from v by generator; keys if v is dictionary, $0, \ldots, n-1$ if v is range with stop n	§A.4
getitem(v, v')	value or abort	subscripting; IndexError for index out of range, KeyError for missing key, TypeError otherwise	§A.4
result(o)	value or abort	statement outcome as result: returned value, `None` if statement completes without returning, abort unchanged	§A.4
dispatch($\rho, v, \vec{p}, \vec{s}$)	bindings and statement	case taken by `match`	§A.4
update(δ, δ')	dictionary entries	δ updated by δ' in order, later replacing earlier	§A.4
setitem(δ, w, v)	dictionary entries	δ with w bound to v, replacing earlier binding	§A.4

Figure S4: Auxiliary functions: classes, modules and values

1 Conformance

This document defines PurePy version 0.18.0. PurePy is a subset of Python: every well-formed program is a syntactically valid Python program, and if PurePy gives a run a result, that run produces the same result under Python. Python means Python 3.12 throughout. Chapter 2 gives the abstract syntax; the lexical syntax is that of Python (§2.1). Chapter 3 defines well-formedness, the static part of the definition, and Chapter 4 the operational semantics, which is defined for well-formed programs. A run is *valid* when the semantics assigns it a result; any other run reaches an operation left undefined by the semantics.

Well-formedness is a property of the program, decided by PurePy's static checks. Semantic validity is a property of a run rather than a program, because some of the Python behaviours excluded by PurePy cannot be rejected statically (summarised in the informative Annex B). An implementation conforms if it accepts exactly the well-formed programs and, on every valid run, produces the Python result; on any other run it may abort at the dynamically excluded operation, or simply produce the Python result. The conformance suite (§1.1) organises its tests by verdict: semantically valid PurePy, excluded by PurePy (syntactically, statically or dynamically), or failing under Python.

1.1 Conformance suite

The conformance suite pairs each test with the verdict given by Python itself, and runs on CPython, GraalPy, and Pyodide. [**TODO:** Diagram showing the various kinds of test.] [**TODO:** Reference interpreter, needed to detect dynamically excluded operations (#64).]

2 Syntax

The syntax of PurePy is given in Figures 2.2, 2.3 and 2.4. Figure S1 summarises the metavariables used throughout. A sequence $\vec{x}$ denotes $x_1, \ldots, x_n$ for some n; inside the brackets of a tuple, list, type application or argument list it stands for the comma-separated elements, so $(\vec{e})$ is $(e_1, \ldots, e_n)$. The sequence $x \cdot \vec{x}$ has head x and tail $\vec{x}$, and $\vec{x} +\!\!+ \vec{y}$ is concatenation. Comma-separated elements inside set braces are pairwise distinct, so $\{x_1, \ldots, x_n\}$ has n elements; the set of the elements of a sequence, $\{\vec{x}\}$, assumes no distinctness. A sequence comprehension $(x_i \mid i \in I)$, where the finite index set I carries a total order, denotes the sequence of the x_i in that order. Two sequences of equal length paired by a colon, $\vec{x} : \vec{\psi}$, denote the sequence of pairs $x_1 : \psi_1, \ldots, x_n : \psi_n$. A judgement applied to sequences of equal length holds pointwise, so that $\Sigma; \Gamma \vdash \vec{\psi} \rhd \vec{\tau}$ abbreviates $\Sigma; \Gamma \vdash \psi_i \rhd \tau_i$ for each i. When the order of its elements matters, a set $\{x_1, \ldots, x_n\}$ is *ordered* by position. A comprehension over an ordered set preserves the order of its source; $X \setminus Y$ keeps the surviving elements of X in their order; $X \uplus Y$, defined when X and Y are disjoint, appends the elements of Y to those of X; $X \cup Y$ appends the elements of Y not in X; and a product of ordered sets is ordered lexicographically.

2.1 Lexical syntax

The grammar of PurePy is an abstract syntax, and its metavariables range over values rather than over source text: n is a number and w is a string, not the lexemes that denote them. Python writes a negative number as unary minus applied to a number literal; the translation to abstract syntax folds the minus into the literal, so a number may be negative. The translation of source text into abstract syntax otherwise follows Python, including the indentation that delimits blocks. A list of type parameters or type arguments is always present in the abstract syntax and may be empty. In source an empty list is always omitted, along with its brackets, since Python's concrete syntax requires at least one element between brackets; the empty tuple type, for example, is written `tuple[()]`.

2.2 Contexts

Metavariables x, f, c and α range over *variables*; idiomatically f is used when the variable is expected to denote a function, c when it is expected to denote a class, and α when it is a type parameter. A *name* q is a non-empty sequence of variables separated by dots (Figure 2.2): a variable is a *simple name*, and a name with at least one dot is *qualified*. A name is *fully qualified* when it is read from the root of the module hierarchy; modules (§2.4) and classes (Definition 2.2) are identified by fully-qualified name.

Definition 2.1 (Context)**.** A context Γ (or Δ) is a finite map from variables to *context entries* θ, which take the following forms:

Entry	Form	Records
variable	τ, $\mathsf{DU}(\tau)$, $\mathsf{PU}(\tau)$ or $\mathsf{Unbound}$	type of definitely assigned variable; $\mathsf{DU}(\tau)$ if declared at τ and definitely unassigned; $\mathsf{PU}(\tau)$ if declared at τ and possibly unassigned; $\mathsf{Unbound}$ if its declaration has not been reached
module stub	$\mathsf{Mod}(q)$	module q, named but not loaded
module reference	$\mathsf{Mod}(q, \Gamma)$	module q with signature Γ, bound by an import
class entry	$\mathsf{Type}(C)$	name denotes class C
type parameter	$\mathsf{TypeVar}$	name is a type parameter in scope
type alias	$\mathsf{TypeAlias}(\vec{\alpha}, \tau)$	name denotes type τ with type parameters $\vec{\alpha}$
type scheme	$\forall\vec{\alpha}.\,\tau$ or $\mathsf{PU}(\forall\vec{\alpha}.\,\tau)$	type of function bound by `def`, generalised over one or more type parameters $\vec{\alpha}$; $\mathsf{PU}(\forall\vec{\alpha}.\,\tau)$ if possibly unassigned
predefined name	Predef	name usable only in annotation or as decorator

A predefined name is a member of a predefined module whose meaning is fixed by the specification, such as `list` or `dataclass`; its entry records only that it is in scope, and it does not denote a value, so it cannot appear in an expression. A type parameter or type alias denotes a type rather than a value, so it too can be mentioned only by a type expression. The entries $\mathsf{Type}(C)$, $\mathsf{TypeVar}$ and $\mathsf{TypeAlias}(\vec{\alpha}, \tau)$ are named after the Python classes `type`, `typing.TypeVar` and `typing.TypeAliasType` of the objects denoted by a class name, type parameter or alias at run time.

The definite-assignment terminology is borrowed from Java [5, §16]: a variable bound to a type is *definitely assigned*. A variable with entry $\mathsf{DU}(\tau)$ is declared at τ and *definitely unassigned*, like a blank final in Java before its assignment. A variable with entry $\mathsf{PU}(\tau)$ is declared at τ and *possibly unassigned*, because it is assigned on some paths but not on all. A function name with entry $\mathsf{PU}(\forall\vec{\alpha}.\,\tau)$ is bound at the type scheme $\forall\vec{\alpha}.\,\tau$ on some paths but not on all. A variable with entry $\mathsf{Unbound}$ is declared later in its scope, which Python reports at runtime as an unbound local. It is useful to represent contexts as total functions on the set of all variables by extending the codomain with $\bot$ to indicate *undefined*. We write $\Gamma, x : \theta$ for the context that sends x to θ and otherwise agrees with Γ, define $\mathrm{dom}(\Gamma) = \{x \mid \Gamma(x) \neq \bot\}$, and write $\emptyset$ for the context in which all variables are undefined.

Definition 2.2 (Class)**.** A class C is a fully-qualified name $q.c$: a module q and a class name c declared in q. A module declares each class name at most once, so the fully-qualified name of a class is the identity of the class. A class is also the type of its instances.

Definition 2.3 (Class table)**.** A class table Σ is a finite map from classes to class table entries, with $\Sigma(C)$ of the form $(\vec{\alpha};\ \vec{x} : \vec{\tau};\ D[\vec{\sigma}])$: the type parameters $\vec{\alpha}$ of C, its distinct fields $\vec{x}$ with their declared types $\vec{\tau}$, and its instantiated base class $D[\vec{\sigma}]$, or $\bot$ if it has none; the field types and the type arguments of the base may mention $\vec{\alpha}$. Every class named in Σ is in $\mathrm{dom}(\Sigma)$, and the relation taking each class of Σ to its base class D is acyclic.

The class table is similar to Featherweight Java's [6]; the well-formedness rules build the table as they check the class declarations of a program (Figure 3.13).

2.2.1 Context operations

Contexts admit a non-commutative monoid of sequential composition $\Gamma \otimes \Delta$ ("override") and an idempotent commutative semigroup of parallel composition $\oplus$ ("merge"). Together these form a *right near-semiring* [13]: $\otimes$ right-distributes over $\oplus$, but left distributivity does not hold. Both are defined pointwise (Figure 2.1): override is right-biased and has $\bot$ as both left and right unit, whereas merge has no unit, since a variable bound on one side only is possibly unassigned.

$\otimes$ (override)

$$\bot \otimes \theta = \theta \otimes \bot = \theta$$
$$\theta \otimes \theta' = \theta' \quad \text{(otherwise)}$$

$\oplus$ (merge)

$$\theta \oplus \theta = \theta$$
$$\theta \oplus \theta' = \mathsf{PU}(\tau) \quad \text{for } \theta \neq \theta' \in \{\tau, \mathsf{DU}(\tau), \mathsf{PU}(\tau), \bot\}$$
$$\theta \oplus \theta' = \mathsf{PU}(\forall\vec{\alpha}.\,\tau) \quad \text{for } \theta \neq \theta' \in \{\forall\vec{\alpha}.\,\tau, \mathsf{PU}(\forall\vec{\alpha}.\,\tau), \bot\}$$

$\boxplus$ (extension)

$$\mathsf{Mod}(q,\Gamma) \boxplus \mathsf{Mod}(q,\Gamma') = \mathsf{Mod}(q,\Gamma \boxplus \Gamma')$$
$$\mathsf{Mod}(q) \boxplus \mathsf{Mod}(q,\Gamma) = \mathsf{Mod}(q,\Gamma) \boxplus \mathsf{Mod}(q) = \mathsf{Mod}(q,\Gamma)$$
$$\bot \boxplus \theta = \theta \boxplus \bot = \theta$$
$$\theta \boxplus \theta' = \theta' \quad \text{(otherwise)}$$

Figure 2.1: Operations on context entries, lifted pointwise to contexts by $(\Gamma \cdot \Delta)(x) = \Gamma(x) \cdot \Delta(x)$

A third operation, *context extension* $\Gamma \boxplus \Delta$ (also Figure 2.1), combines module references for the same module recursively, preferring loaded references to stubs, and is otherwise right-biased like override. It is used in processing an import prefix, in which two imports with the same root, such as `a.b` and `a.c`, both bind the root and must combine rather than shadow.

2.3 Expressions and statements

Statements are sequenced by juxtaposition, $s\ s'$; a function body, branch of a conditional and body of a case are statements. A *top-level statement* t is a statement, class declaration or `type` statement, also sequenced by juxtaposition; the body of a module after its imports is a top-level statement, so class declarations and `type` statements occur only at the top level of a module. A `type` statement declares a *type alias*. Function definitions, class declarations and `type` statements bind *type parameters* $\vec{\alpha}$, written in brackets after the name as in Python 3.12. A *mutual region* $d_1 \ldots d_{n+1}$ is a maximal contiguous block of `def` statements: any other statement separates mutual regions. Maximality is not enforced by the grammar in Figure 2.2; the parser is responsible for grouping consecutive `def`s into a single mutual region. The well-formedness rules treat each mutual region as one mutually-recursive binding.

An *argument list* a (Figure 2.3) is a sequence of positional arguments followed by keyword arguments $x = u$, where an argument u is an expression in a constructor call and a pattern in a constructor pattern. A constructor call may also pass type arguments to the class; a constructor pattern cannot, since Python does not permit type arguments in a class pattern. The auxiliary function field-map$_\Sigma$ (§A.3) pairs each field of the class with an argument: the positional arguments fill the leading fields in order, and a keyword argument $x = u$ fills the field x. It is defined only if every field receives exactly one argument.

2.3.1 Type annotations

Parameters, returns and dataclass fields are annotated with type expressions ψ, and an assignment may be annotated (Figure 2.4). A dataclass specifies its base class by a type expression of the form $q[\vec{\psi}]$. A *type* τ has the forms of a type expression, with an *instantiated class* $C[\vec{\tau}]$, a class applied to

Statements			
s	$::=$	`pass`	(Pass)
	$\mid$	$s\ s'$	(Seq)
	$\mid$	$x\texttt{:}\ \psi$	(Declare)
	$\mid$	$x = e$	(Assign)
	$\mid$	$x\texttt{:}\ \psi = e$	(DeclareAssign)
	$\mid$	e	(ExprStmt)
	$\mid$	`assert` e	(Assert)
	$\mid$	`assert` e, e'	(AssertMsg)
	$\mid$	`return`	(ReturnNone)
	$\mid$	`return` e	(Return)
	$\mid$	`if` $e\texttt{:}\ s$ `elif` $e_1\texttt{:}\ s_1\ \ldots$ `elif` $e_n\texttt{:}\ s_n$	(If)
	$\mid$	`if` $e\texttt{:}\ s$ `elif` $e_1\texttt{:}\ s_1\ \ldots$ `elif` $e_n\texttt{:}\ s_n$ `else:` s'	(IfElse)
	$\mid$	`match` $e\texttt{:}\ b_1\ \ldots\ b_{n+1}$	(Match)
	$\mid$	$d_1\ \ldots\ d_{n+1}$	(MutualRegion)
Top-level statements			
t	$::=$	s	(Stmt)
	$\mid$	$t\ t'$	(TopSeq)
	$\mid$	`@dataclass class` $c[\vec{\alpha}]\texttt{:}\ \phi$	(Dataclass)
	$\mid$	`@dataclass class` $c[\vec{\alpha}](q[\vec{\psi}])\texttt{:}\ \phi$	(DataclassExtend)
	$\mid$	`type` $x[\vec{\alpha}] = \psi$	(TypeAlias)
Imports			
ι	$::=$	`import` q	(Import)
	$\mid$	`from` q `import` $x_1, \ldots, x_{n+1}$	(FromImport)
Cases			
b	$::=$	`case` $p\texttt{:}\ s$	(Case)
Field lists			
ϕ	$::=$	$x_1\texttt{:}\ \psi_1\ \ldots\ x_{n+1}\texttt{:}\ \psi_{n+1}$	(FieldList)
	$\mid$	`pass`	(EmptyFieldList)
Module bodies			
μ	$::=$	$\iota_1\ \ldots\ \iota_n\ \ t$	(ModuleBody)
Names			
q	$::=$	$x_1.\cdots.x_{n+1}$	(Name)
Function definitions			
d	$::=$	`def` $f[\vec{\alpha}](x_1\texttt{:}\ \psi_1, \ldots, x_n\texttt{:}\ \psi_n)$ `->` $\psi\texttt{:}\ s$	(Def)

Figure 2.2: Modules and statements

Expressions			
e	$::=$	x	(Var)
	$\mid$	ℓ	(Literal)
	$\mid$	$e(e_1, \ldots, e_n)$	(Call)
	$\mid$	$q[\vec{\psi}](a)$	(Constr)
	$\mid$	$\{e_1 : e'_1, \ldots, e_n : e'_n\}$	(Dict)
	$\mid$	$e.x$	(Attribute)
	$\mid$	$e[e']$	(Subscript)
	$\mid$	$e \odot e'$	(BinOp)
	$\mid$	$\ominus\, e$	(UnOp)
	$\mid$	e `and` e'	(And)
	$\mid$	e `or` e'	(Or)
	$\mid$	e_1 `if` e `else` e_2	(CondExpr)
	$\mid$	$[e_1, \ldots, e_n]$	(List)
	$\mid$	$(e_1, \ldots, e_n)$	(Tuple)
	$\mid$	$[\, e\ g_1\ \ldots\ g_n]$	(ListComp)
	$\mid$	$\{e : e'\ g_1\ \ldots\ g_n\}$	(DictComp)
	$\mid$	`lambda` $x_1, \ldots, x_n : e$	(Lambda)

Comprehension qualifiers			
g	$::=$	`if` e	(Guard)
	$\mid$	`for` x `in` e	(Generator)

Literals			
ℓ	$::=$	n	(Number)
	$\mid$	w	(String)
	$\mid$	`True`	(True)
	$\mid$	`False`	(False)
	$\mid$	`None`	(None)

Argument lists			
a	$::=$	$u_1, \ldots, u_n, x_{n+1}{=}u_{n+1}, \ldots, x_{n+m}{=}u_{n+m}$	(Args)

Patterns			
p	$::=$	ℓ	(PatLiteral)
	$\mid$	x	(PatVar)
	$\mid$	_	(PatWildcard)
	$\mid$	$[p_1, \ldots, p_n]$	(PatList)
	$\mid$	$(p_1, \ldots, p_n)$	(PatTuple)
	$\mid$	$q(a)$	(PatConstr)
	$\mid$	$\{w_1 : p_1, \ldots, w_n : p_n\}$	(PatDict)
	$\mid$	p `as` x	(PatAs)

Figure 2.3: Expressions and patterns

Primitive types			
ν	$::=$	`object`	(Object)
	$\mid$	`Never`	(Never)
	$\mid$	`None`	(None)
	$\mid$	`bool`	(Bool)
	$\mid$	`int`	(Int)
	$\mid$	`float`	(Float)
	$\mid$	`str`	(Str)
	$\mid$	`Sized`	(Sized)

Type expressions			
ψ	$::=$	ν	(Primitive)
	$\mid$	`list`$[\psi]$	(List)
	$\mid$	`tuple`$[\psi_1, \ldots, \psi_n]$	(Tuple)
	$\mid$	`dict`$[$`str`$, \psi]$	(Dict)
	$\mid$	`Callable`$[[\psi_1, \ldots, \psi_n], \psi]$	(Callable)
	$\mid$	`Literal`$[\ell]$	(Literal)
	$\mid$	α	(TypeVar)
	$\mid$	$q[\psi_1, \ldots, \psi_n]$	(TypeName)
	$\mid$	$\psi \mid \psi'$	(Union)

Types			
τ	$::=$	ν	(Primitive)
	$\mid$	`list`$[\tau]$	(List)
	$\mid$	`tuple`$[\tau_1, \ldots, \tau_n]$	(Tuple)
	$\mid$	`dict`$[$`str`$, \tau]$	(Dict)
	$\mid$	`Callable`$[[\tau_1, \ldots, \tau_n], \tau]$	(Callable)
	$\mid$	`Literal`$[\ell]$	(Literal)
	$\mid$	α	(TypeVar)
	$\mid$	$C[\tau_1, \ldots, \tau_n]$	(Class)
	$\mid$	$\sigma \mid \tau$	(Union)

Schemes			
π	$::=$	$\forall\vec{\alpha}.\,\tau$	(Type scheme)
	$\mid$	$\forall\vec{\alpha}.\,\vec{x}\colon\vec{\tau}$	(Field scheme)

Figure 2.4: Type expressions, types and schemes

type arguments and written C when there are no arguments, in place of $q[\vec{\psi}]$. Resolution (§3.1) takes $q[\vec{\psi}]$ to $C[\vec{\tau}]$ when q denotes the class C, and to the instantiation of the alias when q denotes a type alias. The substitution $[\vec{\tau}/\vec{\alpha}]\sigma$ replaces each occurrence of α_i in σ by τ_i, a plain replacement, since every type variable in a type is free, and is defined only if $\vec{\tau}$ and $\vec{\alpha}$ have the same length. A *scheme* π (Figure 2.4) binds type parameters $\vec{\alpha}$ in its body. The body of a *type scheme* $\forall\vec{\alpha}.\,\tau$ is a type; the body of a *field scheme* $\forall\vec{\alpha}.\,\vec{x}\colon\vec{\tau}$ is a sequence of fields $\vec{x}$ with their types $\vec{\tau}$. The *instantiation* of π at $\vec{\sigma}$, written $\pi[\vec{\sigma}]$, substitutes $\vec{\sigma}$ for $\vec{\alpha}$ in the body: $[\vec{\sigma}/\vec{\alpha}]\tau$ for a type scheme and $\vec{x}\colon[\vec{\sigma}/\vec{\alpha}]\vec{\tau}$ for a field scheme. A scheme without type parameters is its body, with the $\forall\vec{\alpha}$ omitted. The fields of a class (§A.3) form a field scheme.

2.3.2 Operators

The metavariable $\odot$ ranges over the supported binary operators other than `and` and `or`, and $\ominus$ over the supported unary operators, both enumerated in Figure 2.5. The operators `and` and `or` are distinct expression forms, since they need not evaluate both operands. The grammar of Figure 2.3 is intentionally ambiguous on operator forms; disambiguation is by the precedence and associativity given in Figure 2.5, which follow Python.

Level	Operators	Arity	Associativity
(loosest) 1	`or`	binary	left
2	`and`	binary	left
3	`not`	unary	—
4	`==` `!=` `<` `<=` `>` `>=` `in` `not in`	binary	left
5	`+` `-`	binary	left
6	`*` `/` `//` `%`	binary	left
7	`+` `-`	unary	—
(tightest) 8	`**`	binary	right

Figure 2.5: Operator precedence

2.4 Modules and programs

A PurePy *program* is a finite map $\mathcal{M}$ from *fully-qualified module names* (Figure 2.2) to *module bodies* μ, of which one is a distinguished *main module* named `__main__`; here name(q) is the string literal naming q, so name(`__main__`) = `"__main__"`. We identify a module with its fully-qualified name, writing "module q" for the module named q. A module body is a sequence of imports $\vec{\imath}$, its *import prefix*, followed by a top-level statement t; since imports are not statements, they can appear neither within a statement nor after it. The statement begins with the declaration `__name__: str` = name(q), which binds the name of the module when the module loads (§3.5).

A second map $\mathcal{P}$ gives the predefined modules of Figure 2.7, which have no body: $\mathcal{P}$ takes each such name to its environment and context (Definition 2.4). The two domains are disjoint, and their union is prefix-closed: the parent of any module in $\mathrm{dom}(\mathcal{M}) \cup \mathrm{dom}(\mathcal{P})$ is itself in $\mathrm{dom}(\mathcal{M}) \cup \mathrm{dom}(\mathcal{P})$.[1] The maps $\mathcal{M}$ and $\mathcal{P}$ are fixed and ambient in all definitions that follow.

Definition 2.4 (Predefined contexts and environments)**.** $\mathcal{P}$ takes each module q of Figure 2.7 to a pair (Γ, ρ), where Γ binds `__name__` at `str` and each member of q at the context entry listed in Figure 2.6, and ρ binds `__name__` to name(q) and each member to the corresponding value (Figure 2.7). A callable type fixes the number of arguments, so `sys.exit` requires a status argument. These signatures name types that are themselves members, sometimes of another predefined module, so each is well-formed in the union of the predefined signatures. The class table $\Sigma_{\mathcal{P}}$ of the predefined modules has the single entry $\Sigma_{\mathcal{P}}$(`builtins.range`) = (ε; `stop: int`; $\bot$).

[1] A source tree encodes such an $\mathcal{M}$: its modules are the `.py` files and the directories, each named by its path, so the domain is prefix-closed by construction. The statement of a module is the statement sequence of its `.py` file, or of its `__init__.py` for a directory, preceded by the `__name__` assignment; a directory without an `__init__.py` (a namespace package, PEP 420, https://peps.python.org/pep-0420/) contributes that assignment alone.

Module	Member	Entry
builtins	print	Callable[[object], None]
	len	Callable[[Sized], int]
	range	Type(builtins.range)
	None	Predef
	object	Predef
	bool	Predef
	int	Predef
	float	Predef
	str	Predef
	list	Predef
	dict	Predef
	tuple	Predef
math	pi	float
	e	float
	sqrt	Callable[[float], float]
	exp	Callable[[float], float]
	log	Callable[[float], float]
	sin	Callable[[float], float]
	cos	Callable[[float], float]
	tan	Callable[[float], float]
	floor	Callable[[float], int]
	ceil	Callable[[float], int]
sys	argv	list[str]
	exit	Callable[[int], Never]
typing	Callable	Predef
	Literal	Predef
	Never	Predef
	Sized	Predef
dataclasses	dataclass	Predef

Figure 2.6: Signatures of predefined modules

Write $q \preceq q'$ (*prefix*) when $q' = q$ or $q' = q.q^\dagger$ for some $q^\dagger$, and $\prec$ for the strict order; the proper prefixes of a module name are its *containing modules*, and it is *contained* in each of them. The metafunctions on modules and classes are defined in §A.3.

Module	Member	Value	
builtins	print		
	len	Prim(f) where	
		$f(v) =$ val n	if iter$(v) = u_1 \cdot \ldots \cdot u_n$
		$f(\vec{u}) =$ aborts TypeError	otherwise
	range	Type(builtins.range)	
	None	Opaque(builtins.None)	
	object	Opaque(builtins.object)	
	bool	Opaque(builtins.bool)	
	int	Opaque(builtins.int)	
	float	Opaque(builtins.float)	
	str	Opaque(builtins.str)	
	list	Opaque(builtins.list)	
	dict	Opaque(builtins.dict)	
	tuple	Opaque(builtins.tuple)	
math	pi		
	e		
	sqrt		
	exp		
	log		
	sin		
	cos		
	tan		
	floor		
	ceil		
sys	argv		
	exit	Prim(f) where	
		$f(n) =$ aborts SystemExit(n)	
typing	Callable	Opaque(typing.Callable)	
	Literal	Opaque(typing.Literal)	
	Never	Opaque(typing.Never)	
	Sized	Opaque(typing.Sized)	
dataclasses	dataclass	Opaque(dataclasses.dataclass)	

Figure 2.7: Predefined modules and the values of their members

3 Well-formedness

PurePy excludes some Python behaviours statically and others only at runtime. Well-formedness is the static part, a judgement on programs defined in this chapter; the operational semantics (Chapter 4) is defined for well-formed programs. A program is well-formed when it is well typed, so the rules of this chapter give each expression a type, each statement a static outcome recording either the assignments made or the fact that the statement definitely returns, and each module a signature. The chapter introduces types and subtyping, then follows the syntax of Chapter 2, taking statements, expressions and patterns, then modules and programs, and Chapter 4 gives each of these a dynamic counterpart.

The rules for variables serve two aims: that a program has no mutable state, so a variable never changes value, and that a well-typed program is also well typed under mypy (the restrictions this imposes are collected in §B.3). Within these limits the rules permit as much as possible. For the first aim, a variable is introduced by a declaration, parameter, import, pattern or generator, and is assigned at most once, either in its declaration or afterwards; an assignment never introduces a variable nor reassigns one. For the second, a scope is the body of a module, function, lambda or comprehension, as in Python, and the builtins enclose every module; a scope declares a name at most once and may shadow a name of an enclosing scope, hiding it throughout the scope.

Four syntactic analyses support the well-formedness rules: assigns(t), the set of variables declared or assigned anywhere in t (without descending into nested function definitions, which have their own scope, so that assigns(t) = dom(Δ) when t : Assigns(Δ)); declares(t), the sequence of names declared in t, with repeats, so that a scope can require its declarations to be distinct; binds(p), the set of variables introduced by a pattern p, and likewise binds($\vec{g}$) by the generators of a qualifier sequence and binds(s) by the patterns within a statement; and captures(e), the set of variables from the enclosing scope that are captured by lambdas within expression e, and likewise captures($\vec{g}$) within a qualifier sequence. All are defined in §A.1.

3.1 Types

$\Gamma \vdash q : \theta$

simple-var
$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}$$

simple-module
$$\frac{x : \mathsf{Mod}(q, \Gamma') \in \Gamma}{\Gamma \vdash x : \mathsf{Mod}(q, \Gamma')}$$

simple-class
$$\frac{c : \mathsf{Type}(C) \in \Gamma}{\Gamma \vdash c : \mathsf{Type}(C)}$$

simple-alias
$$\frac{x : \mathsf{TypeAlias}(\vec{\alpha}, \tau) \in \Gamma}{\Gamma \vdash x : \mathsf{TypeAlias}(\vec{\alpha}, \tau)}$$

simple-scheme
$$\frac{x : \forall \vec{\alpha}.\, \tau \in \Gamma \qquad |\vec{\alpha}| > 0}{\Gamma \vdash x : \forall \vec{\alpha}.\, \tau}$$

simple-predef
$$\frac{x : \mathsf{Predef} \in \Gamma}{\Gamma \vdash x : \mathsf{Predef}}$$

qualified
$$\frac{\Gamma \vdash q : \mathsf{Mod}(q', \Gamma') \qquad \Gamma' \vdash x : \theta}{\Gamma \vdash q.x : \theta}$$

Figure 3.1: Name q resolves to context entry θ under Γ

$\Sigma; \Gamma \vdash \psi \rhd \tau$

ty-name
$$\frac{\nu : \mathsf{Predef} \in \Gamma}{\Sigma; \Gamma \vdash \nu \rhd \nu}$$

ty-var
$$\frac{\alpha : \mathsf{TypeVar} \in \Gamma}{\Sigma; \Gamma \vdash \alpha \rhd \alpha}$$

ty-class
$$\frac{\Gamma \vdash q : \mathsf{Type}(C) \qquad \Sigma; \Gamma \vdash \vec{\psi} \rhd \vec{\tau} \qquad C \text{ has } |\vec{\tau}| \text{ type parameters in } \Sigma}{\Sigma; \Gamma \vdash q[\vec{\psi}] \rhd C[\vec{\tau}]}$$

ty-alias
$$\frac{\Gamma \vdash q : \mathsf{TypeAlias}(\vec{\alpha}, \tau) \qquad \Sigma; \Gamma \vdash \vec{\psi} \rhd \vec{\sigma}}{\Sigma; \Gamma \vdash q[\vec{\psi}] \rhd [\vec{\sigma}/\vec{\alpha}]\tau}$$

ty-literal
$$\frac{\texttt{Literal} : \mathsf{Predef} \in \Gamma}{\Sigma; \Gamma \vdash \texttt{Literal}[\ell] \rhd \texttt{Literal}[\ell]}$$

ty-list
$$\frac{\texttt{list} : \mathsf{Predef} \in \Gamma \qquad \Sigma; \Gamma \vdash \psi \rhd \tau}{\Sigma; \Gamma \vdash \texttt{list}[\psi] \rhd \texttt{list}[\tau]}$$

ty-dict
$$\frac{\texttt{dict} : \mathsf{Predef} \in \Gamma \qquad \texttt{str} : \mathsf{Predef} \in \Gamma \qquad \Sigma; \Gamma \vdash \psi \rhd \tau}{\Sigma; \Gamma \vdash \texttt{dict}[\texttt{str}, \psi] \rhd \texttt{dict}[\texttt{str}, \tau]}$$

ty-tuple
$$\frac{\texttt{tuple} : \mathsf{Predef} \in \Gamma \qquad \Sigma; \Gamma \vdash \psi_i \rhd \tau_i \quad (\forall i)}{\Sigma; \Gamma \vdash \texttt{tuple}[\vec{\psi}] \rhd \texttt{tuple}[\vec{\tau}]}$$

ty-callable
$$\frac{\texttt{Callable} : \mathsf{Predef} \in \Gamma \qquad \Sigma; \Gamma \vdash \psi_i \rhd \sigma_i \quad (\forall i) \qquad \Sigma; \Gamma \vdash \psi \rhd \tau}{\Sigma; \Gamma \vdash \texttt{Callable}[[\vec{\psi}], \psi] \rhd \texttt{Callable}[[\vec{\sigma}], \tau]}$$

ty-union
$$\frac{\Sigma; \Gamma \vdash \psi \rhd \sigma \qquad \Sigma; \Gamma \vdash \psi' \rhd \tau}{\Sigma; \Gamma \vdash \psi \mid \psi' \rhd \sigma \mid \tau}$$

Figure 3.2: Type expression ψ denotes type τ under Γ and Σ

Function signatures, assignments and dataclass fields are annotated with type expressions, and a type expression denotes a type under a context and class table (Figure 3.2). Every definition carries an annotation for its parameters and return type, so the type of a definition is given rather than inferred from its body. Within a type expression, a class name applied to as many type arguments as the class has parameters resolves to an instantiated class, an alias name applied to type arguments resolves to the instantiation of its alias, type parameters in scope denote the type variables of those names, and all other names must be predefined names (Figure 3.2). The function base-type(τ) discards the precision of a literal type, taking `Literal`[42] to `int` and otherwise acting as the identity (§A.2).

3.1.1 Subtyping

Subtyping (Figure 3.3) has every type below itself, a literal type below its base type, and an instantiated class $C[\vec{\tau}]$ below every type that its base class is below, once $\vec{\tau}$ is substituted into the arguments of the base (subty-class-extend), and below $C[\vec{\sigma}]$ when $\vec{\tau}$ and $\vec{\sigma}$ are equivalent (subty-class), so that type arguments are invariant, as for lists and dictionaries; a type variable is below itself and `object` alone, with `object` above every type and `Never` below every type. A type that is not a union has one *disjunct*, the type itself; the disjuncts of a union $\sigma \mid \tau$ are the disjuncts of σ together with the disjuncts of τ. A union is above each of its disjuncts and below every type that is above all of them. A tuple type is covariant, below another of the same length when each of its components is below the corresponding component. Two types are *equivalent*, $\sigma \equiv \tau$, when each is a subtype of the other; the notations $<:$ and $\equiv$ follow Pierce [11]. Lists and dictionaries are related only when their components are equivalent, since both are invariant in mypy (§B.3). A callable type is contravariant in its parameters and covariant

in its result, following mypy. The types with a length, namely lists, dictionaries, strings, tuples and `builtins.range`, are below `Sized`. An `int` is a `float`, as PEP 484 requires, so `x: float = 0` is well typed. A `bool` is not an `int`, although Python treats it as one.

Lemma 3.1 (Transitivity of subtyping). *If* $\Sigma \vdash \sigma <: \tau$ *and* $\Sigma \vdash \tau <: \tau'$ *then* $\Sigma \vdash \sigma <: \tau'$.

Subtyping is therefore a preorder: reflexive and transitive but not antisymmetric, because distinct types such as $\sigma \mid \tau$ and $\tau \mid \sigma$ are equivalent.

Two types are *comparable* when either is a subtype of the other. Two comparable types *join* to the larger type and two incomparable types to their union. Two comparable types *meet* to the smaller type; a meet of incomparable types distributes over a union, takes tuples componentwise and callables with parameters joined and results combined using meet, and is `Never` otherwise, since no value belongs to both types. So joining a type with itself gives that type, joining a class with its base class gives the base class, and joining `int` with `str` gives `int | str`. A sequence of types is joined by folding binary join over the sequence, with `Never` as the unit (§A.2).

Lemma 3.2 (Types form a lattice). *Types ordered by subtyping in* Σ*, up to equivalence, form a bounded distributive lattice:* $\sigma \vee_\Sigma \tau$ *is the least upper bound of* σ *and* τ*,* $\sigma \wedge_\Sigma \tau$ *is their greatest lower bound,* `object` *is the greatest type and* `Never` *the least.*

$\Sigma \vdash \sigma <: \tau$

subty-refl
$$\frac{}{\Sigma \vdash \tau <: \tau}$$

subty-class-extend
$$\frac{\Sigma(C) = (\vec{\alpha};\ \vec{x}\colon \vec{\sigma}';\ D[\vec{\sigma}]) \qquad \Sigma \vdash D[[\vec{\tau}/\vec{\alpha}]\vec{\sigma}] <: \tau'}{\Sigma \vdash C[\vec{\tau}] <: \tau'}$$

subty-class
$$\frac{\Sigma \vdash \sigma_i \equiv \tau_i \quad (\forall i)}{\Sigma \vdash C[\vec{\sigma}] <: C[\vec{\tau}]}$$

subty-never
$$\frac{}{\Sigma \vdash \texttt{Never} <: \tau}$$

subty-object
$$\frac{}{\Sigma \vdash \tau <: \texttt{object}}$$

subty-int-float
$$\frac{}{\Sigma \vdash \texttt{int} <: \texttt{float}}$$

subty-sized
$$\frac{\tau \text{ a list, dictionary, string or tuple type, or } \texttt{builtins.range}}{\Sigma \vdash \tau <: \texttt{Sized}}$$

subty-list
$$\frac{\Sigma \vdash \sigma \equiv \tau}{\Sigma \vdash \texttt{list}[\sigma] <: \texttt{list}[\tau]}$$

subty-dict
$$\frac{\Sigma \vdash \sigma \equiv \tau}{\Sigma \vdash \texttt{dict}[\texttt{str}, \sigma] <: \texttt{dict}[\texttt{str}, \tau]}$$

subty-callable
$$\frac{\Sigma \vdash \tau_i <: \sigma_i \quad (\forall i) \qquad \Sigma \vdash \sigma <: \tau}{\Sigma \vdash \texttt{Callable}[[\sigma_1, \ldots, \sigma_n], \sigma] <: \texttt{Callable}[[\tau_1, \ldots, \tau_n], \tau]}$$

subty-tuple
$$\frac{\Sigma \vdash \sigma_i <: \tau_i \quad (\forall i)}{\Sigma \vdash \texttt{tuple}[\sigma_1, \ldots, \sigma_n] <: \texttt{tuple}[\tau_1, \ldots, \tau_n]}$$

subty-literal
$$\frac{\Sigma \vdash \text{base-type}(\ell) <: \tau}{\Sigma \vdash \texttt{Literal}[\ell] <: \tau}$$

subty-union-left
$$\frac{\Sigma \vdash \sigma <: \tau}{\Sigma \vdash \sigma <: \tau \mid \tau'}$$

subty-union-right
$$\frac{\Sigma \vdash \sigma <: \tau'}{\Sigma \vdash \sigma <: \tau \mid \tau'}$$

subty-union
$$\frac{\Sigma \vdash \tau <: \sigma \qquad \Sigma \vdash \tau' <: \sigma}{\Sigma \vdash \tau \mid \tau' <: \sigma}$$

Figure 3.3: Subtyping

3.2 Expressions

Within an expression the typing judgement gives each subexpression a type, and is bidirectional. *Synthesis* $\Sigma;\Gamma \vdash e \rhd \tau$ reads a type out of an expression (Figures 3.4, 3.6 and 3.7), and *checking* $\Sigma;\Gamma \vdash e \lhd \tau$ tests an expression against a type supplied by its surroundings (Figure 3.8). The division follows the usual recipe for bidirectional systems [2, 3]: introduction forms check, since a lambda's parameters cannot be annotated and an empty list does not determine its element type, and elimination forms synthesise, since a call, a subscript or an attribute reference starts from a subterm whose type is known. Type systems that follow the recipe strictly usually rely on an annotation form ($e : \tau$), which synthesises by construction, so a checking form written with an annotation can be used in any context where synthesis is required. Python has no such form, allowing annotations only on parameters, returns and assignments, so PurePy gives each introduction form a synthesis rule wherever its parts determine the type; Swift, Kotlin and Scala take similar approaches for similar reasons [1, 7, 10].

Variables, literals, attribute references, calls, constructor calls, operators and subscripts synthesise. The var rule applies whenever $\Gamma(x)$ is a type, so a variable that is not definitely assigned, or a name bound at a type scheme, does not synthesise a type. A literal ℓ synthesises the literal type `Literal`$[\ell]$.

The fields of a class C form a field scheme $\mathrm{fields}_\Sigma(C)$ over its type parameters, with one binding $x : \tau$ per field: the fields of its base class first, recursively and instantiated at the type arguments of the base, then the fields declared by C itself, each at its declared type; the fields of an instantiated class $C[\vec{\tau}]$ are the instantiation $\mathrm{fields}_\Sigma(C)[\vec{\tau}]$. The order matters, because a constructor call pairs positional arguments with fields by position. A constructor call $q(a)$ pairs each argument with a field using field-map_Σ and infers type arguments $\vec{\tau}'$ for the class C that q resolves to, using type-args_Σ (§A.2) over the arguments that synthesise, as in callee-scheme (constr). The arguments are checked against the field types instantiated at $\vec{\tau}'$, and the call synthesises $C[\vec{\tau}']$. A constructor call of the form $q[\vec{\psi}](a)$ takes the type arguments denoted by $\vec{\psi}$ instead (constr-explicit). An attribute reference $e.x$ has the type of the field x of the class synthesised by e (attr-object). The signature of a predefined module binds `__name__` at `str` and each member at the type given in Figure 2.6 (Definition 2.4). A module attribute reference $e.x$ resolves as a qualified name to a definitely assigned member (attr-module). A variable bound to a module or class is not an expression on its own, and occurs only as the head of a qualified name. In a subscript $e[e']$ there is a rule for each container that e can synthesise: a list (subscript-list) or string (subscript-str) with e' of type `int`, and a dictionary with e' of type `str` (subscript-dict). Tuples have three rules: an index that synthesises a literal type gives the component type at that position, counting from the end when the index is negative (subscript-tuple, subscript-tuple-neg), and any other index of type `int` gives the join of the component types (subscript-tuple-int). A subscript, attribute reference or call whose subject synthesises a union is typed at each disjunct, with a variable in place of the subject, and synthesises the join of the results (subscript-union, attr-union, call-union); generators need no such rule, because elem-type_Σ takes a union to the join of its disjuncts' element types. Dunfield and Pfenning [4] state union elimination once, for an arbitrary evaluation context; here union elimination is stated separately for each elimination form.

The call rule for a call $e(\vec{e})$ uses the auxiliary judgement $\Sigma;\Gamma \vdash e, \vec{e} \rhd \vec{\sigma}, \sigma$ (Figure 3.5), which determines parameter types $\vec{\sigma}$ and result type σ from the callee along with its arguments. The remaining premises check the arguments against $\vec{\sigma}$, and the call synthesises σ. The auxiliary judgement has two rules. A callee that synthesises a `Callable` type gives the parameter and result types of that type (callee). A name bound at a type scheme $\forall\vec{\alpha}.\,\texttt{Callable}[[\vec{\sigma}], \sigma]$ instantiates the type parameters from the arguments (callee-scheme). The type arguments are collected by type-args_Σ (§A.2) over the arguments that synthesise, by a one-way unification of each parameter type with the synthesised type:

$\Sigma; \Gamma \vdash e \rhd \tau$

var
$$\frac{x : \tau \in \Gamma}{\Sigma; \Gamma \vdash x \rhd \tau}$$

literal
$$\frac{}{\Sigma; \Gamma \vdash \ell \rhd \texttt{Literal}[\ell]}$$

attr-module
$$\frac{\Gamma \vdash e.x \; : \; \tau}{\Sigma; \Gamma \vdash e.x \rhd \tau}$$

attr-object
$$\frac{\Sigma; \Gamma \vdash e \rhd C[\vec{\tau}] \qquad x : \sigma \in \text{fields}_\Sigma(C)[\vec{\tau}]}{\Sigma; \Gamma \vdash e.x \rhd \sigma}$$

call
$$\frac{\Sigma; \Gamma \vdash e, \vec{e} \rhd \vec{\tau}, \tau \qquad \Sigma; \Gamma \vdash \vec{e} \lhd \vec{\tau} \qquad \tau \neq \texttt{None}}{\Sigma; \Gamma \vdash e(\vec{e}) \rhd \tau}$$

call-lambda-syn
$$\frac{\vec{x} \text{ distinct} \qquad \Sigma; \Gamma \vdash \vec{e} \rhd \vec{\tau} \qquad \Sigma; \Gamma \otimes \{\vec{x} : \vec{\tau}\} \vdash e \rhd \tau}{\Sigma; \Gamma \vdash (\texttt{lambda}\ \vec{x} : \ e)(\vec{e}) \rhd \tau}$$

constr
$$\frac{\begin{array}{c}\Gamma \vdash q \; : \; \mathsf{Type}(C) \qquad \text{fields}_\Sigma(C) = \forall \vec{\alpha}.\, \vec{x} : \vec{\sigma} \qquad \text{field-map}_\Sigma(C, a) = \{\vec{x} : \vec{e}\} \\ I = \{i \mid \Sigma; \Gamma \vdash e_i \rhd \tau_i\} \qquad \text{type-args}_\Sigma((\sigma_i \mid i \in I), (\tau_i \mid i \in I))(\vec{\alpha}) = \vec{\tau}' \qquad \Sigma; \Gamma \vdash \vec{e} \lhd [\vec{\tau}'/\vec{\alpha}]\vec{\sigma}\end{array}}{\Sigma; \Gamma \vdash q(a) \rhd C[\vec{\tau}']}$$

constr-explicit
$$\frac{\begin{array}{c}\Gamma \vdash q \; : \; \mathsf{Type}(C) \\ \text{fields}_\Sigma(C) = \forall \vec{\alpha}.\, \vec{x} : \vec{\sigma} \qquad \Sigma; \Gamma \vdash \vec{\psi} \rhd \vec{\tau} \qquad \text{field-map}_\Sigma(C, a) = \{\vec{x} : \vec{e}\} \qquad \Sigma; \Gamma \vdash \vec{e} \lhd [\vec{\tau}/\vec{\alpha}]\vec{\sigma}\end{array}}{\Sigma; \Gamma \vdash q[\vec{\psi}](a) \rhd C[\vec{\tau}]}$$

binop
$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \qquad \Sigma; \Gamma \vdash e' \rhd \sigma' \qquad \text{resolve-op}_\Sigma(\odot, \sigma, \sigma') = \tau^\dagger}{\Sigma; \Gamma \vdash e \odot e' \rhd \tau^\dagger}$$

unop
$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \qquad \ominus e \neq \texttt{-}\, n \quad (\forall n) \qquad \text{resolve-op}_\Sigma(\ominus, \sigma) = \tau^\dagger}{\Sigma; \Gamma \vdash \ominus e \rhd \tau^\dagger}$$

cond-syn
$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool} \qquad I = \{i \mid \Sigma; \Gamma \vdash e_i \rhd \sigma_i\} \neq \emptyset \qquad \Sigma; \Gamma \vdash e_j \lhd \bigvee_\Sigma(\sigma_i \mid i \in I) \quad (\forall j \notin I)}{\Sigma; \Gamma \vdash e_1 \ \texttt{if}\ e\ \texttt{else}\ e_2 \rhd \bigvee_\Sigma(\sigma_i \mid i \in I)}$$

and
$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool} \qquad \Sigma; \Gamma \vdash e' \lhd \texttt{bool}}{\Sigma; \Gamma \vdash e \ \texttt{and}\ e' \rhd \texttt{bool}}$$

or
$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool} \qquad \Sigma; \Gamma \vdash e' \lhd \texttt{bool}}{\Sigma; \Gamma \vdash e \ \texttt{or}\ e' \rhd \texttt{bool}}$$

Figure 3.4: Expression synthesis: names, literals, attributes, calls and operators

a type variable takes the type found at the same position in the synthesised type, and a type variable found at several positions takes the join of those types. Arguments that do not synthesise, such as lambdas, are left out. The parameter and result types are instantiated at the type arguments given by the context, which must bind every type parameter.

Lambdas, applied lambdas, lists, tuples, dictionaries, conditional expressions and comprehensions check (lambda, call-lambda, list, tuple, dict, cond, list-comp, dict-comp). A name bound at a type scheme $\forall\vec{\alpha}.\,\sigma$ checks against a type τ by instantiating $\vec{\alpha}$ at the type arguments collected by type-args$_\Sigma$ from σ and τ, provided the instantiated σ is below τ (var-scheme), so a function with type parameters can be passed where a `Callable` type is expected. An expression that synthesises σ checks against any supertype of σ (check-syn), which applies only when no other rule concludes $\Sigma; \Gamma \vdash e \lhd \tau$, so both

$\Sigma; \Gamma \vdash e, \vec{e} \rhd \vec{\sigma}, \sigma$

callee
$$\frac{\Sigma; \Gamma \vdash e \rhd \texttt{Callable}[[\vec{\sigma}], \sigma]}{\Sigma; \Gamma \vdash e, \vec{e} \rhd \vec{\sigma}, \sigma}$$

callee-scheme
$$\frac{\Gamma \vdash q : \forall \vec{\alpha}.\, \texttt{Callable}[[\vec{\sigma}], \sigma] \qquad I = \{i \mid \Sigma; \Gamma \vdash e_i \rhd \tau_i\} \qquad \text{type-args}_\Sigma((\sigma_i \mid i \in I), (\tau_i \mid i \in I))(\vec{\alpha}) = \vec{\tau}'}{\Sigma; \Gamma \vdash q, \vec{e} \rhd [\vec{\tau}'/\vec{\alpha}]\vec{\sigma}, [\vec{\tau}'/\vec{\alpha}]\sigma}$$

Figure 3.5: Callee e at arguments $\vec{e}$ synthesises parameter types $\vec{\sigma}$ and result type σ

$\Sigma; \Gamma \vdash e \rhd \tau$ (continued)

subscript-list
$$\frac{\Sigma; \Gamma \vdash e \rhd \texttt{list}[\tau] \qquad \Sigma; \Gamma \vdash e' \lhd \texttt{int}}{\Sigma; \Gamma \vdash e[e'] \rhd \tau}$$

subscript-str
$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \qquad \text{base-type}(\sigma) = \texttt{str} \qquad \Sigma; \Gamma \vdash e' \lhd \texttt{int}}{\Sigma; \Gamma \vdash e[e'] \rhd \texttt{str}}$$

subscript-dict
$$\frac{\Sigma; \Gamma \vdash e \rhd \texttt{dict}[\texttt{str}, \tau] \qquad \Sigma; \Gamma \vdash e' \lhd \texttt{str}}{\Sigma; \Gamma \vdash e[e'] \rhd \tau}$$

subscript-tuple
$$\frac{\Sigma; \Gamma \vdash e \rhd \texttt{tuple}[\tau_1, \ldots, \tau_m] \qquad \Sigma; \Gamma \vdash e' \rhd \texttt{Literal}[n] \qquad 0 \le n < m}{\Sigma; \Gamma \vdash e[e'] \rhd \tau_{n+1}}$$

subscript-tuple-neg
$$\frac{\Sigma; \Gamma \vdash e \rhd \texttt{tuple}[\tau_1, \ldots, \tau_m] \qquad \Sigma; \Gamma \vdash e' \rhd \texttt{Literal}[n] \qquad -m \le n < 0}{\Sigma; \Gamma \vdash e[e'] \rhd \tau_{m+n+1}}$$

subscript-tuple-int
$$\frac{\Sigma; \Gamma \vdash e \rhd \texttt{tuple}[\vec{\tau}] \qquad \Sigma; \Gamma \vdash e' \rhd \texttt{int}}{\Sigma; \Gamma \vdash e[e'] \rhd \bigvee_\Sigma \vec{\tau}}$$

subscript-union
$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \mid \sigma' \qquad \Sigma; \Gamma \otimes \{y : \sigma\} \vdash y[e'] \rhd \tau \qquad \Sigma; \Gamma \otimes \{y : \sigma'\} \vdash y[e'] \rhd \tau' \qquad y \notin \text{fv}(e')}{\Sigma; \Gamma \vdash e[e'] \rhd \tau \vee_\Sigma \tau'}$$

attr-union
$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \mid \sigma' \qquad \Sigma; \Gamma \otimes \{y : \sigma\} \vdash y.x \rhd \tau \qquad \Sigma; \Gamma \otimes \{y : \sigma'\} \vdash y.x \rhd \tau'}{\Sigma; \Gamma \vdash e.x \rhd \tau \vee_\Sigma \tau'}$$

call-union
$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \mid \sigma' \qquad \Sigma; \Gamma \otimes \{y : \sigma\} \vdash y(\vec{e}) \rhd \tau \qquad \Sigma; \Gamma \otimes \{y : \sigma'\} \vdash y(\vec{e}) \rhd \tau' \qquad y \notin \text{fv}(\vec{e})}{\Sigma; \Gamma \vdash e(\vec{e}) \rhd \tau \vee_\Sigma \tau'}$$

Figure 3.6: Expression synthesis: subscripts, and elimination at a union

$\Sigma; \Gamma \vdash e \rhd \tau$ (continued)

syn-list

$$\frac{I = \{i \mid \Sigma; \Gamma \vdash e_i \rhd \sigma_i\} \neq \emptyset \qquad \tau = \bigvee_\Sigma(\text{base-type}(\sigma_i) \mid i \in I) \qquad \Sigma; \Gamma \vdash e_j \lhd \tau \quad (\forall j \notin I)}{\Sigma; \Gamma \vdash [e_1, \ldots, e_{n+1}] \rhd \texttt{list}[\tau]}$$

syn-tuple

$$\frac{\Sigma; \Gamma \vdash e_i \rhd \tau_i \quad (\forall i)}{\Sigma; \Gamma \vdash (\vec{e}) \rhd \texttt{tuple}[\vec{\tau}]}$$

syn-dict

$$\frac{\Sigma; \Gamma \vdash e_i \lhd \texttt{str} \quad (\forall i) \qquad I = \{i \mid \Sigma; \Gamma \vdash e'_i \rhd \sigma_i\} \neq \emptyset \qquad \tau = \bigvee_\Sigma(\text{base-type}(\sigma_i) \mid i \in I) \qquad \Sigma; \Gamma \vdash e'_j \lhd \tau \quad (\forall j \notin I)}{\Sigma; \Gamma \vdash \{e_1 : e'_1, \ldots, e_{n+1} : e'_{n+1}\} \rhd \texttt{dict}[\texttt{str}, \tau]}$$

syn-list-comp

$$\frac{\Sigma; \Gamma \vdash \vec{g} : \Delta \qquad \Sigma; \Gamma \otimes \Delta \vdash e \rhd \tau \qquad \text{captures}(e) \cap \text{dom}(\Delta) = \emptyset}{\Sigma; \Gamma \vdash [\, e \ \vec{g}] \rhd \texttt{list}[\text{base-type}(\tau)]}$$

syn-dict-comp

$$\frac{\Sigma; \Gamma \vdash \vec{g} : \Delta \qquad \Sigma; \Gamma \otimes \Delta \vdash e \lhd \texttt{str} \qquad \Sigma; \Gamma \otimes \Delta \vdash e' \rhd \tau \qquad \text{captures}((e, e')) \cap \text{dom}(\Delta) = \emptyset}{\Sigma; \Gamma \vdash \{e : e' \ \vec{g}\} \rhd \texttt{dict}[\texttt{str}, \text{base-type}(\tau)]}$$

Figure 3.7: Expression synthesis: tuples, lists, dictionaries and comprehensions

judgements are syntax-directed and an expression has at most one derivation in each. The simple-scheme resolution rule (Figure 3.1) requires the type parameters to be non-empty, so callee-scheme and var-scheme do not apply to names bound at `Callable` types, although they would conclude the same as callee and check-syn.

The forms that synthesise are conditional expressions when a branch synthesises, at the join of the two types when both branches synthesise and otherwise at the synthesising branch's type, the other branch checking against it (cond-syn); tuples when every component synthesises (syn-tuple); comprehensions when the body synthesises (syn-list-comp, syn-dict-comp); and non-empty lists and dictionaries when at least one element synthesises, the remaining elements checking against the join of the base types of the synthesising elements (syn-list, syn-dict). A list, dictionary or comprehension takes the base type of the type synthesised by its elements or body, since `list` and `dict` are invariant and a literal element type would keep the container from matching any other element type. An empty list or dictionary and a lambda have no synthesis rule at all, so checking is the only way to type them. A lambda applied to arguments is the exception, since the arguments give the parameter types, and the call then synthesises the type of the body or checks the body against an expected type (call-lambda-syn, call-lambda). The parameters of a lambda must be distinct, as for a function. A non-empty list has both, and the checking rule is still needed when the expected element type is wider than the type synthesised by the elements, since `list` is invariant.

Typing a comprehension types its qualifiers, which bind the variables that can be used in the body (Figure 3.8). A generator's variable is bound at elem-type$_\Sigma$ of the type synthesised by the iterated expression (§A.2). A tuple is iterated at the join of the base types of its components. A lambda within a comprehension may not capture a generator variable, a side condition of list-comp, dict-comp and

$\Sigma; \Gamma \vdash e \lhd \tau$

check-syn

$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \qquad \Sigma \vdash \sigma <: \tau \qquad \text{no other rule concludes } \Sigma; \Gamma \vdash e \lhd \tau}{\Sigma; \Gamma \vdash e \lhd \tau}$$

lambda

$$\frac{\vec{x} \text{ distinct} \qquad \Sigma; \Gamma \otimes \{\vec{x} : \vec{\tau}\} \vdash e \lhd \tau}{\Sigma; \Gamma \vdash \texttt{lambda}\ \vec{x} :\ e \lhd \texttt{Callable}[[\vec{\tau}], \tau]}$$

var-scheme

$$\frac{\Gamma \vdash q : \forall \vec{\alpha}.\, \sigma \qquad \text{type-args}_\Sigma(\sigma, \tau)(\vec{\alpha}) = \vec{\tau}' \qquad \Sigma \vdash [\vec{\tau}'/\vec{\alpha}]\sigma <: \tau}{\Sigma; \Gamma \vdash q \lhd \tau}$$

call-lambda

$$\frac{\vec{x} \text{ distinct} \qquad \Sigma; \Gamma \vdash \vec{e} \rhd \vec{\tau} \qquad \Sigma; \Gamma \otimes \{\vec{x} : \vec{\tau}\} \vdash e \lhd \tau}{\Sigma; \Gamma \vdash (\texttt{lambda}\ \vec{x} :\ e)(\vec{e}) \lhd \tau}$$

list

$$\frac{\Sigma; \Gamma \vdash e_i \lhd \tau \quad (\forall i)}{\Sigma; \Gamma \vdash [\vec{e}] \lhd \texttt{list}[\tau]}$$

tuple

$$\frac{\Sigma; \Gamma \vdash e_i \lhd \tau_i \quad (\forall i)}{\Sigma; \Gamma \vdash (e_1, \ldots, e_n) \lhd \texttt{tuple}[\tau_1, \ldots, \tau_n]}$$

dict

$$\frac{\Sigma; \Gamma \vdash e_i \lhd \texttt{str} \quad (\forall i) \qquad \Sigma; \Gamma \vdash e'_i \lhd \tau \quad (\forall i)}{\Sigma; \Gamma \vdash \{e_1 : e'_1, \ldots, e_n : e'_n\} \lhd \texttt{dict}[\texttt{str}, \tau]}$$

cond

$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool} \qquad \Sigma; \Gamma \vdash e_1 \lhd \tau \qquad \Sigma; \Gamma \vdash e_2 \lhd \tau}{\Sigma; \Gamma \vdash e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \lhd \tau}$$

list-comp

$$\frac{\Sigma; \Gamma \vdash \vec{g} : \Delta \qquad \Sigma; \Gamma \otimes \Delta \vdash e \lhd \tau \qquad \text{captures}(e) \cap \text{dom}(\Delta) = \emptyset}{\Sigma; \Gamma \vdash [\, e\ \vec{g}] \lhd \texttt{list}[\tau]}$$

dict-comp

$$\frac{\Sigma; \Gamma \vdash \vec{g} : \Delta \qquad \Sigma; \Gamma \otimes \Delta \vdash e \lhd \texttt{str} \qquad \Sigma; \Gamma \otimes \Delta \vdash e' \lhd \tau \qquad \text{captures}((e, e')) \cap \text{dom}(\Delta) = \emptyset}{\Sigma; \Gamma \vdash \{e : e'\ \vec{g}\} \lhd \texttt{dict}[\texttt{str}, \tau]}$$

(a) Expression e checks against type τ

$\Sigma; \Gamma \vdash \vec{g} : \Delta$

quals-nil

$$\frac{}{\Sigma; \Gamma \vdash \varepsilon : \emptyset}$$

qual-guard

$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool} \qquad \Sigma; \Gamma \vdash \vec{g} : \Delta}{\Sigma; \Gamma \vdash \texttt{if}\ e \cdot \vec{g} : \Delta}$$

qual-generator

$$\frac{\Sigma; \Gamma \vdash e \rhd \tau \qquad \text{elem-type}_\Sigma(\tau) = \sigma \qquad \Sigma; \Gamma \otimes \{x : \sigma\} \vdash \vec{g} : \Delta \qquad x \notin \text{captures}(\vec{g})}{\Sigma; \Gamma \vdash \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} : \{x : \sigma\} \otimes \Delta}$$

(b) Comprehension qualifiers $\vec{g}$ are well-formed and bind Δ

Figure 3.8: Expression checking

qual-generator. Python closures are late-binding and the comprehension rebinds the variable on each iteration, so the lambda would see the variable's final value; in PurePy a lambda is only able to see the value of the generator variable at each iteration.

3.2.1 Operators

$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\tau_1, \tau_2), \tau)$

$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\sigma_1, \sigma_2), \texttt{bool})$	if $\odot \in \{\texttt{==}, \texttt{!=}\}$ and σ_1, σ_2 comparable equality types
$\Sigma \vdash \odot(\sigma_1, \texttt{list}[\tau]) : ((\sigma_1, \texttt{list}[\tau]), \texttt{bool})$	if $\odot \in \{\texttt{in}, \texttt{not in}\}$ and σ_1, τ comparable
$\Sigma \vdash \odot(\sigma_1, \texttt{tuple}[\vec{\tau}]) : ((\sigma_1, \texttt{tuple}[\vec{\tau}]), \texttt{bool})$	if $\odot \in \{\texttt{in}, \texttt{not in}\}$ and $\sigma_1, \bigvee_\Sigma \vec{\tau}$ comparable
$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\texttt{str}, \texttt{str}), \texttt{bool})$	if $\odot \in \{\texttt{in}, \texttt{not in}\}$ and $\Sigma \vdash \sigma_1, \sigma_2 <: \texttt{str}$
$\Sigma \vdash \odot(\sigma_1, \texttt{dict}[\texttt{str}, \tau]) : ((\texttt{str}, \texttt{dict}[\texttt{str}, \tau]), \texttt{bool})$	if $\odot \in \{\texttt{in}, \texttt{not in}\}$ and $\Sigma \vdash \sigma_1 <: \texttt{str}$
$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\texttt{float}, \texttt{float}), \texttt{bool})$	if $\odot \in \{\texttt{<}, \texttt{<=}, \texttt{>}, \texttt{>=}\}$ and $\Sigma \vdash \sigma_1, \sigma_2 <: \texttt{float}$
$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\texttt{str}, \texttt{str}), \texttt{bool})$	if $\odot \in \{\texttt{<}, \texttt{<=}, \texttt{>}, \texttt{>=}\}$ and $\Sigma \vdash \sigma_1, \sigma_2 <: \texttt{str}$
$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\texttt{int}, \texttt{int}), \texttt{int})$	if $\odot \in \{\texttt{+}, \texttt{-}, \texttt{*}, \texttt{//}, \texttt{\%}\}$ and $\Sigma \vdash \sigma_1, \sigma_2 <: \texttt{int}$
$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\texttt{float}, \texttt{float}), \texttt{float})$	if $\odot \in \{\texttt{+}, \texttt{-}, \texttt{*}, \texttt{//}, \texttt{\%}, \texttt{/}, \texttt{**}\}$ and $\Sigma \vdash \sigma_1, \sigma_2 <: \texttt{float}$
$\Sigma \vdash \texttt{+}(\sigma_1, \sigma_2) : ((\texttt{str}, \texttt{str}), \texttt{str})$	if $\Sigma \vdash \sigma_1, \sigma_2 <: \texttt{str}$
$\Sigma \vdash \texttt{+}(\texttt{list}[\sigma], \texttt{list}[\tau]) : ((\texttt{list}[\sigma], \texttt{list}[\tau]), \texttt{list}[\sigma \vee_\Sigma \tau])$	
$\Sigma \vdash \texttt{+}(\texttt{tuple}[\vec{\sigma}], \texttt{tuple}[\vec{\tau}]) : ((\texttt{tuple}[\vec{\sigma}], \texttt{tuple}[\vec{\tau}]), \texttt{tuple}[\vec{\sigma} +\!\!+ \vec{\tau}])$	
$\Sigma \vdash \texttt{*}(\sigma_1, \sigma_2) : ((\texttt{str}, \texttt{int}), \texttt{str})$	if $\Sigma \vdash \sigma_1 <: \texttt{str}$ and $\Sigma \vdash \sigma_2 <: \texttt{int}$
$\Sigma \vdash \texttt{*}(\sigma_1, \sigma_2) : ((\texttt{int}, \texttt{str}), \texttt{str})$	if $\Sigma \vdash \sigma_1 <: \texttt{int}$ and $\Sigma \vdash \sigma_2 <: \texttt{str}$
$\Sigma \vdash \texttt{*}(\texttt{list}[\tau], \sigma_2) : ((\texttt{list}[\tau], \texttt{int}), \texttt{list}[\tau])$	if $\Sigma \vdash \sigma_2 <: \texttt{int}$
$\Sigma \vdash \texttt{*}(\sigma_1, \texttt{list}[\tau]) : ((\texttt{int}, \texttt{list}[\tau]), \texttt{list}[\tau])$	if $\Sigma \vdash \sigma_1 <: \texttt{int}$
$\Sigma \vdash \texttt{**}(\sigma_1, \texttt{Literal}[n]) : ((\texttt{int}, \texttt{Literal}[n]), \texttt{int})$	if $\Sigma \vdash \sigma_1 <: \texttt{int}$ and n a non-negative integer
$\Sigma \vdash \texttt{**}(\sigma_1, \texttt{Literal}[n]) : ((\texttt{int}, \texttt{Literal}[n]), \texttt{float})$	if $\Sigma \vdash \sigma_1 <: \texttt{int}$ and n a negative integer
$\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\tau_1, \tau_2), \tau)$	if $\Sigma \vdash \odot(\text{base-type}(\sigma_1), \text{base-type}(\sigma_2)) : ((\tau_1, \tau_2), \tau)$

$\Sigma \vdash \ominus(\sigma) : (\tau, \tau')$

$\texttt{not}(\sigma) : (\texttt{bool}, \texttt{bool})$	if $\Sigma \vdash \sigma <: \texttt{bool}$
$\Sigma \vdash \ominus(\sigma) : (\texttt{int}, \texttt{int})$	if $\ominus \in \{\texttt{+}, \texttt{-}\}$ and $\Sigma \vdash \sigma <: \texttt{int}$
$\Sigma \vdash \ominus(\sigma) : (\texttt{float}, \texttt{float})$	if $\ominus \in \{\texttt{+}, \texttt{-}\}$ and $\Sigma \vdash \sigma <: \texttt{float}$
$\Sigma \vdash \ominus(\sigma) : (\tau, \tau')$	if $\Sigma \vdash \ominus(\text{base-type}(\sigma)) : (\tau, \tau')$

Figure 3.9: Operator overloads

Each operator is overloaded as specified in Figure 3.9. A *resolved overload* $((\tau_1, \tau_2), \tau)$ pairs operand *bounds* with a result type; the judgements of the figure relate operand types to the resolved overloads of an operator. The binop and unop rules (Figure 3.4) take the result type of the most specific overload at the operand types, resolve-op$_\Sigma$ (§A.2), so `1 + 2` is an `int` and `1 + 2.0` a `float`.

The values of an *equality type* are compared by structural equality, which is defined except at the dynamically excluded comparisons (§B.2):

$\Sigma \vdash \nu$ eq-type
$\Sigma \vdash \texttt{Literal}[\ell]$ eq-type
$\Sigma \vdash \texttt{list}[\tau]$ eq-type if $\Sigma \vdash \tau$ eq-type

$\Sigma \vdash \texttt{dict}[\texttt{str}, \tau]$ eq-type if $\Sigma \vdash \tau$ eq-type

$\Sigma \vdash \texttt{tuple}[\vec{\tau}]$ eq-type if $\Sigma \vdash \tau_i$ eq-type $(\forall i)$

$\Sigma \vdash \sigma \mid \tau$ eq-type if $\Sigma \vdash \sigma$ eq-type and $\Sigma \vdash \tau$ eq-type

$\Sigma \vdash C[\vec{\sigma}]$ eq-type if $\Sigma \vdash \tau$ eq-type for each $x : \tau \in \mathrm{fields}_\Sigma(C)[\vec{\sigma}]$

A callable type is not an equality type, and neither is any type with a callable component, so comparing two functions is rejected statically. The final rule of each table closes the judgement under base-typing, letting `1 == 0` use the equality overload at `int`, since two distinct literal types are not comparable; `2 ** 3` is still an `int`, since the direct instance's bounds (`int`, `Literal`[3]) lie below the base instance's (`float`, `float`). For this operator the binop rule is more precise than mypy, whose overloads give `Any` whenever the exponent is not a literal, whereas the most specific overload resolves to `float`.

Lemma 3.3 (Most specific overload)**.** *If neither σ_1 nor σ_2 is a subtype of `Never` and some overload of $\odot$ applies at (σ_1, σ_2), then* resolve-op$_\Sigma(\odot, \sigma_1, \sigma_2)$ *is defined, and similarly for* resolve-op$_\Sigma(\ominus, \sigma)$.

3.3 Statements

A well-typed statement has a *static outcome* R: either Returns, when the statement *definitely returns*, or Assigns(Δ) for a context Δ with an entry for any variable declared or assigned by the statement (Definition 2.1), recording whether the variable is definitely assigned, definitely unassigned or possibly unassigned after the statement.

Context operations $\otimes$ and $\oplus$ lift to a right semiring over static outcomes R, with Assigns($\varnothing$) as left and right unit for $\otimes$ and Returns as unit for $\oplus$ and zero for $\otimes$.

Definition 3.1 (Override and merge on static outcomes)**.**

$$\begin{aligned} \mathsf{Assigns}(\Delta) \oplus \mathsf{Assigns}(\Delta') &= \mathsf{Assigns}(\Delta \oplus \Delta') \\ R \oplus \mathsf{Returns} = \mathsf{Returns} \oplus R &= R \end{aligned}$$

$$\begin{aligned} \mathsf{Assigns}(\Delta) \otimes \mathsf{Assigns}(\Delta') &= \mathsf{Assigns}(\Delta \otimes \Delta') \\ R \otimes \mathsf{Returns} = \mathsf{Returns} \otimes R &= \mathsf{Returns} \end{aligned}$$

The Returns $\otimes R$ case never arises because statement typing excludes unreachable statements.

The statement judgement $\Sigma; \Gamma; \zeta \vdash s : R$ says that under Γ and optional return type ζ, statement s has static outcome R (Figure 3.10); every rule passes ζ down unchanged. A sequence $s\ s'$ (seq) types s, then types s' under Γ overridden by the context assigned by s. A conditional combines its branches with $\oplus$, merging in Assigns($\varnothing$) when there is no `else` branch. Top-level statements are checked by a separate judgement, $\Sigma; q; \Gamma \vdash t : R \dashv \Sigma'$ (Figure 3.13): a sequence composes in the same way as a statement sequence, class declarations and `type` statements are top-level forms, and a plain statement is checked with no return type (top-stmt), so a `return` has no derivation at the top level. A `return` checks its expression against ζ, which must be of the form τ, and a bare `return` requires τ to be equivalent to `None`. An expression statement synthesises a type and discards it (expr-stmt). A call to a callable with result `None` is not an expression, because call requires a result other than `None`, and is typed as a statement by call-stmt. The restriction is for compatibility with mypy (§B.3).

A variable declaration $x : \psi$ introduces x at the type denoted by ψ, without assigning it (declare); an assignment $x = e$ requires x to be a declared but unassigned variable, and checks e against its declared type (assign); the form $x : \psi = e$ declares and assigns x at once (declare-assign). A declaration does

$\Sigma; \Gamma; \zeta \vdash s : R$

pass

$$\frac{}{\Sigma; \Gamma; \zeta \vdash \texttt{pass} : \mathsf{Assigns}(\emptyset)}$$

declare

$$\frac{\Sigma; \Gamma \vdash \psi \rhd \tau}{\Sigma; \Gamma; \zeta \vdash x : \psi : \mathsf{Assigns}(\{x : \mathsf{DU}(\tau)\})}$$

assign

$$\frac{\Gamma(x) = \mathsf{DU}(\tau) \qquad \Sigma; \Gamma \vdash e \lhd \tau}{\Sigma; \Gamma; \zeta \vdash x = e : \mathsf{Assigns}(\{x : \tau\})}$$

declare-assign

$$\frac{\Sigma; \Gamma \vdash \psi \rhd \tau \qquad \Sigma; \Gamma \vdash e \lhd \tau}{\Sigma; \Gamma; \zeta \vdash x : \psi = e : \mathsf{Assigns}(\{x : \tau\})}$$

expr-stmt

$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma}{\Sigma; \Gamma; \zeta \vdash e : \mathsf{Assigns}(\emptyset)}$$

call-stmt

$$\frac{\Sigma; \Gamma \vdash e, \vec{e} \rhd \vec{\tau}, \texttt{None} \qquad \Sigma; \Gamma \vdash \vec{e} \lhd \vec{\tau}}{\Sigma; \Gamma; \zeta \vdash e(\vec{e}) : \mathsf{Assigns}(\emptyset)}$$

assert

$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool}}{\Sigma; \Gamma; \zeta \vdash \texttt{assert } e : \mathsf{Assigns}(\emptyset)}$$

assert-msg

$$\frac{\Sigma; \Gamma \vdash e \lhd \texttt{bool} \qquad \Sigma; \Gamma \vdash e' \lhd \texttt{str}}{\Sigma; \Gamma; \zeta \vdash \texttt{assert } e, e' : \mathsf{Assigns}(\emptyset)}$$

return-none

$$\frac{\Sigma \vdash \tau \equiv \texttt{None}}{\Sigma; \Gamma; \tau \vdash \texttt{return} : \mathsf{Returns}}$$

return

$$\frac{\Sigma; \Gamma \vdash e \lhd \tau}{\Sigma; \Gamma; \tau \vdash \texttt{return } e : \mathsf{Returns}}$$

if

$$\frac{\Sigma; \Gamma \vdash e_i \lhd \texttt{bool} \quad (\forall i) \qquad \Sigma; \Gamma; \zeta \vdash s_i : R_i \quad (\forall i)}{\Sigma; \Gamma; \zeta \vdash \texttt{if } e_1 : \ s_1 \texttt{ elif } e_2 : \ s_2 \ \ldots \ \texttt{elif } e_{n+1} : \ s_{n+1} : R_1 \oplus \ldots \oplus R_{n+1} \ \oplus \mathsf{Assigns}(\emptyset)}$$

if-else

$$\frac{\Sigma; \Gamma \vdash e_i \lhd \texttt{bool} \quad (\forall i) \qquad \Sigma; \Gamma; \zeta \vdash s_i : R_i \quad (\forall i)}{\Sigma; \Gamma; \zeta \vdash \texttt{if } e_1 : \ s_1 \texttt{ elif } e_2 : \ s_2 \ \ldots \ \texttt{elif } e_{n+1} : \ s_{n+1} \texttt{ else} : \ s_{n+2} : R_1 \oplus \ldots \oplus R_{n+2}}$$

seq

$$\frac{\Sigma; \Gamma; \zeta \vdash s : \mathsf{Assigns}(\Delta) \qquad \Sigma; \Gamma \otimes \Delta; \zeta \vdash s' : R}{\Sigma; \Gamma; \zeta \vdash s \ s' : \mathsf{Assigns}(\Delta) \otimes R}$$

def

$$\frac{\Sigma; \Gamma \vdash d_i \rhd \pi_i \quad (\forall i) \qquad \Delta' = \{\mathrm{declares}(\vec{d}) : \vec{\pi}\} \qquad \Sigma; \Gamma \otimes \Delta' \vdash d_i \lhd \pi_i \quad (\forall i)}{\Sigma; \Gamma; \zeta \vdash \vec{d} : \mathsf{Assigns}(\Delta')}$$

match

$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \qquad \Sigma; \Gamma; \zeta \vdash \sigma, \mathrm{shapes}_\Sigma(\sigma, \emptyset), \vec{b} : S, \texttt{Never}, \emptyset}{\Sigma; \Gamma; \zeta \vdash \texttt{match } e : \vec{b} : S}$$

match-partial

$$\frac{\Sigma; \Gamma \vdash e \rhd \sigma \qquad \Sigma; \Gamma; \zeta \vdash \sigma, \mathrm{shapes}_\Sigma(\sigma, \emptyset), \vec{b} : S, \tau, L \qquad \tau \neq \texttt{Never}}{\Sigma; \Gamma; \zeta \vdash \texttt{match } e : \vec{b} : S \oplus \mathsf{Assigns}(\emptyset)}$$

Figure 3.10: Given optional return type ζ, statement s has static outcome R

$\Sigma; \Gamma \vdash d \rhd \pi$

def-signature

$$\frac{\Gamma' = \Gamma \otimes \{\alpha_1 : \mathsf{TypeVar}, \ldots, \alpha_n : \mathsf{TypeVar}\} \qquad \Sigma; \Gamma' \vdash \vec{\psi} \rhd \vec{\sigma} \qquad \Sigma; \Gamma' \vdash \psi \rhd \tau}{\Sigma; \Gamma \vdash \texttt{def}\ f[\vec{\alpha}](\vec{x} : \vec{\psi})\ \texttt{->}\ \psi :\ s \rhd \forall \vec{\alpha}.\texttt{Callable}[[\vec{\sigma}], \tau]}$$

(a) Definition d synthesises type scheme π

$\Sigma; \Gamma \vdash d \lhd \pi$

def-body

$$\frac{\begin{array}{c}\vec{x}\ \text{distinct} \qquad \Gamma' = \Gamma \otimes \{\alpha_1 : \mathsf{TypeVar}, \ldots, \alpha_n : \mathsf{TypeVar}\} \\ \Sigma; \Gamma' \otimes \{\vec{x} : \vec{\sigma}\} \otimes \text{scope}(\{\vec{x}\}, s); \tau \vdash s : R \qquad R = \mathsf{Returns}\ \text{or}\ \Sigma \vdash \tau \equiv \texttt{None}\end{array}}{\Sigma; \Gamma \vdash \texttt{def}\ f[\vec{\alpha}](\vec{x} : \vec{\psi})\ \texttt{->}\ \psi :\ s \lhd \forall \vec{\alpha}.\texttt{Callable}[[\vec{\sigma}], \tau]}$$

(b) Definition d checks against type scheme π

Figure 3.11: Function definitions

$\Sigma; \Gamma; \zeta \vdash \tau, L, \vec{b} : S, \tau', L'$

match-case

$$\frac{\Sigma; \Gamma \vdash p \lhd \tau \dashv \Delta \qquad \Sigma; \Gamma \vdash L, p \rightsquigarrow K, L' \qquad \Sigma; \Gamma \otimes \Delta; \zeta \vdash s : R}{\Sigma; \Gamma; \zeta \vdash \tau, L, \texttt{case}\ p :\ s : \mathsf{Assigns}(\Delta) \otimes R, \text{remaining}_\Sigma(\Gamma, \tau, p), L'}$$

match-cases

$$\frac{\Sigma; \Gamma; \zeta \vdash \tau, L, b : S, \sigma, L' \qquad \Sigma; \Gamma; \zeta \vdash \sigma, L', \vec{b} : S', \tau', L^\dagger}{\Sigma; \Gamma; \zeta \vdash \tau, L, b \cdot \vec{b} : S \oplus S', \tau', L^\dagger}$$

Figure 3.12: Given optional return type ζ, cases $\vec{b}$ have static outcome S and transform remaining type τ and residual L to τ' and L'

not check whether its name is already declared; that check is made once for the whole scope, when the rules for function definitions and modules introduce the variables of the scope as unbound, using the function scope(Y, s) (§A.1).

The conditions of `if` and `assert` check against `bool`, and an assertion message against `str`. A class declaration binds the class name to Type(C), resolves the field annotations, which may mention the type parameters of the class,, and requires the fields it declares to be distinct and, when it extends another class, the base to name a class rather than an alias and to resolve to an instantiated class, and the additional fields not to repeat an inherited field (class, class-extend). Class declarations also require the predefined name `dataclass` (Figure 2.6) to be in scope; every PurePy class must declare itself a dataclass. A `type` statement `type` $x[\vec{\alpha}] = \psi$, such as `type Pair[T] = tuple[T, T]`, declares x as an alias of the type denoted by ψ with the type parameters $\vec{\alpha}$ in scope (type-alias). An application of the alias such as `Pair[int]` denotes that type at the given arguments (ty-alias), here `tuple[int, int]`, so types never mention aliases. Since the names declared by a module are unbound before their declarations (§3.5), ψ can name only classes and aliases declared earlier in the module, and cannot

$\Sigma; q; \Gamma \vdash t \ : \ R \dashv \Sigma'$

top-stmt

$$\frac{\Sigma; \Gamma; \varepsilon \vdash s \ : \ R}{\Sigma; q; \Gamma \vdash s \ : \ R \dashv \Sigma}$$

top-seq

$$\frac{\Sigma; q; \Gamma \vdash t \ : \ \mathsf{Assigns}(\Delta) \dashv \Sigma^\dagger \qquad \Sigma^\dagger; q; \Gamma \otimes \Delta \vdash t' \ : \ R \dashv \Sigma'}{\Sigma; q; \Gamma \vdash t \ t' \ : \ \mathsf{Assigns}(\Delta) \otimes R \dashv \Sigma'}$$

class

$$\frac{\begin{array}{c}\texttt{dataclass} : \mathsf{Predef} \in \Gamma \\ \Gamma' = \Gamma \otimes \{\alpha_1 : \mathsf{TypeVar}, \ldots, \alpha_n : \mathsf{TypeVar}\} \qquad \text{own-fields}(\phi) = \vec{x} : \vec{\psi},\ \vec{x} \text{ distinct} \qquad \Sigma; \Gamma' \vdash \psi_i \rhd \sigma_i \quad (\forall i)\end{array}}{\Sigma; q; \Gamma \vdash \texttt{@dataclass class } c[\vec{\alpha}] \ : \ \phi \ : \ \mathsf{Assigns}(\{c : \mathsf{Type}(q.c)\}) \dashv \Sigma, q.c : (\vec{\alpha};\ \vec{x} : \vec{\sigma};\ \bot)}$$

class-extend

$$\frac{\begin{array}{c}\Gamma' = \Gamma \otimes \{\alpha_1 : \mathsf{TypeVar}, \ldots, \alpha_n : \mathsf{TypeVar}\} \\ \Gamma \vdash q' \ : \ \mathsf{Type}(D) \qquad \Sigma; \Gamma' \vdash q'[\vec{\psi}'] \rhd D[\vec{\sigma}'] \qquad \texttt{dataclass} : \mathsf{Predef} \in \Gamma \\ \text{own-fields}(\phi) = \vec{x} : \vec{\psi},\ \vec{x} \text{ distinct} \qquad \Sigma; \Gamma' \vdash \psi_i \rhd \sigma_i \quad (\forall i) \qquad \text{fields}_\Sigma(D) = \forall \vec{\alpha}'.\ \vec{y} : \vec{\tau} \qquad \{\vec{x}\} \cap \{\vec{y}\} = \emptyset\end{array}}{\Sigma; q; \Gamma \vdash \texttt{@dataclass class } c[\vec{\alpha}](q'[\vec{\psi}']) \ : \ \phi \ : \ \mathsf{Assigns}(\{c : \mathsf{Type}(q.c)\}) \dashv \Sigma, q.c : (\vec{\alpha};\ \vec{x} : \vec{\sigma};\ D[\vec{\sigma}'])}$$

type-alias

$$\frac{\Gamma' = \Gamma \otimes \{\alpha_1 : \mathsf{TypeVar}, \ldots, \alpha_n : \mathsf{TypeVar}\} \qquad \Sigma; \Gamma' \vdash \psi \rhd \tau}{\Sigma; q; \Gamma \vdash \texttt{type } x[\vec{\alpha}] = \psi \ : \ \mathsf{Assigns}(\{x : \mathsf{TypeAlias}(\vec{\alpha}, \tau)\}) \dashv \Sigma}$$

Figure 3.13: Top-level statement t of module q has static outcome R, extending Σ to Σ'

name x itself. Each rule that takes an annotation resolves it to a type (Figure 3.2).

The def rule for a mutual region of function definitions synthesises a type scheme for each definition (Figure 3.11), binds all the function names of the mutual region at the synthesised type schemes, and checks each definition against its scheme. For a definition `def` $f[\vec{\alpha}](\vec{x} : \vec{\psi})$ `->` ψ`:` s, the auxiliary judgements $\Sigma; \Gamma \vdash d \rhd \pi$ and $\Sigma; \Gamma \vdash d \lhd \pi$ synthesise the type scheme from the annotations (def-signature) and check the body s (def-body). Both rules extend the context with TypeVar entries for the type parameters $\vec{\alpha}$. A type scheme without type parameters is identical to its `Callable` type, so the def rule binds the name of a function without type parameters as a variable of that type. The type schemes of functions with type parameters are instantiated at a call, by the argument types (callee-scheme), and when checked against a `Callable` type, by that type (var-scheme). Since annotations denote types, not type schemes, declarations and assignments never bind variables at type schemes, only at types.

In the body, the parameters $\vec{x}$ are bound at the types denoted by $\vec{\psi}$; variables declared or assigned in s start unbound, emulating Python's scoping of function bodies, which prefers a locally bound name to one visible in an enclosing scope. A variable of the enclosing scope referenced by the body must be definitely assigned at the point at which the function is defined, whereas Python resolves the reference only when the body runs. As with any scope, the names declared in the function, including the parameters $\vec{x}$, must be distinct from one another and from any pattern variables in s. The function names of the mutual region are declared in the enclosing scope rather than in the function, and must be distinct from the other names declared in that scope.

The body s is checked with the return type denoted by ψ, and must have static outcome Returns unless that type is equivalent to `None`, so a function that declares a result definitely returns one. A return type such as `int | None` does not exempt the body from returning, nor permit a bare `return`, for compatibility with mypy (§B.3).

A `match` scrutinee synthesises a type σ, and its cases $\vec{b}$ are checked in turn by the auxiliary judgement $\Sigma; \Gamma; \zeta \vdash \tau, L, \vec{b} : S, \tau', L'$ (Figure 3.12), which threads a remaining type, starting from σ, and a residual L of shapes, starting from the shapes of σ (Figure 3.15), through the cases, merging their outcomes into S. The match rule applies when the final remaining type is `Never`, meaning the match is exhaustive; the match-partial rule applies otherwise and also merges in $\mathsf{Assigns}(\emptyset)$, the outcome of the path on which the scrutinee is not matched by any of the cases. The remaining type, the residual, reachability and exhaustiveness are the subject of §3.4.

3.4 Pattern matching

To check a `match` with scrutinee type σ, the auxiliary judgement $\Sigma; \Gamma; \zeta \vdash \tau, L, \vec{b} : S, \tau', L'$ (Figure 3.12) threads a *remaining type* τ and a *residual* L through the cases, both over-approximations of the values of σ which are not matched by any of the earlier cases. The residual is an ordered set of *shapes* (Figure 3.15), each denoting a set of values of a single type, and starts from the shapes of σ; the remaining type is coarser, a type rather than a set of shapes, and starts from σ itself. Two auxiliary functions narrow a type τ by a pattern p (§A.2): matched-as$_\Sigma(\Gamma, \tau, p)$ is the type to which τ is narrowed if p matches, and remaining$_\Sigma(\Gamma, \tau, p)$ is the type remaining if p fails. Each case's pattern is checked against the remaining type (§3.4.1), binding its pattern variables, and matched against the residual (§3.4.3). Reachability is decided by the residual: a case is *reachable* if its pattern matches at least one shape of the residual, and a match with an unreachable case is ill-formed. For a reachable case, the body is checked under the bindings and its outcome merged into S; the shapes left after matching and the type remaining if the pattern fails become the residual and remaining type for the case that follows.

A match is *exhaustive* when its cases between them match every value of σ. Exhaustiveness is decided by the final remaining type, which the match rule requires to be `Never`, with match-partial applying otherwise. The remaining type thus determines exhaustiveness as well as the types of pattern variables. The Python typing specification leaves narrowing largely unspecified,[1] so the remaining type emulates the narrowing of a match scrutinee observed in mypy (§B.3), which subtracts a class only for a constructor pattern that matches every value of the class, and a tuple only through one refutable component, i.e. one whose pattern can fail to match a value of the component's type. Cases that between them cover a field of a class therefore leave an empty residual but a remaining type other than `Never`, so a function which returns from each of the cases is rejected as not always returning, and at a class with a field of type `int | None` a case `Box(n)` after a case `Box(None)` binds `n` at `int | None` rather than at `int`. The usefulness algorithm for ML [9] also checks reachability and exhaustiveness with a single relation. The residual plays the role of the term descriptions of Sestoft [12], and the uncovered sets of Karachalias et al. [8] likewise let the type exclude cases.

3.4.1 Pattern checking

A pattern must *check* against the remaining type, $\Sigma; \Gamma \vdash p \lhd \tau \dashv \Delta$ (Figure 3.14b), before it is matched against the residual (match-case). The bindings Δ give the types of the pattern variables. The judgement holds when every tuple pattern in p is at a type which has no list values, every list pattern is at a type which has no tuple values, and every constructor pattern is at a type with at most one comparable instantiation of its class, up to equivalence. List values and tuple values are defined by

[1] https://typing.python.org/en/latest/spec/narrowing.html

$\Sigma \vdash C[\vec{\sigma}] \asymp \tau$

instance-above
$$\frac{\Sigma \vdash \tau <: C[\vec{\sigma}] \qquad \vec{\sigma}' \equiv \vec{\sigma} \text{ whenever } \Sigma \vdash \tau <: C[\vec{\sigma}']}{\Sigma \vdash C[\vec{\sigma}] \asymp \tau}$$

instance-below
$$\frac{\Sigma \vdash C[\vec{\sigma}] <: \tau \qquad \vec{\sigma}' \equiv \vec{\sigma} \text{ whenever } \Sigma \vdash C[\vec{\sigma}'] <: \tau}{\Sigma \vdash C[\vec{\sigma}] \asymp \tau}$$

(a) $C[\vec{\sigma}]$ is the instance of C at τ

$\Sigma; \Gamma \vdash p \lhd \tau \dashv \Delta$

check-var
$$\frac{\Gamma(x) = \mathsf{Unbound}}{\Sigma; \Gamma \vdash x \lhd \tau \dashv \{x : \tau\}}$$

check-wild
$$\frac{}{\Sigma; \Gamma \vdash _ \lhd \tau \dashv \emptyset}$$

check-literal
$$\frac{}{\Sigma; \Gamma \vdash \ell \lhd \tau \dashv \emptyset}$$

check-as
$$\frac{\Gamma(x) = \mathsf{Unbound} \qquad \Sigma; \Gamma \vdash p \lhd \tau \dashv \Delta}{\Sigma; \Gamma \vdash p \texttt{ as } x \lhd \tau \dashv \Delta \uplus \{x : \text{matched-as}_\Sigma(\Gamma, \tau, p)\}}$$

check-union
$$\frac{\Sigma; \Gamma \vdash p \lhd \sigma \dashv \Delta \qquad \Sigma; \Gamma \vdash p \lhd \tau \dashv \Delta'}{\Sigma; \Gamma \vdash p \lhd \sigma \mid \tau \dashv \{x : \Delta(x) \vee_\Sigma \Delta'(x) \mid x \in \text{binds}(p)\}}$$

check-tuple
$$\frac{\Sigma; \Gamma \vdash p_i \lhd \tau_i \dashv \Delta_i \quad (\forall i)}{\Sigma; \Gamma \vdash (p_1, \ldots, p_n) \lhd \texttt{tuple}[\tau_1, \ldots, \tau_n] \dashv \Delta_1 \uplus \cdots \uplus \Delta_n}$$

check-tuple-none
$$\frac{\tau \text{ has no list values} \qquad \text{no disjunct of } \tau \text{ a tuple type of length } |\vec{p}|}{\Sigma; \Gamma \vdash (\vec{p}) \lhd \tau \dashv \{x : \texttt{Never} \mid x \in \text{binds}((\vec{p}))\}}$$

check-list
$$\frac{\Sigma; \Gamma \vdash p_i \lhd \tau \dashv \Delta_i \quad (\forall i)}{\Sigma; \Gamma \vdash [\vec{p}] \lhd \texttt{list}[\tau] \dashv \Delta_1 \uplus \cdots \uplus \Delta_n}$$

check-list-none
$$\frac{\tau \text{ has no tuple values} \qquad \text{no disjunct of } \tau \text{ a list type}}{\Sigma; \Gamma \vdash [\vec{p}] \lhd \tau \dashv \{x : \texttt{Never} \mid x \in \text{binds}([\vec{p}])\}}$$

check-dict
$$\frac{\Sigma; \Gamma \vdash p_i \lhd \tau \dashv \Delta_i \quad (\forall i)}{\Sigma; \Gamma \vdash \{\vec{w} : \vec{p}\} \lhd \texttt{dict}[\texttt{str}, \tau] \dashv \Delta_1 \uplus \cdots \uplus \Delta_n}$$

check-dict-none
$$\frac{\text{no disjunct of } \tau \text{ a dictionary type}}{\Sigma; \Gamma \vdash \{\vec{w} : \vec{p}\} \lhd \tau \dashv \{x : \texttt{Never} \mid x \in \text{binds}(\{\vec{w} : \vec{p}\})\}}$$

check-constr
$$\frac{\Gamma \vdash q : \mathsf{Type}(C) \qquad \Sigma \vdash C[\vec{\sigma}] \asymp \tau \qquad \text{fields}_\Sigma(C)[\vec{\sigma}] = \vec{x} : \vec{\sigma}' \qquad \text{field-map}_\Sigma(C, a) = \{\vec{x} : \vec{p}\} \qquad \Sigma; \Gamma \vdash p_i \lhd \sigma_i' \dashv \Delta_i \quad (\forall i)}{\Sigma; \Gamma \vdash q(a) \lhd \tau \dashv \Delta_1 \uplus \cdots \uplus \Delta_n}$$

check-constr-none
$$\frac{\Gamma \vdash q : \mathsf{Type}(C) \qquad \Sigma \nvdash \tau <: C[\vec{\sigma}] \text{ and } \Sigma \nvdash C[\vec{\sigma}] <: \tau \quad (\forall \vec{\sigma})}{\Sigma; \Gamma \vdash q(a) \lhd \tau \dashv \{x : \texttt{Never} \mid x \in \text{binds}(q(a))\}}$$

(b) Pattern p checks against type τ, binding Δ

Figure 3.14: Pattern checking

subtyping:

$$\tau \text{ has list values} \iff \Sigma \vdash \texttt{list}[\sigma] <: \tau \text{ for some } \sigma, \text{ or a disjunct of } \tau \text{ is a type variable}$$
$$\tau \text{ has tuple values} \iff \Sigma \vdash \texttt{tuple}[\vec{\sigma}] <: \tau \text{ for some } \vec{\sigma}, \text{ or a disjunct of } \tau \text{ is a type variable}$$

So list types, `Sized` and `object` have list values; tuple types, `Sized` and `object` have tuple values; and a type variable, which may stand for any type, has both. The rules check a sub-pattern at the type of its position, a tuple component, list element, dictionary value or field, and a pattern at a union at each of its disjuncts. A variable pattern binds at the type at which it is checked, and a pattern named by `as` is bound at matched-as$_\Sigma(\Gamma, \tau, p)$; the bindings of sub-patterns compose by disjoint union, so a pattern that binds a name twice is ill-formed, and at a union the bindings of the disjuncts are joined. A pattern also checks against a type which has no disjunct of the form matched by the pattern: a tuple pattern when no disjunct is a tuple type of the pattern's length, a list pattern when no disjunct is a list type, a dictionary pattern when no disjunct is a dictionary type, or a constructor pattern when its class has no comparable instantiation (check-tuple-none, check-list-none, check-dict-none, check-constr-none). Such a pattern matches no shape of the type and binds its variables at `Never`, the unit of the join. At one disjunct of a union it therefore leaves the values of that disjunct to later cases, and a case with a pattern matching no shape at all is unreachable.

The conditions on tuple and list patterns ensure that pattern matching always has a result. Under the evaluation rules, matching a tuple pattern against a list or a list pattern against a tuple is undefined (Figure 4.12), although Python permits both. Both forms of pattern therefore fail at `list[int] | tuple[int, int]`, at `Sized | tuple[int, int]`, and at a type variable.

A constructor pattern cannot name `builtins.range`, because Python does not support positional sub-patterns on that class (pat-constr). The condition on constructor patterns supplies the type arguments omitted by the pattern: they are recovered by the judgement $\Sigma \vdash C[\vec{\sigma}] \asymp \tau$ from the type τ at which the pattern is checked, the *instance* of C at τ being the instantiation of C above τ, or the instantiation below τ, when there is one up to equivalence (instance-above, instance-below). Below C the base classes of the type determine the instance: at `IntLeaf`, a subclass of `Leaf[int]`, the instance of `Leaf` is `Leaf[int]`. Above C the base arguments of C determine it: at `Tree[int]` the instance of `Leaf` is `Leaf[int]`, because `Leaf[T]` extends `Tree[T]`, whereas at `object` a class with type parameters has no instance, since every instantiation lies below `object`. The instance judgement is decided as follows. A pattern is checked at the disjuncts of a union separately (check-union), so the type τ at which an instance is taken is never a union. If τ is an instantiated class $D[\vec{\sigma}]$ with C an ancestor of D, the instance is the instantiation of C reached from $D[\vec{\sigma}]$ along the base classes, which is unique because type arguments are invariant (instance-above). If τ is $D[\vec{\sigma}]$ with D an ancestor of C, the instantiations of C below τ are the $C[\vec{\tau}]$ for which the instantiation of D reached from $C[\vec{\tau}]$ along the base classes has arguments equivalent to $\vec{\sigma}$, and Lemma 3.4 bounds the instantiations to be examined to finitely many instantiation candidates: one instantiation candidate up to equivalence gives the instance (instance-below), none means that C has no instance at τ, and two that are not equivalent mean that C has an instantiation comparable with τ but no instance, so that a constructor pattern at τ is ill-formed. At `object` and at `Never` every instantiation of C is comparable with τ, so C has an instance only if it has no type parameters. At any other type C has no instantiation comparable with τ, and no instance.

Lemma 3.4 (Sufficiency of instantiation candidates)**.** *Suppose* $D \in \text{ancestors}_\Sigma(C)$*. An* instantiation candidate *for* C *at* $D[\vec{\sigma}]$ *is an instantiation* $C[\vec{\tau}]$ *in which each* τ_i *is the join of a subset of* $\text{subterms}(\vec{\sigma}) \cup \{\texttt{object}\}$ *(§A.2). If any instantiation of* C *is below* $D[\vec{\sigma}]$*, then some instantiation candidate is below* $D[\vec{\sigma}]$*;*

and if two instantiations of C that are not equivalent are below $D[\vec{\sigma}]$, then two instantiation candidates below $D[\vec{\sigma}]$ are not equivalent.

3.4.2 Shapes and shape typing

Heads			
h	$::=$	ℓ	(Literal)
	$\mid$	C	(Class)
	$\mid$	$\texttt{list}_n$	(List of length n)
H	$::=$	$\{h_1, \ldots, h_n\}$	(Excluded heads)
Shapes			
k	$::=$	$\tau \setminus H$	(Rest)
	$\mid$	$C[\vec{\tau}](k_1, \ldots, k_n) \setminus H$	(Constr)
	$\mid$	$(k_1, \ldots, k_n)$	(Tuple)
	$\mid$	$[k_1, \ldots, k_n]_\tau$	(List)
	$\mid$	$\beta \setminus_\tau H$	(Dict)
K, L	$::=$	$\{k_1, \ldots, k_n\}$	(Shape set)

$\Sigma \vdash h : \tau$

head-literal
$$\frac{\Sigma \vdash \texttt{Literal}[\ell] <: \tau}{\Sigma \vdash \ell : \tau}$$

head-class
$$\frac{\Sigma \vdash C[\vec{\sigma}] <: \tau}{\Sigma \vdash C : \tau}$$

head-list
$$\frac{}{\Sigma \vdash \texttt{list}_n : \texttt{list}[\tau]}$$

$\Sigma \vdash k : \tau$

shape-rest
$$\frac{\Sigma \vdash h : \tau \quad (\forall h \in H) \qquad \tau \text{ neither union nor dictionary type}}{\Sigma \vdash \tau \setminus H : \tau}$$

shape-constr
$$\frac{\text{fields}_\Sigma(C)[\vec{\tau}] = \vec{x} : \vec{\sigma} \qquad \Sigma \vdash h : C[\vec{\tau}] \quad (\forall h \in H) \qquad \Sigma \vdash k_i : \sigma_i \quad (\forall i)}{\Sigma \vdash C[\vec{\tau}](\vec{k}) \setminus H : C[\vec{\tau}]}$$

shape-tuple
$$\frac{\Sigma \vdash k_i : \tau_i \quad (\forall i)}{\Sigma \vdash (\vec{k}) : \texttt{tuple}[\vec{\tau}]}$$

shape-list
$$\frac{\Sigma \vdash k_i : \tau \quad (\forall i)}{\Sigma \vdash [\vec{k}]_\tau : \texttt{list}[\tau]}$$

shape-sub
$$\frac{\Sigma \vdash k : \sigma \qquad \Sigma \vdash \sigma <: \tau}{\Sigma \vdash k : \tau}$$

shape-dict
$$\frac{\Sigma \vdash h : \texttt{str} \quad (\forall h \in H) \qquad \Sigma \vdash \beta(w) : \tau \quad (\forall w \in \text{dom}(\beta)) \qquad \text{dom}(\beta) \cap H = \emptyset}{\Sigma \vdash \beta \setminus_\tau H : \texttt{dict}[\texttt{str}, \tau]}$$

Figure 3.15: Shapes and shape typing

A shape k (Figure 3.15) denotes a set of values of its type shape-type(k) (§A.2). The form $\tau \setminus H$ denotes the values of type τ whose outermost head is not among those in H; the other forms are

constructor, tuple, list and dictionary shapes, each with shapes in place of sub-patterns. A list shape also records its element type, which the element shapes do not determine. A constructor shape also carries an excluded set: $C[\vec{\tau}](\vec{k}) \setminus H$ denotes the instances of $C[\vec{\tau}]$, or of any subclass not below a class of H, whose fields, those of C, have shapes $\vec{k}$. A rest shape at a class type denotes the same values as the constructor shape with a rest shape at each field, and is expanded into that form only by splitting (Figure 3.17): a class may refer to itself through its fields, so expanding in advance would not terminate.

A head itself denotes a set of values: a literal its value, a class its instances, and $\texttt{list}_n$ the lists of length n. Head typing $\Sigma \vdash h : \tau$ holds when the values of h lie within τ. Shape typing $\Sigma \vdash k : \tau$ holds when shape-type(k) is a subtype of τ and k is well-formed: each excluded head is typed at the type of the shape (at `str`, for the keys of a dictionary shape) and each component shape at the type of that component.

The auxiliary function $\text{shapes}_\Sigma(\tau, H)$ (§A.2) gives the shapes of τ once the heads in H are excluded. A union contributes the shapes of its disjuncts and a dictionary type the dictionary form; every other type contributes the single shape $\tau \setminus H$, or no shape at all when H exhausts τ. The rest form therefore never carries a union or dictionary type, and shape-rest states this restriction as a side condition.

3.4.3 Matching

Matching (Figure 3.16) has three forms. The form on a single shape k matches a pattern against that shape, giving matched shapes K and residual L. The form on sequences matches the sub-patterns of a tuple, list, dictionary or constructor pattern against the component shapes position by position. The form on an ordered set of shapes L, used by the match rules, matches each shape of L in turn. The pattern need not match every shape of L, so pat-shapes takes the shapes of L matched by the pattern, of which there must be at least one, and passes the unmatched shapes into the residual untouched.

A pattern matches a shape directly if the shape has the head tested by the pattern. Otherwise the shape is first *split* by the auxiliary judgement $\Sigma; \Gamma \vdash k, p \rightsquigarrow_{\mathsf{h}} K, L$ (Figure 3.17): the shapes K have the head tested by p, with the shapes of the component types drawn from shapes_Σ, and the residual L is the shape k with that head excluded as well. The pat-split rule matches the pattern against K and adds L to the residual. Only the forms with an excluded set split, and no head types at a tuple type, so the excluded set of a tuple shape is empty and split-tuple leaves the empty residual. A constructor pattern whose class lies below a class of the excluded set matches nothing, because an earlier case for the superclass has already matched every instance. A constructor pattern matches the instance of its class at the type of the shape (Figure 3.14a): a pattern `Leaf(v)` at `Tree[int]` matches `Leaf[int]` and binds `v` at `int`.

A dictionary shape has a separate form, $\beta \setminus_\tau H$, standing for the dictionaries that hold each key w in the domain of β with a value of shape $\beta(w)$, but that hold no key of H. A dictionary pattern takes its keys in order, with no key occurring twice. A key in neither β nor H splits the shape at that key (split-dict), adding to the residual the dictionaries that lack the key; once every key of the pattern is in β the keys match as a sequence (pat-dict), each key w against $\beta(w)$. The empty dictionary pattern matches every dictionary and leaves the empty residual.

The cases needed to exhaust a match therefore depend on the type of the scrutinee: a union is exhausted by covering each disjunct, a class type by a pattern for the class itself, `bool` by `True` and `False`, and a dictionary type by the empty dictionary pattern, while `int`, `str` and lists have unboundedly many heads and need a variable or wildcard. A literal-typed scrutinee is exhausted by covering its literals, so `x: Literal[42] = 42` admits a match with a case for `42` alone, while `x: int = 42` keeps the type open. Over `tuple[bool, bool]` the cases `(True, _)`, `(_, False)` and `(False, True)` leave the empty residual, although each covers only part of the type.

$\Sigma; \Gamma \vdash k, p \rightsquigarrow K, L$

pat-var

$$\frac{}{\Sigma; \Gamma \vdash k, x \rightsquigarrow \{k\}, \emptyset}$$

pat-wild

$$\frac{}{\Sigma; \Gamma \vdash k, _ \rightsquigarrow \{k\}, \emptyset}$$

pat-as

$$\frac{\Sigma; \Gamma \vdash k, p \rightsquigarrow K, L}{\Sigma; \Gamma \vdash k, p \texttt{ as } x \rightsquigarrow K, L}$$

pat-split

$$\frac{\Sigma; \Gamma \vdash k, p \rightsquigarrow_{\mathsf{h}} K, L \qquad \Sigma; \Gamma \vdash K, p \rightsquigarrow K', L'}{\Sigma; \Gamma \vdash k, p \rightsquigarrow K', L \uplus L'}$$

pat-literal

$$\frac{}{\Sigma; \Gamma \vdash \texttt{Literal}[\ell] \setminus \emptyset, \ell \rightsquigarrow \{\texttt{Literal}[\ell] \setminus \emptyset\}, \emptyset}$$

pat-tuple

$$\frac{\Sigma; \Gamma \vdash \vec{k}, \vec{p} \rightsquigarrow K, L}{\Sigma; \Gamma \vdash (\vec{k}), (\vec{p}) \rightsquigarrow \{(\vec{k}') \mid \vec{k}' \in K\}, \{(\vec{l}) \mid \vec{l} \in L\}}$$

pat-list

$$\frac{\Sigma; \Gamma \vdash \vec{k}, \vec{p} \rightsquigarrow K, L}{\Sigma; \Gamma \vdash [\vec{k}]_\tau, [\vec{p}] \rightsquigarrow \{[\vec{k}']_\tau \mid \vec{k}' \in K\}, \{[\vec{l}]_\tau \mid \vec{l} \in L\}}$$

pat-dict

$$\frac{\vec{w} \text{ distinct} \qquad k_i = \beta(w_i) \quad (\forall i) \qquad \Sigma; \Gamma \vdash \vec{k}, \vec{p} \rightsquigarrow K, L}{\Sigma; \Gamma \vdash \beta \setminus_\tau H, \{\vec{w} \colon \vec{p}\} \rightsquigarrow \{\beta \otimes \{\vec{w} \colon \vec{k}'\} \setminus_\tau H \mid \vec{k}' \in K\}, \{\beta \otimes \{\vec{w} \colon \vec{l}\} \setminus_\tau H \mid \vec{l} \in L\}}$$

pat-constr

$$\frac{\Gamma \vdash q : \mathsf{Type}(D) \qquad D \neq \texttt{builtins.range} \qquad D \in \mathrm{ancestors}_\Sigma(C) \qquad \mathrm{fields}_\Sigma(D) = \forall \vec{\alpha}.\, \vec{x} \colon \vec{\sigma} \qquad \text{field-map}_\Sigma(D, a) = \{\vec{x} \colon \vec{p}\} \qquad p'_i = _ \quad (\forall i) \qquad \Sigma; \Gamma \vdash \vec{k}, \vec{p} +\!\!+ \vec{p}' \rightsquigarrow K, L}{\Sigma; \Gamma \vdash C[\vec{\tau}](\vec{k}) \setminus H, q(a) \rightsquigarrow \{C[\vec{\tau}](\vec{k}') \setminus H \mid \vec{k}' \in K\}, \{C[\vec{\tau}](\vec{l}) \setminus H \mid \vec{l} \in L\}}$$

$\Sigma; \Gamma \vdash \vec{k}, \vec{p} \rightsquigarrow K, L$

pat-nil

$$\frac{}{\Sigma; \Gamma \vdash \varepsilon, \varepsilon \rightsquigarrow \{\varepsilon\}, \emptyset}$$

pat-cons

$$\frac{\Sigma; \Gamma \vdash k, p \rightsquigarrow K, L \qquad \Sigma; \Gamma \vdash \vec{k}, \vec{p} \rightsquigarrow K', L'}{\Sigma; \Gamma \vdash k \cdot \vec{k}, p \cdot \vec{p} \rightsquigarrow K \times K', (L \times \{\vec{k}\}) \uplus (K \times L')}$$

$\Sigma; \Gamma \vdash L, p \rightsquigarrow K, L'$

pat-shapes

$$\frac{L' = \{k \in L \mid \Sigma; \Gamma \vdash k, p \rightsquigarrow K_k, L_k\} \qquad L' \neq \emptyset}{\Sigma; \Gamma \vdash L, p \rightsquigarrow \biguplus_{k \in L'} K_k,\ \biguplus_{k \in L'} L_k \uplus (L \setminus L')}$$

Figure 3.16: Pattern p matches shape k, giving matched shapes K and residual L; lifted to sequences and ordered sets of shapes

$\Sigma;\Gamma \vdash k, p \rightsquigarrow_{\mathsf{h}} K, L$

split-literal

$$\frac{\ell \notin H \qquad \Sigma \vdash \texttt{Literal}[\ell] <: \tau \qquad \tau \text{ not a literal type}}{\Sigma;\Gamma \vdash \tau \setminus H, \ell \rightsquigarrow_{\mathsf{h}} \{\texttt{Literal}[\ell] \setminus \emptyset\}, \text{shapes}_{\Sigma}(\tau, H \cup \{\ell\})}$$

split-tuple

$$\frac{|\vec{\tau}| = |\vec{p}|}{\Sigma;\Gamma \vdash \texttt{tuple}[\vec{\tau}] \setminus \emptyset, (\vec{p}) \rightsquigarrow_{\mathsf{h}} \{(\vec{k}) \mid \vec{k} \in \text{shapes}_{\Sigma}(\vec{\tau})\}, \emptyset}$$

split-list

$$\frac{n = |\vec{p}| \qquad \texttt{list}_n \notin H}{\Sigma;\Gamma \vdash \texttt{list}[\tau] \setminus H, [\vec{p}] \rightsquigarrow_{\mathsf{h}} \{[\vec{k}]_{\tau} \mid \vec{k} \in \text{shapes}_{\Sigma}(\tau, \emptyset)^n\}, \text{shapes}_{\Sigma}(\texttt{list}[\tau], H \cup \{\texttt{list}_n\})}$$

split-dict

$$\frac{w_i \notin \text{dom}(\beta) \cup H \qquad w_j \in \text{dom}(\beta) \quad (\forall j < i)}{\Sigma;\Gamma \vdash \beta \setminus_{\tau} H, \{\vec{w} : \vec{p}\} \rightsquigarrow_{\mathsf{h}} \{\beta \otimes \{w_i : k\} \setminus_{\tau} H \mid k \in \text{shapes}_{\Sigma}(\tau, \emptyset)\}, \{\beta \setminus_{\tau} (H \cup \{w_i\})\}}$$

split-class

$$\frac{\begin{array}{c}\Gamma \vdash q : \mathsf{Type}(C) \qquad \text{ancestors}_{\Sigma}(C) \cap H = \emptyset \\ \Sigma \vdash C[\vec{\tau}'] \asymp \tau \qquad \tau \wedge_{\Sigma} C[\vec{\tau}'] = D[\vec{\sigma}] \qquad \text{fields}_{\Sigma}(D)[\vec{\sigma}] = \vec{x} : \vec{\sigma}' \qquad H' = \{h \in H \mid \Sigma \vdash h : D[\vec{\sigma}]\}\end{array}}{\Sigma;\Gamma \vdash \tau \setminus H, q(a) \rightsquigarrow_{\mathsf{h}} \{D[\vec{\sigma}](\vec{k}) \setminus H' \mid \vec{k} \in \text{shapes}_{\Sigma}(\vec{\sigma}')\}, \text{shapes}_{\Sigma}(\tau, H \cup \{C\})}$$

split-subclass

$$\frac{\begin{array}{c}\Gamma \vdash q : \mathsf{Type}(D) \qquad \Sigma \vdash D[\vec{\sigma}] \asymp C[\vec{\tau}] \qquad \Sigma \vdash D[\vec{\sigma}] <: C[\vec{\tau}] \qquad D \neq C \\ \text{ancestors}_{\Sigma}(D) \cap H = \emptyset \qquad \text{fields}_{\Sigma}(D)[\vec{\sigma}] = \text{fields}_{\Sigma}(C)[\vec{\tau}] +\!\!+ \vec{y} : \vec{\sigma}' \qquad H' = \{h \in H \mid \Sigma \vdash h : D[\vec{\sigma}]\}\end{array}}{\Sigma;\Gamma \vdash C[\vec{\tau}](\vec{k}) \setminus H, q(a) \rightsquigarrow_{\mathsf{h}} \{D[\vec{\sigma}](\vec{k} +\!\!+ \vec{k}') \setminus H' \mid \vec{k}' \in \text{shapes}_{\Sigma}(\vec{\sigma}')\}, \{C[\vec{\tau}](\vec{k}) \setminus (H \cup \{D\})\}}$$

Figure 3.17: Shape k splits at the head tested by p, giving shapes K and residual L

Lemma 3.5 (Preservation of shape type)**.** *If $\Sigma \vdash h : \tau$ for every $h \in H$ then $\Sigma \vdash k : \tau$ for each $k \in \text{shapes}_{\Sigma}(\tau, H)$. Moreover, if $\Sigma \vdash k : \tau$ for each $k \in L$ and $\Sigma;\Gamma \vdash L, p \rightsquigarrow K, L'$ then $\Sigma \vdash k : \tau$ for each $k \in K \uplus L'$.*

3.5 Modules and programs

Static loading types a module's body and computes its signature Γ, the judgement $\Sigma \vdash_{\mathsf{load}} q : \Gamma \dashv \Sigma'$ (Figure 3.20); it mirrors loading at run time (Chapter 4). The module rule types the body as a top-level statement, and the predefined rule gives a predefined module the signature of Definition 2.4. Both import forms statically load their target, together with its containing modules (Figure 3.19). A plain import statically loads the root and then each module along the chain of names, recording each as a member of its parent (loads-as-cons), so that the binding for the root gives attribute access along the whole chain. A from-import also statically loads any imported name that denotes a submodule, and the names it imports must be distinct. A containing module of the target is exempt from static loading if it is the importing module or one of its containing modules (§A.3), because at runtime it may still be loading when the import executes. The exemption applies only to from-imports, because for a plain import the only containing module that could never load first is the importing module itself, and

an import naming a module contained in the importing module is excluded outright. Static loading is defined inductively, so a program with an import cycle is ill-formed. Program well-formedness (Figure 3.20) requires the main module to be well typed and hence, recursively, every module that the program can load; a module that is never imported is not checked. The rule module types a module body under the context of $\mathcal{P}(\texttt{builtins})$, overridden by the imported context and then, as in def-body, by an unbound entry for each name declared in the body or bound by a pattern in it. The members of `builtins` are thus in scope unqualified, and a name bound by the body shadows a builtin throughout the module, so a use of the name before its declaration is rejected. Under mypy such a use instead resolves to the builtin, following Python's run-time lookup. As with any scope, the names declared in the module, including its imported names, must be distinct from one another and from any pattern variables in its body. Since every module declares `__name__` (§2.4), a module is ill-formed if its body declares that name again, imports it, or binds it in a pattern.

$\Sigma \vdash \Gamma, x \text{ imports } \theta \dashv \Sigma'$

imp-member

$$\frac{\Gamma \vdash x : \theta}{\Sigma \vdash \Gamma, x \text{ imports } \theta \dashv \Sigma}$$

imp-submodule

$$\frac{x : \mathsf{Mod}(q) \in \Gamma \qquad \Sigma \vdash_{\mathsf{load}} q : \Delta \dashv \Sigma'}{\Sigma \vdash \Gamma, x \text{ imports } \mathsf{Mod}(q, \Delta) \dashv \Sigma'}$$

Figure 3.18: Name x imported from module signature Γ has context entry θ

$\Sigma; q \vdash \vec{\imath} : \Gamma \dashv \Sigma'$

imports-nil

$$\frac{}{\Sigma; q \vdash \varepsilon : \emptyset \dashv \Sigma}$$

import

$$\frac{q' = x \cdot \vec{x} \qquad q \not\asymp q' \qquad \Sigma \vdash_{\mathsf{load}} x : \Delta \dashv \Sigma_1 \qquad \Sigma_1 \vdash \mathsf{Mod}(x, \Delta), \vec{x} \text{ loads-as } \theta \dashv \Sigma_2 \qquad \Sigma_2; q \vdash \vec{\imath} : \Gamma \dashv \Sigma'}{\Sigma; q \vdash \texttt{import } q' \cdot \vec{\imath} : \{x : \theta\} \boxplus \Gamma \dashv \Sigma'}$$

from-import

$$\frac{\begin{array}{c} x_1, \ldots, x_{n+1} \text{ distinct} \qquad \Sigma \vdash_{\mathsf{load}} q' : \Delta \dashv \Sigma_0 \qquad q_1^\dagger \cdot \cdots \cdot q_m^\dagger = \text{containing}(q, q') \\ \Sigma_{j-1} \vdash_{\mathsf{load}} q_j^\dagger : \Delta_j \dashv \Sigma_j \quad (\forall j) \qquad \Sigma_{m+i-1} \vdash \Delta, x_i \text{ imports } \theta_i \dashv \Sigma_{m+i} \quad (\forall i) \qquad \Sigma_{m+n+1}; q \vdash \vec{\imath} : \Gamma \dashv \Sigma' \end{array}}{\Sigma; q \vdash \texttt{from } q' \texttt{ import } x_1, \ldots, x_{n+1} \cdot \vec{\imath} : \{x_1 : \theta_1, \ldots, x_{n+1} : \theta_{n+1}\} \boxplus \Gamma \dashv \Sigma'}$$

(a) Import prefix $\vec{\imath}$ of module q produces bindings Γ

$\Sigma \vdash \theta, \vec{x} \text{ loads-as } \theta' \dashv \Sigma'$

loads-as-nil

$$\frac{}{\Sigma \vdash \theta, \varepsilon \text{ loads-as } \theta \dashv \Sigma}$$

loads-as-cons

$$\frac{\Sigma \vdash_{\mathsf{load}} q.x : \Gamma' \dashv \Sigma^\dagger \qquad \Sigma^\dagger \vdash \mathsf{Mod}(q.x, \Gamma'), \vec{x} \text{ loads-as } \theta' \dashv \Sigma'}{\Sigma \vdash \mathsf{Mod}(q, \Gamma), x \cdot \vec{x} \text{ loads-as } \mathsf{Mod}(q, \Gamma \boxplus \{x : \theta'\}) \dashv \Sigma'}$$

(b) Loaded module θ with submodule chain $\vec{x}$ loads as θ'

Figure 3.19: Import typing

$\Sigma \vdash_{\mathsf{load}} q : \Gamma \dashv \Sigma'$

predefined

$$\frac{\mathcal{P}(q) = (\Gamma, \rho)}{\Sigma \vdash_{\mathsf{load}} q : \Gamma \dashv \Sigma}$$

module

$$\frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad \mathcal{P}(\texttt{builtins}) = (\Gamma, \rho) \qquad \Sigma; q \vdash \vec{\imath} : \Gamma' \dashv \Sigma^\dagger \qquad \Sigma^\dagger; q; \Gamma \otimes \Gamma' \otimes \mathrm{scope}(\{\mathrm{declares}(\vec{\imath})\}, t) \vdash t : \mathsf{Assigns}(\Delta) \dashv \Sigma' \qquad (\mathrm{dom}(\Gamma') \cup \mathrm{dom}(\Delta)) \cap \mathrm{dom}(\mathrm{submods}(q)) = \emptyset}{\Sigma \vdash_{\mathsf{load}} q : \mathrm{submods}(q) \otimes \Delta \dashv \Sigma'}$$

$\vdash \mathcal{M}$

program

$$\frac{\Sigma_{\mathcal{P}} \vdash_{\mathsf{load}} \texttt{__main__} : \Gamma \dashv \Sigma}{\vdash \mathcal{M}}$$

Figure 3.20: Module typing

The signature Γ produced by static loading (Figure 3.20) comprises the classes, type aliases and variables defined by a module, together with its intrinsic `__name__` and a stub for each submodule, but not the names that it imports: following PEP 484[2], an imported name is a private dependency, not re-exported as part of the interface. Submodule stubs and the bindings of the module body are disjoint, because the module rules exclude a module that binds a name that is also the name of a submodule.

[2] https://peps.python.org/pep-0484/

4 Operational semantics

4.1 Values and environments

Values, environments, outcomes and results are given in Figure 4.1. A class value $\mathsf{Type}(C)$ is the context entry of the class, and an object $\mathsf{Obj}(C, \rho)$ records its class by name; the value Unbound marks a variable of the current scope before its declaration is reached, a state reported by Python as an unbound local, and an environment binds a variable to Unbound wherever the corresponding context binds the variable to the entry Unbound, so that a shadowed binding of an enclosing scope is not visible. Throughout this section, Σ is the class table produced by static loading of the program (§3.5) and, like $\mathcal{M}$ and $\mathcal{P}$, is ambient. For a statement s, the outcome $\mathsf{assigns}(\rho)$ represents s running to its end, with accumulated environment ρ; the outcome $\mathsf{returns}\ v$ represents s returning the value v; and the outcome $\mathsf{aborts}\ \kappa$ represents the whole run aborting, with termination kind κ. Composing statements sequentially composes their outcomes according to the following partial monoid, analogous to sequential composition of static outcomes:

$$\begin{aligned} \mathsf{assigns}(\rho) \otimes \mathsf{assigns}(\rho') &= \mathsf{assigns}(\rho \otimes \rho') \\ \mathsf{assigns}(\rho) \otimes \mathsf{returns}\ v &= \mathsf{returns}\ v \\ \mathsf{assigns}(\rho) \otimes \mathsf{aborts}\ \kappa &= \mathsf{aborts}\ \kappa \end{aligned}$$

with $\mathsf{assigns}(\emptyset)$ as left and right unit.

A *termination kind* κ records how a run ends; all but $\mathsf{SystemExit}(n)$ are named after the exceptions raised by the same program under Python in the same circumstances. A primitive $\mathsf{Prim}(f)$ is a predefined function whose meaning f is a partial map from arguments to results (Figure 2.7). Calling $\mathsf{Prim}(f)$ with arguments $\vec{u}$ has result $f(\vec{u})$, and no derivation if $f(\vec{u})$ is undefined.

The rules are given in a monadic metalanguage over results (Figure 4.1), in which $\mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \ldots$ binds the value of a result; for example, eval-binop concludes $\mathsf{let}\ v' \leftarrow r\ \mathsf{in}\ \hat{\odot}(v, v')$, which is $\hat{\odot}(v, v')$ when r is $\mathsf{val}\ v'$, and $\mathsf{aborts}\ \kappa$ when r is $\mathsf{aborts}\ \kappa$. A final premise needs no propagation rule; earlier premises have one each, recording the subexpressions evaluated before the failing one (Figure 4.10). The statement, sequence and module judgements use results of the same form, carrying an outcome, a sequence of values or an environment in place of a value.

4.2 Auxiliary definitions

The operators and the other auxiliary functions the evaluation rules use are defined in §A.4, and module and class resolution in §A.3.

4.3 Expressions and statements

A value is well-formed at a context entry, and an environment at a context (Figure 4.2). For any context Γ, any $\rho : \Gamma$ and any statement s with $\Gamma \vdash s : R$, the statement evaluation judgement $\rho, s \Rightarrow o$

Values			
v	$::=$	ℓ	(Literal)
	$\mid$	$[v_1, \ldots, v_n]$	(List)
	$\mid$	$(v_1, \ldots, v_n)$	(Tuple)
	$\mid$	$\{\delta\}$	(Dict)
	$\mid$	$\mathsf{Lam}_\Gamma(\rho, \vec{x}, e)$	(LambdaClosure)
	$\mid$	$\mathsf{Def}_\Gamma(\rho, \vec{d}, i)$	(DefClosure)
	$\mid$	$\mathsf{Obj}(C, \rho)$	(Object)
	$\mid$	$\mathsf{Mod}(q)$	(ModuleStub)
	$\mid$	$\mathsf{Mod}(q, \rho)$	(LoadedModule)
	$\mid$	$\mathsf{Type}(C)$	(Class)
	$\mid$	$\mathsf{TypeAlias}(\vec{\alpha}, \tau)$	(TypeAlias)
	$\mid$	$\mathsf{Unbound}$	(Unbound)
	$\mid$	$\mathsf{Prim}(f)$	(Primitive)
	$\mid$	$\mathsf{Opaque}(q.x)$	(Opaque)
Environments			
ρ	$::=$	$\{x_1 : v_1, \ldots, x_n : v_n\}$	
Dictionary entries			
δ	$::=$	$w_1 : v_1 \cdot \ldots \cdot w_n : v_n$	
Outcomes			
o	$::=$	$\mathsf{assigns}(\rho)$	(Assigns)
	$\mid$	$\mathsf{returns}\ v$	(Returns)
	$\mid$	$\mathsf{aborts}\ \kappa$	(Aborts)
Results			
r	$::=$	$\mathsf{val}\ v$	(Val)
	$\mid$	$\mathsf{aborts}\ \kappa$	(Aborts)
Match results			
m	$::=$	$\mathsf{match}\ \rho$	(Match)
	$\mid$	$\mathsf{no\text{-}match}$	(NoMatch)
Termination kinds			
κ	$::=$	$\mathsf{TypeError} \mid \mathsf{IndexError} \mid \mathsf{KeyError} \mid \mathsf{ZeroDivisionError}$	
	$\mid$	$\mathsf{AssertionError} \mid \mathsf{AssertionError}(w) \mid \mathsf{AttributeError} \mid \mathsf{SystemExit}(n)$	

Figure 4.1: Values, environments, outcomes and results

says that s evaluates to outcome o in ρ (Figures 4.3 and 4.4). Top-level statements are evaluated by a separate judgement, $q, \rho, t \Rightarrow o$ (Figure 4.5), mirroring the static judgement (Figure 3.13): a sequence composes in the same way as a statement sequence, a class declaration binds its name to the class, a `type` statement binds its name to the alias, and a plain statement is evaluated by the statement judgement (eval-top-stmt). Function definitions delimit the scope of sequential-composition effects, so as black boxes functions are always pure. The auxiliary function result (§A.4) reads the outcome of a statement as the result of a call; if the statement completes without returning, the result is val `None`.

For any context Γ, any $\rho : \Gamma$ and any expression e with $\Gamma \vdash e \triangleright \tau$, the expression evaluation judgement $\rho, e \Rightarrow r$ (Figures 4.6 and 4.7) says that e evaluates to result r in ρ, where r is either val v or aborts κ (Figure 4.1). The operators `and` and `or` evaluate their second operand only when the first

leaves the result open, and a conditional expression requires its condition to be `True` or `False`, like the `if` statement (eval-if). A comprehension evaluates its body once for each binding produced by its qualifiers, and concatenates the results; a generator draws its bindings from the elements of a value, given by the auxiliary function iter (§A.4): the elements of a list, tuple or string, and the keys of a dictionary, as in Python. Subscripting, given by the auxiliary function getitem (§A.4), indexes a list, tuple or string by an integer, counting from the end when negative, and a dictionary by a string key, aborting with IndexError, KeyError or TypeError when Python raises the exception of that name; a dictionary subscripted by a value other than a string is undefined. Likewise an attribute reference on a value other than a module or object has no derivation: the attributes of built-in values are undefined.

$\vdash v : \theta$

val-lit
$$\frac{}{\vdash \ell : \texttt{Literal}[\ell]}$$

val-list
$$\frac{\vdash v_i : \tau \quad (\forall i)}{\vdash [\vec{v}] : \texttt{list}[\tau]}$$

val-tuple
$$\frac{\vdash v_i : \tau_i \quad (\forall i)}{\vdash (\vec{v}) : \texttt{tuple}[\vec{\tau}]}$$

val-dict
$$\frac{\vdash v_i : \tau \quad (\forall i) \qquad \vec{w} \text{ distinct}}{\vdash \{\vec{w} : \vec{v}\} : \texttt{dict}[\texttt{str}, \tau]}$$

val-lambda
$$\frac{\vdash \rho : \Gamma \qquad \Gamma \otimes \{\vec{x} : \vec{\sigma}\} \vdash e \lhd \tau}{\vdash \mathsf{Lam}_\Gamma(\rho, \vec{x}, e) : \texttt{Callable}[[\vec{\sigma}], \tau]}$$

val-def
$$\frac{\vdash \rho : \Gamma \qquad \text{declares}(\vec{d}) = \vec{f} \qquad \Gamma \vdash \vec{d} : \mathsf{Assigns}(\Delta)}{\vdash \mathsf{Def}_\Gamma(\rho, \vec{d}, i) : \Delta(f_i)}$$

val-obj
$$\frac{\vdash \rho(x) : \tau \quad (\forall x : \tau \in \text{fields}_\Sigma(C))}{\vdash \mathsf{Obj}(C, \rho) : C}$$

val-sub
$$\frac{\vdash v : \sigma \qquad \Sigma \vdash \sigma <: \tau}{\vdash v : \tau}$$

val-prim
$$\frac{\mathcal{P}(q) = (\Gamma, \rho) \qquad \rho(x) = \mathsf{Prim}(f) \qquad \Gamma(x) = \theta}{\vdash \mathsf{Prim}(f) : \theta}$$

val-opaque
$$\frac{}{\vdash \mathsf{Opaque}(q.x) : \mathsf{Predef}}$$

val-mod
$$\frac{q \in \text{dom}(\mathcal{M}) \cup \text{dom}(\mathcal{P})}{\vdash \mathsf{Mod}(q) : \mathsf{Mod}(q)}$$

val-mod-loaded
$$\frac{\vdash \rho : \Gamma}{\vdash \mathsf{Mod}(q, \rho) : \mathsf{Mod}(q, \Gamma)}$$

val-class
$$\frac{C \in \text{dom}(\Sigma)}{\vdash \mathsf{Type}(C) : \mathsf{Type}(C)}$$

val-type-alias
$$\frac{}{\vdash \mathsf{TypeAlias}(\vec{\alpha}, \tau) : \mathsf{TypeAlias}(\vec{\alpha}, \tau)}$$

val-unbound
$$\frac{\theta \in \{\mathsf{Unbound}, \mathsf{DU}(\tau)\}}{\vdash \mathsf{Unbound} : \theta}$$

val-pu
$$\frac{v = \mathsf{Unbound} \text{ or } \vdash v : \tau}{\vdash v : \mathsf{PU}(\tau)}$$

$\vdash \rho : \Gamma$

env-nil
$$\frac{}{\vdash \emptyset : \emptyset}$$

env-cons
$$\frac{\vdash v : \theta \qquad \vdash \rho : \Gamma}{\vdash \{x : v\} \otimes \rho : \{x : \theta\} \otimes \Gamma}$$

Figure 4.2: Value and environment well-formedness

$\rho, s \Rightarrow o$

eval-pass

$$\frac{}{\rho, \texttt{pass} \Rightarrow \mathsf{assigns}(\emptyset)}$$

eval-assign

$$\frac{\rho, e \Rightarrow r}{\rho, x = e \Rightarrow \mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \mathsf{assigns}(\{x : v\})}$$

eval-declare-assign

$$\frac{\rho, e \Rightarrow r}{\rho, x : \psi = e \Rightarrow \mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \mathsf{assigns}(\{x : v\})}$$

eval-declare

$$\frac{}{\rho, x : \psi \Rightarrow \mathsf{assigns}(\emptyset)}$$

eval-expr-stmt

$$\frac{\rho, e \Rightarrow r}{\rho, e \Rightarrow \mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \mathsf{assigns}(\emptyset)}$$

eval-return

$$\frac{\rho, e \Rightarrow r}{\rho, \texttt{return}\ e \Rightarrow \mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \mathsf{returns}\ v}$$

eval-return-none

$$\frac{}{\rho, \texttt{return} \Rightarrow \mathsf{returns}\ \texttt{None}}$$

eval-def

$$\frac{\mathsf{declares}(\vec{d}) = \vec{f} \qquad v_i = \mathsf{Def}_\Gamma(\rho, \vec{d}, i) \quad (\forall i)}{\rho, \vec{d} \Rightarrow \mathsf{assigns}(\{\vec{f} : \vec{v}\})}$$

eval-assert

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True}}{\rho, \texttt{assert}\ e \Rightarrow \mathsf{assigns}(\emptyset)}$$

eval-assert-msg

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True}}{\rho, \texttt{assert}\ e, e' \Rightarrow \mathsf{assigns}(\emptyset)}$$

eval-assert-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False}}{\rho, \texttt{assert}\ e \Rightarrow \mathsf{aborts}\ \mathsf{AssertionError}}$$

eval-assert-msg-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False} \qquad \rho, e' \Rightarrow r}{\rho, \texttt{assert}\ e, e' \Rightarrow \mathsf{let}\ w \leftarrow r\ \mathsf{in}\ \mathsf{aborts}\ \mathsf{AssertionError}(w)}$$

eval-if

$$\frac{\rho, e_j \Rightarrow \mathsf{val}\ \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, s_i \Rightarrow o}{\rho, \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ s_{n+1} \Rightarrow o}$$

eval-if-none

$$\frac{\rho, e_j \Rightarrow \mathsf{val}\ \texttt{False} \quad (\forall j \in \{1, \ldots, n+1\})}{\rho, \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ s_{n+1} \Rightarrow \mathsf{assigns}(\emptyset)}$$

eval-if-else

$$\frac{\rho, e_j \Rightarrow \mathsf{val}\ \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, s_i \Rightarrow o}{\rho, \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ s_{n+1}\ \texttt{else} :\ s_{n+2} \Rightarrow o}$$

eval-else

$$\frac{\rho, e_j \Rightarrow \mathsf{val}\ \texttt{False} \quad (\forall j \in \{1, \ldots, n+1\}) \qquad \rho, s_{n+2} \Rightarrow o}{\rho, \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ s_{n+1}\ \texttt{else} :\ s_{n+2} \Rightarrow o}$$

eval-match

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \mathsf{dispatch}(\rho, v, \vec{p}, \vec{s}) = (\rho', s') \qquad \rho \otimes \rho', s' \Rightarrow o}{\rho, \texttt{match}\ e\texttt{:}\ \texttt{case}\ p_1\texttt{:}\ s_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ s_{n+1} \Rightarrow \mathsf{assigns}(\rho') \otimes o}$$

eval-seq

$$\frac{\rho, s \Rightarrow \mathsf{assigns}(\rho') \qquad \rho \otimes \rho', s' \Rightarrow o}{\rho, s\ s' \Rightarrow \mathsf{assigns}(\rho') \otimes o}$$

Figure 4.3: Statement s evaluates to outcome o under environment $\rho : \Gamma$

$\rho, s \Rightarrow o$ (continued)

eval-assert-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{assert } e \Rightarrow \text{aborts } \kappa}$$

eval-assert-msg-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{assert } e, e' \Rightarrow \text{aborts } \kappa}$$

eval-if-fail

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \text{aborts } \kappa}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \Rightarrow \text{aborts } \kappa}$$

eval-if-else-fail

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \text{aborts } \kappa}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \texttt{ else } : s_{n+2} \Rightarrow \text{aborts } \kappa}$$

eval-match-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{match } e\texttt{: case } p_1\texttt{: } s_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{: } s_{n+1} \Rightarrow \text{aborts } \kappa}$$

eval-seq-fail

$$\frac{\rho, s \Rightarrow \text{aborts } \kappa}{\rho, s\ s' \Rightarrow \text{aborts } \kappa}$$

Figure 4.4: Statement evaluation, continued (aborting cases)

$q, \rho, t \Rightarrow o$

eval-top-stmt

$$\frac{\rho, s \Rightarrow o}{q, \rho, s \Rightarrow o}$$

eval-top-seq

$$\frac{q, \rho, t \Rightarrow \text{assigns}(\rho') \qquad q, \rho \otimes \rho', t' \Rightarrow o}{q, \rho, t\ t' \Rightarrow \text{assigns}(\rho') \otimes o}$$

eval-top-seq-fail

$$\frac{q, \rho, t \Rightarrow \text{aborts } \kappa}{q, \rho, t\ t' \Rightarrow \text{aborts } \kappa}$$

eval-class

$$\frac{}{q, \rho, \texttt{@dataclass class } c[\vec{\alpha}] : \ \phi \Rightarrow \text{assigns}(\{c : \text{Type}(q.c)\})}$$

eval-class-extend

$$\frac{}{q, \rho, \texttt{@dataclass class } c[\vec{\alpha}](q'[\vec{\psi}]) : \ \phi \Rightarrow \text{assigns}(\{c : \text{Type}(q.c)\})}$$

eval-type-alias

$$\frac{\Sigma; \Gamma \otimes \{\alpha_1 : \text{TypeVar}, \ldots, \alpha_n : \text{TypeVar}\} \vdash \psi \rhd \tau}{q, \rho, \texttt{type } x[\vec{\alpha}] = \psi \Rightarrow \text{assigns}(\{x : \text{TypeAlias}(\vec{\alpha}, \tau)\})}$$

Figure 4.5: Top-level statement t of module q evaluates to outcome o under environment $\rho : \Gamma$

4.4 Pattern matching

A `match` statement takes the first case whose pattern matches the value, running its body under the bindings produced by the pattern, and does nothing if no case matches; the auxiliary function dispatch (§A.4) selects the matching case. Whether a pattern matches is determined by the pattern-matching judgement $\rho, p, v \rightsquigarrow m$ (Figure 4.12). A *match result* m is either match ρ', a match, where ρ' binds the variables of the pattern to the values that they matched, or no-match, a failure to match. When a pattern and a value are not comparable, such as a list pattern against a tuple, there is no derivation

$\rho, e \Rightarrow r$

eval-var
$$\frac{x : v \in \rho}{\rho, x \Rightarrow \mathsf{val}\ v}$$

eval-literal
$$\frac{}{\rho, \ell \Rightarrow \mathsf{val}\ \ell}$$

eval-lambda
$$\frac{}{\rho, \texttt{lambda}\ \vec{x} :\ e \Rightarrow \mathsf{val}\ \mathsf{Lam}_\Gamma(\rho, \vec{x}, e)}$$

eval-unop
$$\frac{\rho, e \Rightarrow r}{\rho, \ominus\, e \Rightarrow \mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \hat{\ominus}(v)}$$

eval-binop
$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \rho, e' \Rightarrow r}{\rho, e \odot e' \Rightarrow \mathsf{let}\ v' \leftarrow r\ \mathsf{in}\ \hat{\odot}(v, v')}$$

eval-list
$$\frac{\rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{v}}{\rho, [\vec{e}] \Rightarrow \mathsf{val}\ [\vec{v}]}$$

eval-tuple
$$\frac{\rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{v}}{\rho, (\vec{e}) \Rightarrow \mathsf{val}\ (\vec{v})}$$

eval-dict
$$\frac{\rho, e_1 \cdot e'_1 \cdot \ldots \cdot e_n \cdot e'_n \Rightarrow \mathsf{val}\ (w_1 \cdot v_1 \cdot \ldots \cdot w_n \cdot v_n)}{\rho, \{e_1 : e'_1, \ldots, e_n : e'_n\} \Rightarrow \mathsf{val}\ \{\mathrm{update}(\varepsilon, w_1 : v_1 \cdot \ldots \cdot w_n : v_n)\}}$$

eval-list-comp
$$\frac{\rho, \vec{g} \Rightarrow \mathsf{val}\ \vec{\rho} \qquad \rho_i, e \Rightarrow \mathsf{val}\ v_i \quad (\forall i)}{\rho, [\ e\ \vec{g}] \Rightarrow \mathsf{val}\ [\vec{v}]}$$

eval-dict-comp
$$\frac{\rho, \vec{g} \Rightarrow \mathsf{val}\ \vec{\rho} \qquad \rho_i, e \Rightarrow \mathsf{val}\ w_i \quad (\forall i) \qquad \rho_i, e' \Rightarrow \mathsf{val}\ v_i \quad (\forall i)}{\rho, \{e :\ e'\ \vec{g}\} \Rightarrow \mathsf{val}\ \{\mathrm{update}(\varepsilon, \vec{w} : \vec{v})\}}$$

eval-attr-module
$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Mod}(q', \rho') \qquad x : v \in \rho'}{\rho, e.x \Rightarrow \mathsf{val}\ v}$$

eval-attr-object
$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Obj}(C, \rho')}{\rho, e.x \Rightarrow \mathsf{val}\ \rho'(x)}$$

eval-constr
$$\frac{\rho, e_1 \cdot \ldots \cdot e_{n+m} \Rightarrow \mathsf{val}\ (v_1 \cdot \ldots \cdot v_{n+m}) \qquad \mathrm{resolve}(\rho, q) = \mathsf{Type}(C)}{\rho, q[\vec{\psi}](e_1, \ldots, e_n, y_1{=}e_{n+1}, \ldots, y_m{=}e_{n+m}) \Rightarrow \mathsf{val}\ \mathsf{Obj}(C, \text{field-map}_\Sigma(C, (v_1, \ldots, v_n, y_1{=}v_{n+1}, \ldots, y_m{=}v_{n+m})))}$$

Figure 4.6: Expression e evaluates to result r under environment $\rho : \Gamma$

and the run has no result. Sub-results compose with $\otimes$, matching sequentially:

$$\mathsf{match}\ \rho \otimes \mathsf{match}\ \rho' = \mathsf{match}\ (\rho \uplus \rho')$$
$$m \otimes \mathsf{no\text{-}match} = \mathsf{no\text{-}match} \otimes m = \mathsf{no\text{-}match}$$

with $\mathsf{match}\ \emptyset$ as left and right unit. A constructor pattern matches any object whose class has the pattern's class among its ancestors, and its sub-patterns bind the fields of the pattern's class. Since an object records its class by name, the class test in eval-pat-constr is a comparison of fully-qualified names.

4.5 Modules and programs

Module and program evaluation are given in Figure 4.15; import evaluation in Figure 4.14; importing names from a loaded module in Figure 4.13. Context extension (Figure 2.1) applies to environments as well, with the same equations on values in place of context entries, except that a loaded module value $\mathsf{Mod}(q, \rho)$ carries an environment whereas the context entry $\mathsf{Mod}(q, \Gamma)$ carries a context. *Loading*, the judgement $q \Rightarrow_{\mathsf{load}} r$, evaluates the import prefix of q and then its statement, mirroring static loading $\vdash_{\mathsf{load}} q : \Gamma$ (Figure 3.20); as with the static rules, the mutual recursion between loading and evaluation is grounded by program well-formedness, which rules out import cycles (§2.4). Loading takes a single module; an import loads the modules containing its target from the root down, each before the module

$\rho, e \Rightarrow r$ (continued)

eval-subscript

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \rho, e' \Rightarrow r}{\rho, e[e'] \Rightarrow \mathsf{let}\ v' \leftarrow r\ \mathsf{in}\ \mathrm{getitem}(v, v')}$$

eval-call-lambda

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Lam}_\Delta(\rho', \vec{x}, e') \qquad \rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{u} \qquad \rho' \otimes \{\vec{x} : \vec{u}\}, e' \Rightarrow r}{\rho, e(\vec{e}) \Rightarrow r}$$

eval-call-def

$$\frac{\begin{array}{c}\rho, e \Rightarrow \mathsf{val}\ \mathsf{Def}_\Delta(\rho', \vec{d}, j) \qquad d_j = \texttt{def}\ f[\vec{\alpha}](\vec{x} : \vec{\psi})\ \texttt{->}\ \psi : s \\ v_i = \mathsf{Def}_\Delta(\rho', \vec{d}, i)\quad (\forall i) \qquad \rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{u} \qquad \rho' \otimes \{\mathrm{declares}(\vec{d}) : \vec{v}\} \otimes \{\vec{x} : \vec{u}\} \otimes \mathrm{scope}(\{\vec{x}\}, s), s \Rightarrow o\end{array}}{\rho, e(\vec{e}) \Rightarrow \mathrm{result}(o)}$$

eval-call-prim

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Prim}(f) \qquad \rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{u}}{\rho, e(\vec{e}) \Rightarrow f(\vec{u})}$$

eval-and-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False}}{\rho, e\ \texttt{and}\ e' \Rightarrow \mathsf{val}\ \texttt{False}}$$

eval-and-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, e' \Rightarrow r}{\rho, e\ \texttt{and}\ e' \Rightarrow r}$$

eval-or-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True}}{\rho, e\ \texttt{or}\ e' \Rightarrow \mathsf{val}\ \texttt{True}}$$

eval-or-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False} \qquad \rho, e' \Rightarrow r}{\rho, e\ \texttt{or}\ e' \Rightarrow r}$$

eval-cond-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, e_1 \Rightarrow r}{\rho, e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \Rightarrow r}$$

eval-cond-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False} \qquad \rho, e_2 \Rightarrow r}{\rho, e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \Rightarrow r}$$

Figure 4.7: Expression evaluation, continued

$\rho, \vec{e} \Rightarrow r$

eval-exprs-nil

$$\frac{}{\rho, \varepsilon \Rightarrow \mathsf{val}\ \varepsilon}$$

eval-exprs-cons

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \rho, \vec{e} \Rightarrow r}{\rho, e \cdot \vec{e} \Rightarrow \mathsf{let}\ \vec{v} \leftarrow r\ \mathsf{in}\ \mathsf{val}\ (v \cdot \vec{v})}$$

eval-exprs-fail

$$\frac{\rho, e \Rightarrow \mathsf{aborts}\ \kappa}{\rho, e \cdot \vec{e} \Rightarrow \mathsf{aborts}\ \kappa}$$

Figure 4.8: Expression sequence $\vec{e}$ evaluates left to right to result r under environment $\rho : \Gamma$

it contains, mirroring the static rules, by the loads-as judgement for a plain import and as premises of the from-import rule. If loading one of these modules aborts, the import aborts for the same reason and the modules after it are not loaded.

Since imports precede statements, every module that a statement can reference is loaded before the statement runs, and loading is deterministic, so the semantics needs no cache of loaded modules. The rule eval-module runs a module body in $\mathcal{P}(\texttt{builtins})$ overridden by the environment of its imports, with each name declared in the body or bound by a pattern in it starting unbound, mirroring module (§3.5). A program evaluates to a termination kind (Figure 4.1): SystemExit(0) if the main module loads, and otherwise the kind it aborted with. SystemExit(n) is a run that exited with status n. An implementation must agree with the semantics on the termination kind of every run; the form in which the kind is reported is not prescribed.

$\rho, e \Rightarrow r$ (continued)

eval-binop-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e \odot e' \Rightarrow \text{aborts } \kappa}$$

eval-and-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e \texttt{ and } e' \Rightarrow \text{aborts } \kappa}$$

eval-or-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e \texttt{ or } e' \Rightarrow \text{aborts } \kappa}$$

eval-cond-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e_1 \texttt{ if } e \texttt{ else } e_2 \Rightarrow \text{aborts } \kappa}$$

eval-attr-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e.x \Rightarrow \text{aborts } \kappa}$$

eval-subscript-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e[e'] \Rightarrow \text{aborts } \kappa}$$

eval-call-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e(\vec{e}) \Rightarrow \text{aborts } \kappa}$$

eval-call-fail-args
$$\frac{\rho, e \Rightarrow \text{val } v \qquad \rho, \vec{e} \Rightarrow \text{aborts } \kappa}{\rho, e(\vec{e}) \Rightarrow \text{aborts } \kappa}$$

eval-list-fail
$$\frac{\rho, \vec{e} \Rightarrow \text{aborts } \kappa}{\rho, [\vec{e}] \Rightarrow \text{aborts } \kappa}$$

eval-tuple-fail
$$\frac{\rho, \vec{e} \Rightarrow \text{aborts } \kappa}{\rho, (\vec{e}) \Rightarrow \text{aborts } \kappa}$$

eval-dict-fail
$$\frac{\rho, e_1 \cdot e'_1 \cdot \ldots \cdot e_n \cdot e'_n \Rightarrow \text{aborts } \kappa}{\rho, \{e_1 : e'_1, \ldots, e_n : e'_n\} \Rightarrow \text{aborts } \kappa}$$

eval-list-comp-fail-quals
$$\frac{\rho, \vec{g} \Rightarrow \text{aborts } \kappa}{\rho, [\ e\ \vec{g}] \Rightarrow \text{aborts } \kappa}$$

eval-list-comp-fail
$$\frac{\rho, \vec{g} \Rightarrow \text{val } \vec{\rho} \qquad \rho_i, e \Rightarrow \text{val } v_i \quad (\forall i < j) \qquad \rho_j, e \Rightarrow \text{aborts } \kappa}{\rho, [\ e\ \vec{g}] \Rightarrow \text{aborts } \kappa}$$

eval-dict-comp-fail-quals
$$\frac{\rho, \vec{g} \Rightarrow \text{aborts } \kappa}{\rho, \{e : e'\ \vec{g}\} \Rightarrow \text{aborts } \kappa}$$

eval-dict-comp-fail-key
$$\frac{\rho, \vec{g} \Rightarrow \text{val } \vec{\rho} \qquad \rho_i, e \Rightarrow \text{val } w_i \quad \rho_i, e' \Rightarrow \text{val } v_i \quad (\forall i < j) \qquad \rho_j, e \Rightarrow \text{aborts } \kappa}{\rho, \{e : e'\ \vec{g}\} \Rightarrow \text{aborts } \kappa}$$

eval-dict-comp-fail-value
$$\frac{\rho, \vec{g} \Rightarrow \text{val } \vec{\rho} \qquad \rho_i, e \Rightarrow \text{val } w_i \quad \rho_i, e' \Rightarrow \text{val } v_i \quad (\forall i < j) \qquad \rho_j, e \Rightarrow \text{val } w_j \qquad \rho_j, e' \Rightarrow \text{aborts } \kappa}{\rho, \{e : e'\ \vec{g}\} \Rightarrow \text{aborts } \kappa}$$

eval-constr-fail
$$\frac{\rho, e_1 \cdot \ldots \cdot e_{n+m} \Rightarrow \text{aborts } \kappa}{\rho, q[\vec{\psi}](e_1, \ldots, e_n, y_1{=}e_{n+1}, \ldots, y_m{=}e_{n+m}) \Rightarrow \text{aborts } \kappa}$$

Figure 4.9: Expression evaluation, continued (propagation from subexpressions)

$\rho, e \Rightarrow r$ (continued)

eval-attr-missing
$$\frac{\rho, e \Rightarrow \text{val Obj}(C, \rho') \qquad x \notin \text{dom}(\rho')}{\rho, e.x \Rightarrow \text{aborts AttributeError}}$$

eval-call-lambda-arity
$$\frac{\rho, e \Rightarrow \text{val Lam}_\Delta(\rho', \vec{x}, e') \qquad \rho, \vec{e} \Rightarrow \text{val } \vec{u} \qquad |\vec{x}| \neq |\vec{u}|}{\rho, e(\vec{e}) \Rightarrow \text{aborts TypeError}}$$

eval-call-def-arity
$$\frac{\rho, e \Rightarrow \text{val Def}_\Delta(\rho', \vec{d}, j) \qquad d_j = \texttt{def } f[\vec{\alpha}](\vec{x} : \vec{\psi})\ \texttt{->}\ \psi : s \qquad \rho, \vec{e} \Rightarrow \text{val } \vec{u} \qquad |\vec{x}| \neq |\vec{u}|}{\rho, e(\vec{e}) \Rightarrow \text{aborts TypeError}}$$

eval-call-nonfun
$$\frac{\rho, e \Rightarrow \text{val } v \qquad \rho, \vec{e} \Rightarrow \text{val } \vec{u} \qquad v \text{ neither closure nor primitive}}{\rho, e(\vec{e}) \Rightarrow \text{aborts TypeError}}$$

Figure 4.10: Expression evaluation, continued (aborting cases)

$\rho, \vec{g} \Rightarrow r$

eval-quals-nil

$$\frac{}{\rho, \varepsilon \Rightarrow \mathsf{val}\ \rho}$$

eval-qual-guard-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, \vec{g} \Rightarrow r}{\rho, \texttt{if}\ e \cdot \vec{g} \Rightarrow r}$$

eval-qual-guard-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False}}{\rho, \texttt{if}\ e \cdot \vec{g} \Rightarrow \mathsf{val}\ \varepsilon}$$

eval-qual-guard-fail

$$\frac{\rho, e \Rightarrow \mathsf{aborts}\ \kappa}{\rho, \texttt{if}\ e \cdot \vec{g} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-qual-generator

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \mathrm{iter}(v) = u_1 \cdot \ldots \cdot u_n \qquad \rho \otimes \{x : u_i\}, \vec{g} \Rightarrow \mathsf{val}\ \vec{\rho}_i \quad (\forall i)}{\rho, \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} \Rightarrow \mathsf{val}\ (\vec{\rho}_1 +\!\!+ \ldots +\!\!+ \vec{\rho}_n)}$$

eval-qual-generator-fail

$$\frac{\rho, e \Rightarrow \mathsf{aborts}\ \kappa}{\rho, \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-qual-generator-nonseq

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \mathrm{iter}(v)\ \text{undefined}}{\rho, \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} \Rightarrow \mathsf{aborts}\ \mathsf{TypeError}}$$

eval-qual-generator-fail-body

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \mathrm{iter}(v) = u_1 \cdot \ldots \cdot u_n \qquad \rho \otimes \{x : u_i\}, \vec{g} \Rightarrow \mathsf{val}\ \vec{\rho}_i \quad (\forall i < j) \qquad \rho \otimes \{x : u_j\}, \vec{g} \Rightarrow \mathsf{aborts}\ \kappa}{\rho, \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} \Rightarrow \mathsf{aborts}\ \kappa}$$

Figure 4.11: Comprehension qualifiers $\vec{g}$ evaluate to result r under environment $\rho : \Gamma$

$\rho, p, v \rightsquigarrow m$

eval-pat-lit
$$\frac{\mathrm{eq}(\ell, v) = \texttt{True}}{\rho, \ell, v \rightsquigarrow \text{match } \emptyset}$$

eval-pat-lit-no
$$\frac{\mathrm{eq}(\ell, v) = \texttt{False}}{\rho, \ell, v \rightsquigarrow \text{no-match}}$$

eval-pat-var
$$\frac{}{\rho, x, v \rightsquigarrow \text{match } \{x : v\}}$$

eval-pat-wild
$$\frac{}{\rho, _, v \rightsquigarrow \text{match } \emptyset}$$

eval-pat-list
$$\frac{\rho, p_i, v_i \rightsquigarrow m_i \quad (\forall i)}{\rho, [p_1, \ldots, p_n], [v_1, \ldots, v_n] \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-list-no
$$\frac{v \text{ neither list nor tuple, or list of length} \neq n}{\rho, [p_1, \ldots, p_n], v \rightsquigarrow \text{no-match}}$$

eval-pat-tuple
$$\frac{\rho, p_i, v_i \rightsquigarrow m_i \quad (\forall i)}{\rho, (p_1, \ldots, p_n), (v_1, \ldots, v_n) \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-tuple-no
$$\frac{v \text{ neither list nor tuple, or tuple of length} \neq n}{\rho, (p_1, \ldots, p_n), v \rightsquigarrow \text{no-match}}$$

eval-pat-dict
$$\frac{w_i : v_i \in \delta \quad (\forall i) \qquad \rho, p_i, v_i \rightsquigarrow m_i \quad (\forall i)}{\rho, \{w_1 : p_1, \ldots, w_n : p_n\}, \delta \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-dict-no
$$\frac{v \text{ not dictionary, or } \delta \text{ with } w_i \notin \mathrm{dom}(\delta) \text{ for some } i}{\rho, \{w_1 : p_1, \ldots, w_n : p_n\}, v \rightsquigarrow \text{no-match}}$$

eval-pat-as
$$\frac{\rho, p, v \rightsquigarrow m}{\rho, p \texttt{ as } x, v \rightsquigarrow m \otimes \text{match } \{x : v\}}$$

eval-pat-constr
$$\frac{\mathrm{resolve}(\rho, q) = \mathsf{Type}(C) \qquad C \in \mathrm{ancestors}_\Sigma(D) \qquad \mathrm{fields}_\Sigma(C) = \forall \vec{\alpha}.\, \vec{x} : \vec{\sigma} \qquad \rho, \text{field-map}_\Sigma(C, \vec{p})(x_i), \rho'(x_i) \rightsquigarrow m_i \quad (\forall i)}{\rho, q(\vec{p}), \mathsf{Obj}(D, \rho') \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-constr-no
$$\frac{\mathrm{resolve}(\rho, q) = \mathsf{Type}(C) \qquad v \text{ not object, or } \mathsf{Obj}(D, \rho') \text{ with } C \notin \mathrm{ancestors}_\Sigma(D)}{\rho, q(\vec{p}), v \rightsquigarrow \text{no-match}}$$

Figure 4.12: Pattern p matches value v under environment ρ, with result m

ρ, x imports r

eval-imp-member
$$\frac{x : v \in \rho \qquad v \neq \mathsf{Mod}(q') \quad (\forall q')}{\rho, x \text{ imports val } v}$$

eval-imp-submodule
$$\frac{x : \mathsf{Mod}(q') \in \rho \qquad q' \Rightarrow_{\mathsf{load}} \text{val } \rho'}{\rho, x \text{ imports val } \mathsf{Mod}(q', \rho')}$$

eval-imp-submodule-fail
$$\frac{x : \mathsf{Mod}(q') \in \rho \qquad q' \Rightarrow_{\mathsf{load}} \text{aborts } \kappa}{\rho, x \text{ imports aborts } \kappa}$$

Figure 4.13: Name x imported from module environment ρ has result r

$q, \vec{\imath} \Rightarrow r$

eval-imports-nil

$$\frac{}{q, \varepsilon \Rightarrow \mathsf{val}\ \emptyset}$$

eval-import

$$\frac{q' = x \cdot \vec{x} \qquad x \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad \mathsf{Mod}(x, \rho), \vec{x}\ \mathsf{loads\text{-}as}\ \mathsf{val}\ v \qquad q, \vec{\imath} \Rightarrow r}{q, \texttt{import}\ q' \cdot \vec{\imath} \Rightarrow \mathsf{let}\ \rho' \leftarrow r\ \mathsf{in}\ \mathsf{val}\ (\{x : v\} \boxplus \rho')}$$

eval-import-fail

$$\frac{q' = x \cdot \vec{x} \qquad x \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{q, \texttt{import}\ q' \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-import-fail-chain

$$\frac{q' = x \cdot \vec{x} \qquad x \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad \mathsf{Mod}(x, \rho), \vec{x}\ \mathsf{loads\text{-}as}\ \mathsf{aborts}\ \kappa}{q, \texttt{import}\ q' \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-from-import

$$\frac{\vec{q}^{\dagger} = \mathrm{containing}(q, q') \qquad q_j^{\dagger} \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho_j \quad (\forall j) \qquad q' \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad \rho, x_i\ \mathsf{imports}\ \mathsf{val}\ v_i \quad (\forall i) \qquad q, \vec{\imath} \Rightarrow r}{q, \texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{let}\ \rho^{\dagger} \leftarrow r\ \mathsf{in}\ \mathsf{val}\ (\{x_1 : v_1, \ldots, x_{n+1} : v_{n+1}\} \boxplus \rho^{\dagger})}$$

eval-from-import-fail

$$\frac{\vec{q}^{\dagger} = \mathrm{containing}(q, q') \qquad q_j^{\dagger} \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho_j \quad (\forall j) \qquad q' \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{q, \texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-from-import-fail-containing

$$\frac{\vec{q}^{\dagger} = \mathrm{containing}(q, q') \qquad q_j^{\dagger} \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa \qquad q_k^{\dagger} \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho_k \quad (\forall k < j)}{q, \texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-from-import-fail-member

$$\frac{\vec{q}^{\dagger} = \mathrm{containing}(q, q') \qquad q_k^{\dagger} \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho_k \quad (\forall k) \qquad q' \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad \rho, x_i\ \mathsf{imports}\ \mathsf{val}\ v_i \quad (\forall i < j) \qquad \rho, x_j\ \mathsf{imports}\ \mathsf{aborts}\ \kappa}{q, \texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

(a) Import prefix $\vec{\imath}$ of module q evaluates to result r

$v, \vec{x}\ \mathsf{loads\text{-}as}\ r$

eval-loads-as-nil

$$\frac{}{v, \varepsilon\ \mathsf{loads\text{-}as}\ \mathsf{val}\ v}$$

eval-loads-as-cons

$$\frac{q.x \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho' \qquad \mathsf{Mod}(q.x, \rho'), \vec{x}\ \mathsf{loads\text{-}as}\ r}{\mathsf{Mod}(q, \rho), x \cdot \vec{x}\ \mathsf{loads\text{-}as}\ \mathsf{let}\ v \leftarrow r\ \mathsf{in}\ \mathsf{val}\ \mathsf{Mod}(q, \rho \boxplus \{x : v\})}$$

eval-loads-as-fail

$$\frac{q.x \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{\mathsf{Mod}(q, \rho), x \cdot \vec{x}\ \mathsf{loads\text{-}as}\ \mathsf{aborts}\ \kappa}$$

(b) Loaded module v with submodule chain $\vec{x}$ loads to result r

Figure 4.14: Import evaluation

$q \Rightarrow_{\mathsf{load}} r$

eval-predefined

$$\frac{\mathcal{P}(q) = (\Gamma, \rho)}{q \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho}$$

eval-module

$$\frac{\mathcal{P}(\texttt{builtins}) = (\Gamma, \rho) \qquad q, \vec{\imath} \Rightarrow \mathsf{val}\ \rho' \qquad \mathcal{M}(q) = \vec{\imath}\ t \qquad q, \rho \otimes \rho' \otimes \mathrm{scope}(\{\mathrm{declares}(\vec{\imath})\}, t), t \Rightarrow \mathsf{assigns}(\rho^{\dagger})}{q \Rightarrow_{\mathsf{load}} \mathsf{val}\ (\mathrm{submods}(q) \otimes \rho^{\dagger})}$$

eval-module-fail

$$\frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad q, \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}{q \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}$$

eval-module-fail-body

$$\frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad \mathcal{P}(\texttt{builtins}) = (\Gamma, \rho) \qquad q, \vec{\imath} \Rightarrow \mathsf{val}\ \rho' \qquad q, \rho \otimes \rho' \otimes \mathrm{scope}(\{\mathrm{declares}(\vec{\imath})\}, t), t \Rightarrow \mathsf{aborts}\ \kappa}{q \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}$$

$\mathcal{M} \Rightarrow \kappa$

eval-program

$$\frac{\texttt{__main__} \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho}{\mathcal{M} \Rightarrow \mathsf{SystemExit}(0)}$$

eval-program-abort

$$\frac{\texttt{__main__} \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{\mathcal{M} \Rightarrow \kappa}$$

Figure 4.15: Module and program evaluation

Annex A

Auxiliary functions (normative)

This annex is normative. It defines the auxiliary functions used by the well-formedness rules (Chapter 3) and the operational semantics (Chapter 4). When more than one case applies, the first defines the value, so a function with overlapping cases is still determined: two equivalent types join to the second of them, for instance.

A.1 Syntactic analyses

The syntactic analyses introduced in Chapter 3 are defined by their principal cases; the remaining cases are stated with each.

assigns(t)	*set of variables*

Variables declared or assigned anywhere in t, not descending into nested functions, which have their own scope. It is empty on the statements not shown and is the union over the branches of a conditional, and on a module body $\vec{\imath}\ t$ acts on t.

$$\begin{aligned}
\text{assigns}(x\colon\ \psi) = \text{assigns}(x = e) = \text{assigns}(x\colon\ \psi = e) &= \{x\} \\
\text{assigns}(t\ \ t') &= \text{assigns}(t) \cup \text{assigns}(t') \\
\text{assigns}(\vec{d}) &= \{\vec{f}\} \quad \text{where } d_i = \texttt{def}\ f_i[\vec{\alpha}_i](\vec{x}_i\colon\ \vec{\psi}_i)\ \texttt{->}\ \psi_i\colon\ s_i \\
\text{assigns}(\texttt{match}\ e\colon\ \texttt{case}\ p_1\colon\ s_1\ \ \ldots\ \ \texttt{case}\ p_{n+1}\colon\ s_{n+1}) &= \bigcup_i \left(\text{binds}(p_i) \cup \text{assigns}(s_i)\right) \\
\text{assigns}(\texttt{@dataclass class}\ c[\vec{\alpha}]\ :\ \ \phi) &= \{c\} \\
\text{assigns}(\texttt{@dataclass class}\ c[\vec{\alpha}](q[\vec{\psi}])\ :\ \ \phi) &= \{c\} \\
\text{assigns}(\texttt{type}\ x[\vec{\alpha}] = \psi) &= \{x\}
\end{aligned}$$

binds(p), binds(s)	*set of variables*

Variables introduced by pattern p, by the generators of qualifier sequence $\vec{g}$, or by the patterns within statement s, not descending into nested functions. On the cases not shown it recurses structurally, taking the union over the immediate subpatterns, and on a statement is the union over the branches of

a conditional and empty on the statements not shown.

$$\begin{aligned}
\text{binds}(x) &= \{x\} \\
\text{binds}(p \texttt{ as } x) &= \text{binds}(p) \cup \{x\} \\
\text{binds}(\varepsilon) &= \emptyset \\
\text{binds}(\texttt{if } e \cdot \vec{g}) &= \text{binds}(\vec{g}) \\
\text{binds}(\texttt{for } x \texttt{ in } e \cdot \vec{g}) &= \{x\} \cup \text{binds}(\vec{g}) \\
\text{binds}(\texttt{match } e\texttt{: case } p_1\texttt{: } s_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{: } s_{n+1}) &= \textstyle\bigcup_i \left(\text{binds}(p_i) \cup \text{binds}(s_i)\right) \\
\text{binds}(s\ s') &= \text{binds}(s) \cup \text{binds}(s')
\end{aligned}$$

declares(t), declares($\vec{\imath}$) *sequence of variables*

Names declared anywhere in t, or bound by the imports of prefix $\vec{\imath}$, in textual order and with repeats, not descending into nested functions. It is empty on the statements not shown and on the empty prefix, and is the concatenation over the branches of a conditional and the cases of a match.

$$\begin{aligned}
\text{declares}(x\texttt{: } \psi) = \text{declares}(x\texttt{: } \psi = e) &= x \\
\text{declares}(t\ t') &= \text{declares}(t) +\!\!+ \text{declares}(t') \\
\text{declares}(\vec{d}) &= \vec{f} \\
&\quad \text{where } d_i = \texttt{def } f_i[\vec{\alpha}_i](\vec{x}_i\texttt{: } \vec{\psi}_i)\ \texttt{->}\ \psi_i\texttt{: } s_i \\
\text{declares}(\texttt{@dataclass class } c[\vec{\alpha}]\ \texttt{:}\ \ \phi) &= c \\
\text{declares}(\texttt{@dataclass class } c[\vec{\alpha}](q[\vec{\psi}])\ \texttt{:}\ \ \phi) &= c \\
\text{declares}(\texttt{type } x[\vec{\alpha}] = \psi) &= x \\
\text{declares}(\texttt{import } q \cdot \vec{\imath}) &= x \cdot \text{declares}(\vec{\imath}) \quad \text{where } q = x \cdot \vec{x} \\
\text{declares}(\texttt{from } q \texttt{ import } x_1, \ldots, x_{n+1} \cdot \vec{\imath}) &= x_1 \cdot \cdots \cdot x_{n+1} \cdot \text{declares}(\vec{\imath})
\end{aligned}$$

scope(Y, s) *context*

Context for the body s of a function with parameters Y, or of a module with imported names Y, in which each variable declared or assigned in s is unbound and the names in Y keep their bindings. Undefined if s declares a name twice or declares a name in Y, or if a pattern in s binds a name declared in s or in Y. The context serves as an environment as well, Unbound being a value as well as a context entry.

$$\begin{aligned}
\text{scope}(Y, s) = \{x\texttt{: } \mathsf{Unbound} \mid x \in \text{assigns}(s) \setminus Y\} \\
\text{if declares}(s) \text{ distinct and } Y, \{\text{declares}(s)\}, \text{binds}(s) \text{ pairwise disjoint}
\end{aligned}$$

captures(e) *set of variables*

Variables of the enclosing scope captured by lambdas within expression e; the generator variables of a comprehension are local to it. On the cases not shown it recurses structurally, taking the union over

the immediate subexpressions.

$$\begin{aligned}\text{captures}(\texttt{lambda}\ \vec{x}\colon\ e) &= \text{fv}(e) \setminus \{\vec{x}\}\\ \text{captures}([\ e\ \vec{g}\,]) &= \text{captures}(\vec{g}) \cup (\text{captures}(e) \setminus \text{binds}(\vec{g}))\\ \text{captures}(\{e\colon\ e'\ \vec{g}\}) &= \text{captures}(\vec{g}) \cup (\text{captures}((e, e')) \setminus \text{binds}(\vec{g}))\end{aligned}$$

$\text{captures}(\vec{g})$	*set of variables*

Variables of the enclosing scope captured by lambdas within the qualifier sequence $\vec{g}$ of a comprehension.

$$\begin{aligned}\text{captures}(\varepsilon) &= \emptyset\\ \text{captures}(\texttt{if}\ e \cdot \vec{g}) &= \text{captures}(e) \cup \text{captures}(\vec{g})\\ \text{captures}(\texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g}) &= \text{captures}(e) \cup (\text{captures}(\vec{g}) \setminus \{x\})\end{aligned}$$

$\text{fv}(e), \text{fv}(\vec{e})$	*set of variables*

Free variables of expression e, defined in the usual way, and of a sequence of expressions $\vec{e}$, the union over its elements.

A.2 Types

The functions on types and shapes introduced in §3.1, each defined by its cases.

$\text{base-type}(\ell), \text{base-type}(\tau)$	*type*

Base type of literal ℓ, and of type τ that type with a literal type replaced by the base type of its literal.

$$\begin{aligned}\text{base-type}(n) &= \texttt{int} && \text{if } n \text{ is an integer literal}\\ \text{base-type}(n) &= \texttt{float} && \text{if } n \text{ is a float literal}\\ \text{base-type}(w) &= \texttt{str}\\ \text{base-type}(\texttt{True}) = \text{base-type}(\texttt{False}) &= \texttt{bool}\\ \text{base-type}(\texttt{None}) &= \texttt{None}\\ \\ \text{base-type}(\texttt{Literal}[\ell]) &= \text{base-type}(\ell)\\ \text{base-type}(\tau) &= \tau && \text{otherwise}\end{aligned}$$

$\sigma \vee_\Sigma \tau, \bigvee_\Sigma \vec{\tau}$	*type*

Larger of two comparable types, and their union otherwise. A sequence joins its first type with the

join of the rest, with `Never` as the unit.

$$\begin{aligned}
\sigma \vee_\Sigma \tau &= \tau && \text{if } \Sigma \vdash \sigma <: \tau \\
\sigma \vee_\Sigma \tau &= \sigma && \text{if } \Sigma \vdash \tau <: \sigma \text{ and } \Sigma \nvdash \sigma \equiv \tau \\
\sigma \vee_\Sigma \tau &= \sigma \mid \tau && \text{otherwise}
\end{aligned}$$

$$\begin{aligned}
\bigvee_\Sigma \varepsilon &= \texttt{Never} \\
\bigvee_\Sigma \sigma \cdot \vec{\tau} &= \sigma \vee_\Sigma \bigvee_\Sigma \vec{\tau}
\end{aligned}$$

$\sigma \wedge_\Sigma \tau$ *type*

Cases after the first two apply only when σ and τ are incomparable.

$$\begin{aligned}
\sigma \wedge_\Sigma \tau &= \sigma && \text{if } \Sigma \vdash \sigma <: \tau \\
\sigma \wedge_\Sigma \tau &= \tau && \text{if } \Sigma \vdash \tau <: \sigma \text{ and } \Sigma \nvdash \sigma \equiv \tau \\
\sigma \mid \sigma' \wedge_\Sigma \tau &= \sigma \wedge_\Sigma \tau \vee_\Sigma \sigma' \wedge_\Sigma \tau \\
\sigma \wedge_\Sigma \tau \mid \tau' &= \sigma \wedge_\Sigma \tau \vee_\Sigma \sigma \wedge_\Sigma \tau' && \text{if } \sigma \text{ not union} \\
\texttt{Callable}[[\vec{\sigma}], \sigma] \wedge_\Sigma \texttt{Callable}[[\vec{\tau}], \tau] &= \texttt{Callable}[[\sigma_1 \vee_\Sigma \tau_1, \dots, \sigma_n \vee_\Sigma \tau_n], \sigma \wedge_\Sigma \tau] \\
\texttt{tuple}[\vec{\sigma}] \wedge_\Sigma \texttt{tuple}[\vec{\tau}] &= \texttt{tuple}[\sigma_1 \wedge_\Sigma \tau_1, \dots, \sigma_n \wedge_\Sigma \tau_n] \\
\sigma \wedge_\Sigma \tau &= \texttt{Never} && \text{otherwise}
\end{aligned}$$

$\text{subterms}(\tau), \text{subterms}(\vec{\tau})$ *set of types*

Subterms of τ other than unions, including τ itself unless it is a union, and for a sequence the subterms of each of its types. The type arguments of the instantiation candidates of Lemma 3.4 are joins of these.

$$\begin{aligned}
\text{subterms}(\sigma \mid \tau) &= \text{subterms}(\sigma) \cup \text{subterms}(\tau) \\
\text{subterms}(\texttt{list}[\tau]) &= \{\texttt{list}[\tau]\} \cup \text{subterms}(\tau) \\
\text{subterms}(\texttt{dict}[\texttt{str}, \tau]) &= \{\texttt{dict}[\texttt{str}, \tau]\} \cup \text{subterms}(\tau) \\
\text{subterms}(\texttt{tuple}[\vec{\tau}]) &= \{\texttt{tuple}[\vec{\tau}]\} \cup \text{subterms}(\vec{\tau}) \\
\text{subterms}(\texttt{Callable}[[\vec{\tau}], \tau]) &= \{\texttt{Callable}[[\vec{\tau}], \tau]\} \cup \text{subterms}(\vec{\tau}) \cup \text{subterms}(\tau) \\
\text{subterms}(C[\vec{\tau}]) &= \{C[\vec{\tau}]\} \cup \text{subterms}(\vec{\tau}) \\
\text{subterms}(\tau) &= \{\tau\} && \text{otherwise}
\end{aligned}$$

$$\text{subterms}(\vec{\tau}) = \text{subterms}(\tau_1) \cup \cdots \cup \text{subterms}(\tau_n)$$

$\text{type-args}_\Sigma(\sigma, \tau), \text{type-args}_\Sigma(\vec{\sigma}, \vec{\tau})$ *context binding type variables at types*

Arguments for the type variables of σ, found by one-way unification of σ with τ and given as a context binding type variables at types. A type variable takes the base type of τ; a union τ contributes by each disjunct; lists, dictionaries, tuples and callables are taken componentwise; an instantiated class is taken through the instance of the class at τ (Figure 3.14a); and a union σ contributes nothing when τ is below one of its disjuncts, as `None` is below `T | None`, and otherwise contributes by each disjunct. In every other case, in particular for a type without type variables, the context is empty. Positions are

covariant (tuple components and `Callable` results), contravariant (`Callable` parameters) or invariant (list, dictionary and class arguments), following the subtyping rules (Figure 3.3). The type argument for a type variable must accordingly be a supertype, a subtype or an equivalent of the type found, which the argument check of the call rule enforces once the type arguments are chosen. Contexts combine by pointwise join, written $\vee_\Sigma$ as for types.

$$
\begin{aligned}
\text{type-args}_\Sigma(\alpha, \tau) &= \{\alpha : \text{base-type}(\tau)\} \\
\text{type-args}_\Sigma(\sigma, \tau \mid \tau') &= \text{type-args}_\Sigma(\sigma, \tau) \vee_\Sigma \text{type-args}_\Sigma(\sigma, \tau') \\
\text{type-args}_\Sigma(\texttt{list}[\sigma], \texttt{list}[\tau]) &= \text{type-args}_\Sigma(\sigma, \tau) \\
\text{type-args}_\Sigma(\texttt{dict}[\texttt{str}, \sigma], \texttt{dict}[\texttt{str}, \tau]) &= \text{type-args}_\Sigma(\sigma, \tau) \\
\text{type-args}_\Sigma(\texttt{tuple}[\vec{\sigma}], \texttt{tuple}[\vec{\tau}]) &= \text{type-args}_\Sigma(\vec{\sigma}, \vec{\tau}) \\
\text{type-args}_\Sigma(\texttt{Callable}[[\vec{\sigma}], \sigma], \texttt{Callable}[[\vec{\tau}], \tau]) &= \text{type-args}_\Sigma(\sigma \cdot \vec{\sigma}, \tau \cdot \vec{\tau}) \\
\text{type-args}_\Sigma(C[\vec{\sigma}], \tau) &= \text{type-args}_\Sigma(\vec{\sigma}, \vec{\tau}') \\
&\qquad \text{if } \Sigma \vdash C[\vec{\tau}'] \asymp \tau \\
\text{type-args}_\Sigma(\sigma \mid \sigma', \tau) &= \emptyset \\
&\qquad \text{if } \Sigma \vdash \tau <: \sigma \text{ or } \Sigma \vdash \tau <: \sigma' \\
\text{type-args}_\Sigma(\sigma \mid \sigma', \tau) &= \text{type-args}_\Sigma(\sigma, \tau) \vee_\Sigma \text{type-args}_\Sigma(\sigma', \tau) \quad \text{(otherwise)} \\
\text{type-args}_\Sigma(\sigma, \tau) &= \emptyset \quad \text{(otherwise)}
\end{aligned}
$$

$$
\begin{aligned}
\text{type-args}_\Sigma(\varepsilon, \varepsilon) &= \emptyset \\
\text{type-args}_\Sigma(\sigma \cdot \vec{\sigma}, \tau \cdot \vec{\tau}) &= \text{type-args}_\Sigma(\sigma, \tau) \vee_\Sigma \text{type-args}_\Sigma(\vec{\sigma}, \vec{\tau})
\end{aligned}
$$

$$
\begin{aligned}
(\Gamma \vee_\Sigma \Gamma')(\alpha) &= \Gamma(\alpha) \vee_\Sigma \Gamma'(\alpha) && \text{if } \alpha \in \text{dom}(\Gamma) \cap \text{dom}(\Gamma') \\
(\Gamma \vee_\Sigma \Gamma')(\alpha) &= \Gamma(\alpha) && \text{if } \alpha \in \text{dom}(\Gamma) \setminus \text{dom}(\Gamma') \\
(\Gamma \vee_\Sigma \Gamma')(\alpha) &= \Gamma'(\alpha) && \text{if } \alpha \in \text{dom}(\Gamma') \setminus \text{dom}(\Gamma)
\end{aligned}
$$

$\text{elem-type}_\Sigma(\tau)$ — *type*

Type of the elements drawn by a generator from a list, string, dictionary, tuple or range type or a union of these, mirroring iter (§A.4). The element type of a tuple is the join of the base types of its components. Undefined at any other type.

$$
\begin{aligned}
\text{elem-type}_\Sigma(\texttt{list}[\tau]) &= \tau \\
\text{elem-type}_\Sigma(\texttt{str}) &= \texttt{str} \\
\text{elem-type}_\Sigma(\texttt{Literal}[w]) &= \texttt{str} \\
\text{elem-type}_\Sigma(\texttt{dict}[\texttt{str}, \tau]) &= \texttt{str} \\
\text{elem-type}_\Sigma(\texttt{builtins.range}) &= \texttt{int} \\
\text{elem-type}_\Sigma(\texttt{tuple}[\vec{\tau}]) &= \bigvee\nolimits_\Sigma \text{base-type}(\tau_1), \ldots, \text{base-type}(\tau_n) \\
\text{elem-type}_\Sigma(\sigma \mid \tau) &= \text{elem-type}_\Sigma(\sigma) \vee_\Sigma \text{elem-type}_\Sigma(\tau)
\end{aligned}
$$

$\text{shapes}_\Sigma(\tau, H), \text{shapes}_\Sigma(\vec{\tau})$ — *ordered set of shapes or of shape sequences*

Shapes of type τ that remain once the heads in H are excluded. A union has no head to exclude, so

the disjuncts of the union are taken separately, each with the heads of H typed at that disjunct. A dictionary is refined by its keys rather than by a head, so a dictionary type gives the dictionary form with no key bound. Every other type gives the rest form $\tau \setminus H$, except that no shape remains when H exhausts τ: a literal type whose literal is in H, `bool` when both `True` and `False` are in H, `None` when `None` is in H, an instantiated class with an ancestor in H, and `Never`. A sequence of types yields the sequences of shapes of its components.

$$
\begin{aligned}
\text{shapes}_\Sigma(\sigma \mid \tau, H) &= \text{shapes}_\Sigma(\sigma, H_\sigma) \cup \text{shapes}_\Sigma(\tau, H_\tau) \\
&\qquad \text{where } H_\sigma = \{h \in H \mid \Sigma \vdash h : \sigma\} \text{ and } H_\tau = \{h \in H \mid \Sigma \vdash h : \tau\} \\
\text{shapes}_\Sigma(\texttt{Literal}[\ell], H) &= \emptyset && \text{if } \ell \in H \\
\text{shapes}_\Sigma(\texttt{bool}, H) &= \emptyset && \text{if } \{\texttt{True}, \texttt{False}\} \subseteq H \\
\text{shapes}_\Sigma(\texttt{None}, H) &= \emptyset && \text{if } \texttt{None} \in H \\
\text{shapes}_\Sigma(C[\vec{\tau}], H) &= \emptyset && \text{if } D \in \text{ancestors}_\Sigma(C) \text{ for some } D \in H \\
\text{shapes}_\Sigma(\texttt{Never}, H) &= \emptyset \\
\text{shapes}_\Sigma(\texttt{dict}[\texttt{str}, \tau], H) &= \{\emptyset \setminus_\tau H\} \\
\text{shapes}_\Sigma(\tau, H) &= \{\tau \setminus H\} && \text{otherwise} \\
\text{shapes}_\Sigma(\vec{\tau}) &= \text{shapes}_\Sigma(\tau_1, \emptyset) \times \cdots \times \text{shapes}_\Sigma(\tau_n, \emptyset)
\end{aligned}
$$

shape-type(k) *type*

Least type of shape k, with $\Sigma \vdash k : \text{shape-type}(k)$ (Figure 3.15): the class of a constructor, componentwise for a tuple, and the type recorded by the form otherwise.

$$
\begin{aligned}
\text{shape-type}(\tau \setminus H) &= \tau \\
\text{shape-type}(C(\vec{k}) \setminus H) &= C \\
\text{shape-type}((\vec{k})) &= \texttt{tuple}[\text{shape-type}(k_1), \ldots, \text{shape-type}(k_n)] \\
\text{shape-type}([\vec{k}]_\tau) &= \texttt{list}[\tau] \\
\text{shape-type}(\beta \setminus_\tau H) &= \texttt{dict}[\texttt{str}, \tau]
\end{aligned}
$$

matched-as$_\Sigma(\Gamma, \tau, p)$ *type*

Type to which τ is narrowed if pattern p matches: an over-approximation of the matched values, as a type. Variable and wildcard patterns leave τ unchanged; literal patterns narrow to their literal types; tuple patterns narrow each component; list and dictionary patterns narrow to the list or dictionary type; constructor patterns narrow to the instance of their class at τ, or to τ itself when τ is below the instance; unions are narrowed disjunct by disjunct. Any other pattern does not match any value of τ, and narrows to `Never`. The function emulates the narrowing of a match scrutinee observed in mypy

within a case, and gives the types of patterns named by `as` (check-as).

$$
\begin{aligned}
\text{matched-as}_\Sigma(\Gamma, \sigma \mid \tau, p) &= \text{matched-as}_\Sigma(\Gamma, \sigma, p) \vee_\Sigma \text{matched-as}_\Sigma(\Gamma, \tau, p) \\
\text{matched-as}_\Sigma(\Gamma, \tau, x) &= \tau \\
\text{matched-as}_\Sigma(\Gamma, \tau, _) &= \tau \\
\text{matched-as}_\Sigma(\Gamma, \tau, p \texttt{ as } x) &= \text{matched-as}_\Sigma(\Gamma, \tau, p) \\
\text{matched-as}_\Sigma(\Gamma, \tau, \ell) &= \texttt{Literal}[\ell] \wedge_\Sigma \tau \\
\text{matched-as}_\Sigma(\Gamma, \texttt{tuple}[\tau_1, \ldots, \tau_n], (p_1, \ldots, p_n)) &= \texttt{tuple}[\sigma_1, \ldots, \sigma_n] \\
&\qquad \text{where } \sigma_i = \text{matched-as}_\Sigma(\Gamma, \tau_i, p_i) \quad (\forall i) \\
\text{matched-as}_\Sigma(\Gamma, \texttt{list}[\tau], [\vec{p}]) &= \texttt{list}[\tau] \\
\text{matched-as}_\Sigma(\Gamma, \texttt{dict}[\texttt{str}, \tau], \{\vec{w} \colon \vec{p}\}) &= \texttt{dict}[\texttt{str}, \tau] \\
\text{matched-as}_\Sigma(\Gamma, \tau, q(a)) &= \tau \wedge_\Sigma C[\vec{\sigma}] \\
&\qquad \text{if } \Gamma \vdash q : \mathsf{Type}(C) \text{ and } \Sigma \vdash C[\vec{\sigma}] \asymp \tau \\
\text{matched-as}_\Sigma(\Gamma, \tau, p) &= \texttt{Never} \quad \text{(otherwise)}
\end{aligned}
$$

$\text{remaining}_\Sigma(\Gamma, \tau, p)$	*type*

Type remaining from τ if pattern p fails to match: an over-approximation of the unmatched values, as a type rather than a set of shapes. Nothing remains after a variable or wildcard pattern, after a literal pattern at its own literal type or at `None`, after the empty dictionary pattern at a dictionary type, after a tuple pattern each of whose sub-patterns leaves nothing, or after a constructor pattern at a type below the instance of its class each of whose sub-patterns leaves nothing at the field types. The other Boolean literal remains after a literal pattern at `bool`; tuple patterns with exactly one sub-pattern that leaves something narrow that component; unions are narrowed disjunct by disjunct; and any other pattern leaves τ unchanged. The function emulates the narrowing of a match scrutinee observed in mypy, so that a match which is exhaustive by the remaining type is also exhaustive under mypy. The remaining type is coarser than the residual (§3.4), since constructor patterns with a refutable sub-pattern, and

tuple patterns with two refutable components, leave the type untouched.

$$
\begin{aligned}
\text{remaining}_\Sigma(\Gamma, \sigma \mid \tau, p) &= \text{remaining}_\Sigma(\Gamma, \sigma, p) \vee_\Sigma \text{remaining}_\Sigma(\Gamma, \tau, p) \\
\text{remaining}_\Sigma(\Gamma, \tau, x) &= \texttt{Never} \\
\text{remaining}_\Sigma(\Gamma, \tau, _) &= \texttt{Never} \\
\text{remaining}_\Sigma(\Gamma, \tau, p \texttt{ as } x) &= \text{remaining}_\Sigma(\Gamma, \tau, p) \\
\text{remaining}_\Sigma(\Gamma, \texttt{Literal}[\ell], \ell) &= \texttt{Never} \\
\text{remaining}_\Sigma(\Gamma, \texttt{bool}, \texttt{True}) &= \texttt{Literal}[\texttt{False}] \\
\text{remaining}_\Sigma(\Gamma, \texttt{bool}, \texttt{False}) &= \texttt{Literal}[\texttt{True}] \\
\text{remaining}_\Sigma(\Gamma, \texttt{None}, \texttt{None}) &= \texttt{Never} \\
\text{remaining}_\Sigma(\Gamma, \texttt{tuple}[\vec{\tau}], (\vec{p})) &= \texttt{Never} \\
&\qquad \text{if } \text{remaining}_\Sigma(\Gamma, \tau_i, p_i) = \texttt{Never} \quad (\forall i) \\
\text{remaining}_\Sigma(\Gamma, \texttt{tuple}[\vec{\tau}], (\vec{p})) &= \texttt{tuple}[\tau_1, \ldots, \tau_{i-1}, \text{remaining}_\Sigma(\Gamma, \tau_i, p_i), \tau_{i+1}, \ldots, \tau_n] \\
&\qquad \text{if } \text{remaining}_\Sigma(\Gamma, \tau_j, p_j) = \texttt{Never} \quad (\forall j \neq i) \\
\text{remaining}_\Sigma(\Gamma, \texttt{dict}[\texttt{str}, \tau], \{\}) &= \texttt{Never} \\
\text{remaining}_\Sigma(\Gamma, \tau, q(a)) &= \texttt{Never} \\
&\qquad \text{if } \Gamma \vdash q : \mathsf{Type}(C), \Sigma \vdash C[\vec{\sigma}] \asymp \tau, \Sigma \vdash \tau <: C[\vec{\sigma}], \\
&\qquad \text{fields}_\Sigma(C)[\vec{\sigma}] = \vec{x} : \vec{\sigma}', \text{field-map}_\Sigma(C, a) = \{\vec{x} : \vec{p}\} \\
&\qquad \text{and } \text{remaining}_\Sigma(\Gamma, \sigma'_i, p_i) = \texttt{Never} \quad (\forall i) \\
\text{remaining}_\Sigma(\Gamma, \tau, p) &= \tau
\end{aligned}
$$

$\text{resolve-op}_\Sigma(\odot, \sigma_1, \sigma_2), \text{resolve-op}_\Sigma(\ominus, \sigma)$	*type*

Result type of the most specific overload of an operator at the operand types. The overloads of $\odot$ at (σ_1, σ_2) are the resolved overloads $((\tau_1, \tau_2), \tau)$ with $\Sigma \vdash \odot(\sigma_1, \sigma_2) : ((\tau_1, \tau_2), \tau)$ (Figure 3.9), ordered by componentwise subtyping in Σ on their bounds (τ_1, τ_2); the overloads of $\ominus$ at σ are defined in the same way from conclusions $\Sigma \vdash \ominus(\sigma) : (\tau, \tau')$. The most specific overload is the least element under this order, so the function is undefined when no least element exists. A least element exists when some overload applies and neither operand type is below `Never` (Lemma 3.3).

$$
\begin{aligned}
\text{resolve-op}_\Sigma(\odot, \sigma_1, \sigma_2) &= \tau && \text{if } ((\tau_1, \tau_2), \tau) \text{ is the most specific overload of } \odot \text{ at } (\sigma_1, \sigma_2) \\
\text{resolve-op}_\Sigma(\ominus, \sigma) &= \tau' && \text{if } (\tau, \tau') \text{ is the most specific overload of } \ominus \text{ at } \sigma
\end{aligned}
$$

A.3 Classes and modules

The functions on module names, field lists and classes introduced in §2.4, and the resolution of names in an environment.

$\text{submods}(q)$	*context*

Context that binds each submodule of q to a stub: a context entry $x : \mathsf{Mod}(q.x)$ for each module $q.x$ of

the program or of the predefined modules.

$$\text{submods}(q) = \{x : \mathsf{Mod}(q.x) \mid q.x \in \text{dom}(\mathcal{M}) \cup \text{dom}(\mathcal{P})\}$$

containing(q, q') *sequence of module names*

Containing modules of q', excluding q and the modules containing q, in prefix order. A from-import of q' in module q loads these as well as q' itself.

$$\text{containing}(q, q') = q_1^\dagger \cdot \cdots \cdot q_m^\dagger \qquad \text{where } \{q_1^\dagger, \ldots, q_m^\dagger\} = \{q^\dagger \prec q' \mid q^\dagger \not\preceq q\},\ q_1^\dagger \prec \cdots \prec q_m^\dagger$$

own-fields(ϕ) *sequence of bindings*

Fields declared by a field list, with their types, in order.

$$\begin{aligned}\text{own-fields}(x_1 : \psi_1\ \ldots\ x_{n+1} : \psi_{n+1}) &= x_1 : \psi_1, \ldots, x_{n+1} : \psi_{n+1}\\ \text{own-fields}(\texttt{pass}) &= \varepsilon\end{aligned}$$

fields$_\Sigma(C)$ *field scheme*

Fields of class C with their declared types, as a field scheme over the type parameters of C: the fields of its base class, if any, instantiated at the type arguments of the base, followed by the fields declared by C, distinct from the inherited fields by class-extend (Figure 3.13). The fields of an instantiated class $C[\vec{\tau}]$ are the instantiation $\text{fields}_\Sigma(C)[\vec{\tau}]$.

$$\begin{aligned}\text{fields}_\Sigma(C) &= \forall\vec{\alpha}.\,\vec{x} : \vec{\tau} && \text{if } \Sigma(C) = (\vec{\alpha};\ \vec{x} : \vec{\tau};\ \bot)\\ \text{fields}_\Sigma(C) &= \forall\vec{\alpha}.\,(\text{fields}_\Sigma(D)[\vec{\sigma}] +\!\!+ \vec{x} : \vec{\tau}) && \text{if } \Sigma(C) = (\vec{\alpha};\ \vec{x} : \vec{\tau};\ D[\vec{\sigma}])\end{aligned}$$

ancestors$_\Sigma(C)$ *set of classes*

Class C together with all its base classes.

$$\begin{aligned}\text{ancestors}_\Sigma(C) &= \{C\} && \text{if } \Sigma(C) = (\vec{\alpha};\ \vec{x} : \vec{\tau};\ \bot)\\ \text{ancestors}_\Sigma(C) &= \{C\} \cup \text{ancestors}_\Sigma(D) && \text{if } \Sigma(C) = (\vec{\alpha};\ \vec{x} : \vec{\tau};\ D[\vec{\sigma}])\end{aligned}$$

field-map$_\Sigma(C, \vec{u})$ *map from variables*

Argument supplied for each field of C: positional arguments fill the fields in order and keyword arguments match by name. Defined if and only if the arguments cover the fields exactly, so constructor patterns and expressions check and evaluate field by field without regard to the split of the arguments between positional and keyword.

$$\begin{aligned}\text{field-map}_\Sigma(C, (u_1, \ldots, u_n, y_1{=}u_{n+1}, \ldots, y_m{=}u_{n+m})) = \{x_1 : u_1, \ldots, x_n : u_n, y_1 : u_{n+1}, \ldots, y_m : u_{n+m}\}\\ \text{if } \text{fields}_\Sigma(C) = \forall\vec{\alpha}.\, x_1 : \sigma_1 \cdot \ldots \cdot x_n : \sigma_n \cdot \vec{y}\,' : \vec{\sigma}\,'\\ \text{and } \{\vec{y}\} = \{\vec{y}\,'\}\end{aligned}$$

resolve(ρ, q)	*value*

Value denoted by name q in ρ, mirroring static judgement $\Gamma \vdash q : \theta$ (Figure 3.1): a simple name is looked up in ρ directly, and a qualified name in the environment of the loaded module denoted by its prefix. Undefined if the prefix denotes a module stub or a value other than a module.

$$\begin{aligned} \text{resolve}(\rho, x) &= \rho(x) \\ \text{resolve}(\rho, q.x) &= \rho'(x) \qquad \text{where resolve}(\rho, q) = \mathsf{Mod}(q', \rho') \end{aligned}$$

name(q)	*string*

String literal naming module q, so that name(`__main__`) = `"__main__"`.

A.4 Values

The functions on values used by the evaluation rules. The comparisons `==`, `!=`, `in` and `not in` are given in terms of structural equality eq and membership contains. When defined, the operators agree with Python, yielding aborts κ when Python raises an exception. When undefined, evaluation of the expression has no derivation, and an implementation may abort or produce the result that Python gives (Chapter 1). Because equality is partial, the order in which elements are compared is observable: equality and membership short-circuit at the first element that decides the result, so an undefined comparison later in a sequence does not make the whole comparison undefined.

$\hat{\odot}(v, v')$	*result*

Meaning of a strict binary operator.

$\odot$	$\hat{\odot}(v, v')$
`==`	val `False` if both operands are `nan`, and val eq(v, v') otherwise
`!=`	val `True` if both operands are `nan` or eq(v, v') = `False`, and val `False` if eq(v, v') = `True`
`in`	val contains(v', v)
`not in`	val `True` if contains(v', v) = `False`, and val `False` if contains(v', v) = `True`
`<` `<=` `>` `>=`	[**TODO:** Ordering, as in Python]
`+` `-` `*` `/` `//` `%` `**`	[**TODO:** Arithmetic, as in Python. Float precision is left to the implementation. aborts ZeroDivisionError where the divisor is zero, and aborts TypeError where an operand is not a number]

$\hat{\ominus}(v)$	*result*

Meaning of a unary operator.

$\ominus$	$\hat{\ominus}(v)$
`not`	val `True` if v = `False`, and val `False` if v = `True`
`+`	val v if v is a number, and aborts TypeError otherwise
`-`	val $-v$ if v is a number, and aborts TypeError otherwise

$\mathrm{eq}(v, v')$	*Boolean*

Structural equality: literals by value, lists and tuples by their elements, dictionaries with the same keys by their values, and objects of the same class by their field values, each through eq-elems, except ranges, which are compared by their sequences, so that `range(-1) == range(-2)`. The forms of values are those of Figure 4.1, with literals distinguished as numbers, strings, Booleans and `None`, and closures and primitives taken together as functions. Values of different forms are unequal, except a Boolean and a number, for which equality is undefined because Python compares them numerically. Equality is also undefined between functions and between two `nan` values, which Python compares by identity. Dictionaries are compared in the entry order of the left operand.

$$\begin{aligned}
\mathrm{eq}(\ell, \ell) &= \texttt{True} && \text{if } \ell \text{ is not } \texttt{nan} \\
\mathrm{eq}(n, n') &= \texttt{False} && \text{if } n \neq n' \\
\mathrm{eq}(w, w') &= \texttt{False} && \text{if } w \neq w' \\
\mathrm{eq}(\texttt{None}, v) &= \texttt{False} && \text{if } v \neq \texttt{None} \\
\mathrm{eq}(v, \texttt{None}) &= \texttt{False} && \text{if } v \neq \texttt{None} \\
\mathrm{eq}(\texttt{True}, \texttt{False}) &= \texttt{False} \\
\mathrm{eq}(\texttt{False}, \texttt{True}) &= \texttt{False}
\end{aligned}$$

$$\begin{aligned}
\mathrm{eq}([\vec{v}], [\vec{v}']) &= \text{eq-elems}(\vec{v}, \vec{v}') \\
\mathrm{eq}((\vec{v}), (\vec{v}')) &= \text{eq-elems}(\vec{v}, \vec{v}')
\end{aligned}$$

$$\begin{aligned}
\mathrm{eq}(\delta, \delta') &= \text{eq-elems}(v_1 \cdot \ldots \cdot v_n, \delta'(w_1) \cdot \ldots \cdot \delta'(w_n)) \\
&\qquad \text{if } \mathrm{dom}(\delta) = \mathrm{dom}(\delta'), \text{ where } \delta = w_1 : v_1 \cdot \ldots \cdot w_n : v_n \\
\mathrm{eq}(\delta, \delta') &= \texttt{False} \\
&\qquad \text{if } \mathrm{dom}(\delta) \neq \mathrm{dom}(\delta')
\end{aligned}$$

$$\begin{aligned}
\mathrm{eq}(v, v') &= \text{eq-elems}(\mathrm{iter}(v), \mathrm{iter}(v')) \\
&\qquad \text{if } v \text{ and } v' \text{ are values of } \texttt{builtins.range} \\
\mathrm{eq}(\mathsf{Obj}(C, \rho), \mathsf{Obj}(C, \rho')) &= \text{eq-elems}(\rho(x_1) \cdot \ldots \cdot \rho(x_n), \rho'(x_1) \cdot \ldots \cdot \rho'(x_n)) \\
&\qquad \text{if } \mathrm{fields}_\Sigma(C) = \forall \vec{\alpha}.\, \vec{x} : \vec{\sigma} \text{ and } C \neq \texttt{builtins.range} \\
\mathrm{eq}(\mathsf{Obj}(C, \rho), \mathsf{Obj}(D, \rho')) &= \texttt{False} && \text{if } C \neq D
\end{aligned}$$

$$\begin{aligned}
\mathrm{eq}(v, v') &= \texttt{False} \\
&\qquad \text{if } v \text{ and } v' \text{ are of different forms and are not a Boolean and a number}
\end{aligned}$$

$\text{eq-elems}(\vec{v}, \vec{v}')$	*Boolean*

Equality of two sequences of values, compared from the first element: `False` at the first unequal pair

or if the lengths differ, and `True` when every pair is equal.

$$\begin{aligned}
\text{eq-elems}(\varepsilon, \varepsilon) &= \texttt{True} && \\
\text{eq-elems}(v \cdot \vec{v}, v' \cdot \vec{v}\,') &= \texttt{False} && \text{if } \text{eq}(v, v') = \texttt{False} \\
\text{eq-elems}(v \cdot \vec{v}, v' \cdot \vec{v}\,') &= \text{eq-elems}(\vec{v}, \vec{v}\,') && \text{if } \text{eq}(v, v') = \texttt{True} \\
\text{eq-elems}(\vec{v}, \vec{v}\,') &= \texttt{False} && \text{if } |\vec{v}| \neq |\vec{v}\,'|
\end{aligned}$$

contains(v, v') *Boolean*

Whether v' is an element of v: of a list or tuple by structural equality, of a dictionary as a key, and of a string as a substring.

$$\begin{aligned}
\text{contains}([\vec{v}], v) &= \text{contains-elems}(\vec{v}, v) \\
\text{contains}((\vec{v}), v) &= \text{contains-elems}(\vec{v}, v)
\end{aligned}$$

$$\begin{aligned}
\text{contains}(\delta, w) &= \texttt{True} && \text{if } w \in \text{dom}(\delta) \\
\text{contains}(\delta, w) &= \texttt{False} && \text{if } w \notin \text{dom}(\delta)
\end{aligned}$$

$$\begin{aligned}
\text{contains}(w', w) &= \texttt{True} && \text{if } w' = w_1 \mathbin{+\!\!+} w \mathbin{+\!\!+} w_2 \text{ for some } w_1, w_2 \\
\text{contains}(w', w) &= \texttt{False} && \text{otherwise}
\end{aligned}$$

contains-elems($\vec{v}, v$) *Boolean*

Whether a sequence of values has an element structurally equal to v, taking the elements in order and stopping at the first element equal to v.

$$\begin{aligned}
\text{contains-elems}(\varepsilon, v) &= \texttt{False} && \\
\text{contains-elems}(v' \cdot \vec{v}, v) &= \texttt{True} && \text{if } \text{eq}(v, v') = \texttt{True} \\
\text{contains-elems}(v' \cdot \vec{v}, v) &= \text{contains-elems}(\vec{v}, v) && \text{if } \text{eq}(v, v') = \texttt{False}
\end{aligned}$$

elems(v) *sequence of values*

Elements of a list or tuple, or the characters of a string, in order.

$$\begin{aligned}
\text{elems}([\vec{v}]) &= \vec{v} \\
\text{elems}((\vec{v})) &= \vec{v} \\
\text{elems}(w) &= w_1 \cdot \ldots \cdot w_m && \text{where } w = w_1 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} w_m, \text{ each } w_i \text{ one character}
\end{aligned}$$

iter(v) *sequence of values*

Sequence drawn from v by a generator: the keys when v is a dictionary, the numbers $0, \ldots, n-1$ when

v is a range with stop n, and otherwise the elements of v.

$$\begin{aligned}
\text{iter}(\delta) &= w_1 \cdot \ldots \cdot w_n && \text{where } \delta = w_1 : v_1 \cdot \ldots \cdot w_n : v_n \\
\text{iter}(\text{Obj}(\texttt{builtins.range}, \{\texttt{stop}: n\})) &= 0 \cdot \ldots \cdot n-1 && \text{empty if } n \le 0 \\
\text{iter}(v) &= \text{elems}(v) && \text{otherwise}
\end{aligned}$$

getitem(v, v') *result*

Subscripting: the value under key v' of a dictionary, or the element at index v' of a sequence, counting from the end when the index is negative. Aborts with KeyError, IndexError or TypeError when Python raises the exception of that name.

$$\begin{aligned}
\text{getitem}(\delta, w) &= \text{val } \delta(w) && \text{if } w \in \text{dom}(\delta) \\
\text{getitem}(\delta, w) &= \text{aborts KeyError} && \text{if } w \notin \text{dom}(\delta) \\
\text{getitem}(v, n) &= \text{val } v_{n+1} && \text{if elems}(v) = v_1 \cdot \ldots \cdot v_m \text{ and } 0 \le n < m \\
\text{getitem}(v, n) &= \text{val } v_{m+n+1} && \text{if elems}(v) = v_1 \cdot \ldots \cdot v_m \text{ and } -m \le n < 0 \\
\text{getitem}(v, n) &= \text{aborts IndexError} && \text{if elems}(v) = v_1 \cdot \ldots \cdot v_m \text{ and } (n < -m \text{ or } n \ge m) \\
\text{getitem}(v, v') &= \text{aborts TypeError} && \text{otherwise; } v \text{ not a dictionary}
\end{aligned}$$

result(o) *result*

Result of a call from the outcome of its body: the value returned, or `None` if the body completes without returning, or the abort if the body aborts.

$$\begin{aligned}
\text{result}(\text{returns } v) &= \text{val } v \\
\text{result}(\text{assigns}(\rho)) &= \text{val } \texttt{None} \\
\text{result}(\text{aborts } \kappa) &= \text{aborts } \kappa
\end{aligned}$$

dispatch$(\rho, v, \vec{p}, \vec{s})$ *bindings and statement*

Case taken by a `match` statement for value v: the bindings and statement of the first pattern that matches, or no bindings and `pass` if no pattern matches.

$$\begin{aligned}
\text{dispatch}(\rho, v, \varepsilon, \varepsilon) &= (\varnothing, \texttt{pass}) \\
\text{dispatch}(\rho, v, p \cdot \vec{p}, s \cdot \vec{s}) &= (\rho', s) && \text{if } \rho, p, v \rightsquigarrow \text{match } \rho' \\
\text{dispatch}(\rho, v, p \cdot \vec{p}, s \cdot \vec{s}) &= \text{dispatch}(\rho, v, \vec{p}, \vec{s}) && \text{if } \rho, p, v \rightsquigarrow \text{no-match}
\end{aligned}$$

update(δ, δ') *dictionary entries*

Dictionary entries δ updated by the bindings of δ' in order, each replacing any earlier entry under the same key.

$$\begin{aligned}
\text{update}(\delta, \varepsilon) &= \delta \\
\text{update}(\delta, w : v \cdot \delta') &= \text{update}(\text{setitem}(\delta, w, v), \delta')
\end{aligned}$$

setitem(δ, w, v)	*dictionary entries*

Dictionary entries δ with w bound to v, replacing any earlier binding of w.

$$\begin{aligned}
\text{setitem}(\varepsilon, w, v) &= w : v \\
\text{setitem}(w : u \cdot \delta, w, v) &= w : v \cdot \delta \\
\text{setitem}(w' : u \cdot \delta, w, v) &= w' : u \cdot \text{setitem}(\delta, w, v) && \text{if } w \neq w'
\end{aligned}$$

Annex B

Excluded Python behaviours (informative)

This annex is informative and imposes no requirements. It summarises the Python behaviours excluded by PurePy and the stage at which each exclusion takes effect.

B.1 Mutable state

Figure B.1 lists the Python constructs associated with mutation and shared state, and says how and when PurePy excludes each.

Construct	Python	Exclusion	Mechanism
`global` and `nonlocal`	Direct assignment to variable of enclosing scope	Syntactic	Not in grammar
`del`	Removes binding	Syntactic	Not in grammar
Augmented assignment `+=`, etc.	Updates in place if type supports it	Syntactic	Not in grammar
Walrus operator `:=`	Assigns within expression	Syntactic	Not in grammar
Item assignment `xs[0] = v`	Updates container in place	Syntactic	Not in grammar; assignment targets are variables
Attribute assignment `p.x = v`	Updates object in place	Syntactic	Not in grammar; assignment targets are variables
Mutating methods, such as `list.append`	Update value in place	Static	Only objects and modules have attributes
Reassignment	Changes value of variable	Static	Variable declared once and assigned at most once on any path
Lambdas capturing comprehension variable	Variable mutated on each iteration; lambda sees final value	Static	Lambda may not capture generator variable

Figure B.1: Python mutation of state excluded by PurePy

B.2 Implicit coercions

Python accepts several implicit coercions and lax matches. Figure B.2 lists those excluded by PurePy, and records at which stage each exclusion takes effect. A dynamically excluded operation leaves the run without a result, so the program runs under Python but not under PurePy.

B.3 Compatibility with mypy

Some rules reject programs that could be accepted without loss of soundness, only so that a well-typed program is also well typed under mypy (Chapter 3). Figure B.3 lists each restriction with mypy's

Construct	Python	Exclusion	Mechanism
Non-Boolean test of `if` or `assert`	Coerced to `bool`	Static	Test checked against `bool` (if, assert)
List pattern at tuple, or tuple pattern at list	Matches, since list and tuple patterns both match lists and tuples	Static	Pattern does not match any value; case unreachable (check-list-none, check-tuple-none)
Pattern `a, b` without brackets or parentheses	Matches lists and tuples alike	Syntactic	Tuple patterns parenthesised, list patterns bracketed
Positional class pattern with fewer sub-patterns than fields	Matches, ignoring remaining fields	Static	Arguments cover fields exactly (field-map$_\Sigma$)
Non-string dictionary key	Any hashable value accepted, with `1`, `1.0` and `True` equivalent as keys	Syntactic in patterns, static in expressions	Keys are string literals in patterns, of type `str` in expressions (dict)
Non-string `assert` message	Any object accepted, shown in traceback	Static	Message checked against `str` (assert-msg)
== or != between operands of unrelated types	Defined, giving `False`	Static	Operand types must be comparable (binop)
Boolean against number under == or !=	Compared as numbers, so `True` equals `1`	Static; dynamic at `object`	`bool` and `int` incomparable (binop); at `object`, equality undefined
Function under == or !=	Compared by identity, so two textually identical lambdas are unequal	Static; dynamic at `object`	Callable types not equality types (binop); at `object`, undefined between functions
`nan` under container equality	Elements compared by identity before equality, so identical NaNs match but distinct ones do not	Dynamic	Undefined when both elements are `nan`

Figure B.2: Python coercions and lax matches excluded by PurePy

verdict and the rule that enforces it.

B.4 Runtime errors

A well-formed PurePy program avoids the Python runtime errors listed in Figure B.4, under the conditions noted there. Well-formedness does not prevent errors from ill-typed operations, such as wrong operand types, calling a non-function, calling a function with the wrong number of arguments, or attribute access on an object lacking the attribute. Each of these has a termination kind and a rule in the semantics (Figure 4.10).

Construct	mypy	Mechanism
Call returning `None` as an expression	Rejects any use of the value	call requires a result other than `None`; the call is a statement (call-stmt)
Body that may fall through at a return type such as `int | None`	Reports a missing return	def-body requires outcome Returns unless the return type is equivalent to `None`
Bare `return` at such a return type	Reports a missing return value	return-none requires the return type equivalent to `None`
Match whose cases cover a class field by field	Narrows the scrutinee by whole classes, so reports a missing return	Exhaustiveness decided by the remaining type (match)
Pattern variable after a case excluding a field value	Binds at the field type	Bindings taken from the remaining type (match-case, check-var)
`list[bool]` below `list[int]`	Rejects; list and dictionary types invariant	subty-list and subty-dict require equivalent components
Declaration of a name already a parameter, imported name, pattern variable or declaration of the scope	Rejects the redeclaration	Context scope(Y, s) of the body undefined unless the names are pairwise distinct

Figure B.3: Restrictions imposed for compatibility with mypy

Python error	Ruled out when	See
`NameError`, `UnboundLocalError`	Always: a variable may be referenced only where it is definitely assigned.	Figure 3.10, Figure 3.4
`ModuleNotFoundError`	Always: an import requires its target to be a module of the program.	Figure 3.19
`ImportError`	Always: a from-import requires each imported name to be a member of the source module, and import cycles are rejected. (A Python cycle may run or fail, depending on where the cycle is entered.)	Figure 3.18, Figure 3.20
`AttributeError`	The reference is a module attribute, which must resolve to a definitely assigned member.	Figure 3.4
`TypeError`	The operation is a constructor call or class pattern, which must match the arity of the class exactly. (Only excess sub-patterns are a Python error; a pattern with fewer sub-patterns is excluded even though Python permits it.)	Figure 3.4, Figure 3.16

Figure B.4: Python errors prevented by well-formedness

Bibliography

[1] Apple Inc. *The Swift Programming Language*. Apple, 2025. URL https://docs.swift.org/swift-book/documentation/the-swift-programming-language/.

[2] Rowan Davies and Frank Pfenning. Intersection types and computational effects. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 198–208, 2000. doi: 10.1145/351240.351259.

[3] Jana Dunfield and Neel Krishnaswami. Bidirectional typing. *ACM Computing Surveys*, 54(5):1–38, 2021. doi: 10.1145/3450952.

[4] Jana Dunfield and Frank Pfenning. Tridirectional typechecking. In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '04)*, pages 281–292, 2004. doi: 10.1145/964001.964025.

[5] James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. *The Java Language Specification, Java SE 24 Edition*. Oracle, 2025. URL https://docs.oracle.com/javase/specs/jls/se24/html/index.html.

[6] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: A minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems*, 23(3): 396–450, May 2001. ISSN 0164-0925, 1558-4593. doi: 10.1145/503502.503505.

[7] JetBrains. *Kotlin Language Documentation*. JetBrains, 2025. URL https://kotlinlang.org/docs/home.html.

[8] Georgios Karachalias, Tom Schrijvers, Dimitrios Vytiniotis, and Simon Peyton Jones. GADTs meet their match: Pattern-matching warnings that account for GADTs, guards, and laziness. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming*, pages 424–436. ACM, 2015. doi: 10.1145/2784731.2784748.

[9] Luc Maranget. Warnings for pattern matching. *Journal of Functional Programming*, 17(3):387–421, 2007. doi: 10.1017/S0956796807006223.

[10] Martin Odersky, Philippe Altherr, Vincent Cremet, Iulian Dragos, Gilles Dubochet, Burak Emir, Sean McDirmid, Stéphane Micheloud, Nikolay Mihaylov, Michel Schinz, Erik Stenman, Lex Spoon, and Matthias Zenger. *The Scala Language Specification, Version 2.13*. EPFL, 2025. URL https://scala-lang.org/files/archive/spec/2.13/.

[11] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[12] Peter Sestoft. ML pattern match compilation and partial evaluation. In *Partial Evaluation*, volume 1110 of *Lecture Notes in Computer Science*, pages 446–464. Springer, 1996. doi: 10.1007/3-540-61580-6_23.

[13] W. G. van Hoorn and B. van Rootselaar. Fundamental notions in the theory of seminearrings. *Compositio Mathematica*, 18(1-2):65–78, 1967. URL https://www.numdam.org/item/CM_1967__18_1-2_65_0/.